



CCA – COMPETENCE CENTRE

HTL Anichstraße

DIPLOMARBEIT

3D SCANNER

Höhere Technische Bundeslehr- und Versuchsanstalt Anichstraße

**Abteilung
ELEKTRONIK UND TECHNISCHE INFORMATIK**

Ausgeführt im Schuljahr 2025/26 von:

Alexander Schiechl 5aHEL
Felix Granegger 5aHEL

Betreuer/Betreuerin:

Dipl.-Ing. (FH) Klaus Bichler
Engelbert Gruber

Projektpartner:

LEITNER AG

Ansprechperson: Dipl.-Ing. (FH) Matthias Gogl

Innsbruck, am 27.03.2026

Abgabevermerk:

Datum:

Betreuer/in:

Kurzfassung

Die effiziente Erfassung von Artikelstammdaten wie Maßen, Gewicht und visuellen Merkmalen stellt im neuen Lager der *Leitner AG* in Sterzing eine logistische Herausforderung dar. Derzeitige manuelle Prozesse sind zeitintensiv, während kommerzielle Datenerfassungssysteme (z. B. Cubiscan) für die reine Lagerplatzvergabe oft zu kostenintensiv und überdimensioniert sind.

Ziel dieser Diplomarbeit ist die Entwicklung und Konstruktion eines mobilen 3D-Scannersystems, das speziell auf die Bedürfnisse der Lagerlogistik zugeschnitten ist. Das entwickelte System auf einem mobilen Arbeitsplatz kombiniert Sensorik zur automatisierten Erfassung von Gewicht und Abmessungen (Länge, Breite, Höhe) mit Kameras für Produktfotos sowie Scannern für EAN- und interne Materialnummern. Ein zentraler Bestandteil ist die Software-Schnittstelle, die eine intuitive Bedienung ermöglicht.

Der Fokus der Entwicklung lag auf Kosteneffizienz und Prozessgeschwindigkeit. Das Ergebnis ist ein funktionsfähiger Prototyp, der die Datenerfassung pro Artikel auf unter 30 Sekunden verkürzt. Damit steht der *Leitner AG* eine wirtschaftliche und praxisnahe Lösung zur Verfügung, um den Arbeitsaufwand bei der Einlagerung signifikant zu minimieren.

Abstract

The efficient capture of product master data – such as dimensions, weight, and visual characteristics – presents a logistical challenge at Leitner AG's new warehouse in Vipiteno. Current manual processes are time-consuming, while commercial data capture systems (e.g., Cubiscan) are often too costly and over-engineered for the sole purpose of assigning storage locations.

The goal of this thesis is to develop and design a mobile 3D scanning system specifically tailored to the needs of warehouse logistics. The system, built on a mobile workstation, combines sensors for the automated capture of weight and dimensions (length, width, height) with cameras for product photos and scanners for EAN and internal material numbers. A central component is the software interface, which enables intuitive operation.

The development focused on cost efficiency and process speed. The result is a functional prototype that reduces data capture per item to under 30 seconds. This provides *Leitner AG* with a cost-effective and practical solution to significantly minimize the workload associated with warehousing.

Gendern

Aus Gründen der besseren Lesbarkeit wird in dieser Diplomarbeit die Sprachform des generischen Maskulinums angewendet. Es wird darauf hingewiesen, dass die ausschließliche Verwendung der männlichen Form geschlechtsunabhängig verstanden werden soll.

Verwendung der Künstlichen Intelligenz

Die Erstellung dieser Diplomarbeit wurde durch den Einsatz von Künstlicher Intelligenz (KI) gezielt unterstützt, insbesondere in den Bereichen Grammatik, Rechtschreibung und Zeichensetzung. Mithilfe KI-gestützter Sprachverarbeitung konnten Fehler identifiziert und korrigiert werden, wodurch die sprachliche Qualität der Arbeit verbessert wurde. Diese Unterstützung diente dazu, die formale Korrektheit des Textes zu gewährleisten, ohne dabei inhaltliche oder analytische Aspekte der Arbeit zu beeinflussen.

Inhaltsverzeichnis

1	Einleitung.....	1
1.1	Vertiefende Aufgabenstellung Schiechtl (Hardware).....	3
1.2	Vertiefende Aufgabenstellung Granegger (Software).....	4
2	Dokumentation Schiechtl Alexander	5
2.1	Einleitung.....	5
2.1.1	Elektrische Systemkomponenten	5
2.1.2	Mechanische Konstruktion	6
2.1.3	Elektrisches Blockschaltbild	6
2.2	Theoretische Grundlagen	7
2.2.1	Steuerung	7
2.2.2	Räumliche Vermessung	9
2.2.3	Wägetechnik.....	11
2.2.4	Beleuchtung.....	13
2.2.5	Verkabelung	14
2.2.6	Schnittstellen	17
2.2.7	Gehäusetechnik.....	20
2.2.8	Verwendung von Literatur, Internetquellen und KI	21
2.3	Praktische Umsetzung.....	22
2.3.1	Projektplanung.....	22
2.3.2	Komponentenauswahl	24
2.3.3	Ansteuerung der Beleuchtung.....	32
2.3.4	Schaltpläne und Fertigungsunterlagen	35
2.3.5	Gehäuseentwicklung	41
2.3.6	Gehäusemontage	49
2.4	Weitere Individuelle Arbeitsschritte	52
2.5	Zusammenfassung individueller Teil	53
3	Dokumentation Granegger Felix	54
3.1	Einleitung.....	54
3.2	Theoretische Grundlagen	55
3.2.1	Tiefenmessung mit Stereo-Vision	56
3.2.2	Bildverarbeitung und Barcode-Dekodierung	62
3.2.3	Gewichtserfassung.....	65
3.2.4	Datenmanagement.....	66
3.2.5	User Interface für die Datenverarbeitung	67
3.2.6	Verwendung von Literatur, Internetquellen und KI	70
3.3	Praktische Umsetzung.....	71

3.3.1	Tiefenmessung mit Stereo-Vision	72
3.3.2	Bildverarbeitung und Barcode-Dekodierung	77
3.3.3	Gewichtserfassung	83
3.3.4	Beleuchtung	87
3.3.5	Datenmanagement	88
3.3.6	Raspberry Pi – Installation und Systemeinrichtung	90
3.3.7	User Interface	94
3.4	Zusammenfassung individueller Teil	98
4	Zusammenfassung und Ausblick	100
4.1	Ergebnisse und Diskussion	100
4.1.1	Messung 1	100
4.1.2	Messung 2	101
4.1.3	Messung 3	102
4.1.4	Messung 4	103
4.1.5	Diskussion	104
4.2	Zusammenfassung der gesamten Arbeit	105
4.3	Ausblick	107
4.3.1	Hardwareseitig	107
4.3.2	Softwareseitig	108
5	Erklärung der Eigenständigkeit	109
6	Arbeitsnachweise	110
6.1	Schiechtl Alexander	110
6.2	Granegger Felix	114
7	Verzeichnisse	116
7.1	Quellenverzeichnis	116
7.2	Abkürzungsverzeichnis	120
7.3	Abbildungen	121
8	Anhang	123

1 Einleitung

In modernen Logistikzentren bildet die exakte Kenntnis über Artikelstammdaten – wie Gewicht, Abmessungen und visuelle Beschaffenheit – die Basis für eine effiziente Lagerplatzvergabe und Prozessoptimierung. Im neu errichteten Lager der Leitner AG in Sterzing erfolgt die Erfassung dieser Daten derzeit manuell. Dieser Vorgang bindet wertvolle Personalressourcen und ist anfällig für Übertragungsfehler.

Marktübliche Automatisierungslösungen, wie beispielsweise Systeme der Firma Cubiscan, bieten zwar eine hohe technologische Reife, stehen jedoch in keinem wirtschaftlichen Verhältnis zum spezifischen Anforderungszweck der reinen Lagerplatzvergabe. Es besteht daher der Bedarf an einer maßgeschneiderten Lösung, die Mobilität, Kosteneffizienz und eine nahtlose Integration in die bestehende IT-Infrastruktur vereint.

Im Rahmen dieser Diplomarbeit wird ein mobiles 3D-Scannersystem entwickelt, das als intuitiver Arbeitsplatz für die Lagermitarbeiter fungiert. Das technische Kernziel ist die automatisierte Erfassung von Gewichten mittels Wägezellen sowie die visuelle und dimensionale Erfassung durch ein Multi-Kamera-Setup, inklusive Tiefenmessung. Ein wesentlicher Bestandteil der Arbeit ist zudem die softwareseitige Speicherung der Messdaten, um eine fehlerfreie Übergabe an weiterführende Datenbanksysteme zu gewährleisten.

Das Projekt strebt an, die Datenerfassung pro Artikel auf maximal 30 Sekunden zu beschleunigen und dabei die Materialkosten auf einem Niveau von ca. 1.000 € zu halten, um eine valide Alternative zu teuren Industriesystemen zu bieten.

Aufbau der Arbeit:

Die vorliegende Diplomarbeit gliedert sich in fünf zentrale Phasen. Nach der Analyse der Anforderungen und der Recherche geeigneter Hardware-Komponenten im ersten Teil, beschreibt der Hauptteil die technische Entwicklung und Vernetzung der Sensorik mit der Steuerungseinheit. Darauf aufbauend werden die Konstruktion des Gehäuses sowie die Implementierung der Software-Schnittstellen erläutert. Abschließend erfolgt die Validierung des Prototyps anhand von Testmessungen im Realbetrieb.

1.1 Vertiefende Aufgabenstellung Schiechtl (Hardware)

Die technische Umsetzung beginnt mit der Evaluation und Auswahl geeigneter Hardwarekomponenten. In dieser Phase liegt der Fokus auf der Sicherstellung der Systemkompatibilität sowie der Überprüfung, ob die technischen Parameter der Bauteile den definierten Anforderungen entsprechen. Ein wesentliches Entscheidungskriterium ist hierbei die Wirtschaftlichkeit: Es gilt, eine technische Überdimensionierung zu vermeiden, um den vorgegebenen Kostenrahmen einzuhalten.

Auf Basis der ausgewählten Komponenten werden im nächsten Schritt detaillierte Schaltpläne und Fertigungsunterlagen erstellt, welche als Grundlage für die Produktion der benötigten Leiterplatten dienen. Parallel dazu erfolgt das Design der Gehäusekomponenten mittels CAD-Software. Dies umfasst sowohl die Konstruktion spezifischer Schutzgehäuse für die einzelnen Sensoreinheiten als auch die Entwicklung eines zentralen Gesamtgehäuses, in dem das System final verbaut und verkabelt wird.

Nach der Montage des Prototyps wird eine umfassende Betriebsanleitung verfasst. Diese dient dazu, den korrekten Betrieb zu gewährleisten, Bedienfehler zu minimieren und sowohl den Schutz des Geräts als auch die Sicherheit des Bedienpersonals sicherzustellen. Abschließend wird der gesamte Entwicklungsprozess, der technische Aufbau sowie die erzielten Ergebnisse im Rahmen der Diplomarbeit dokumentiert.

1.2 Vertiefende Aufgabenstellung Granegger (Software)

Softwareentwicklung und Datenverarbeitung

Die Aufgabenstellung konzentriert sich auf die Entwicklung der Software für den 3D-Scanner, um die automatisierte Erfassung, Auswertung und Speicherung der Sensordaten sicherzustellen und den Arbeitsablauf im Lager der Firma Leitner zu optimieren.

Dabei werden die Kameras und Sensoren softwareseitig angesteuert, um die Länge, Breite, Höhe sowie das Gewicht der Artikel präzise zu bestimmen. Gleichzeitig erfolgen die automatische Erkennung und Auswertung von Barcodes und QR-Codes, wodurch manuelle Eingaben minimiert und Fehlerquellen reduziert werden.

Die Messwerte werden in einer intuitiven grafischen Benutzeroberfläche übersichtlich dargestellt, die zudem die Möglichkeit bietet, Daten bei Bedarf manuell zu korrigieren. Alle Messergebnisse werden strukturiert in standardisierten Formaten wie CSV oder JSON gespeichert, um eine fehlerfreie Integration in das SAP-System der Firma Leitner zu gewährleisten. Zusätzlich werden Mechanismen zur Fehlererkennung, Wiederherstellung und Ausfallsicherheit implementiert, sodass die Software auch bei unerwarteten Störungen stabil arbeitet.

Der Entwicklungsprozess umfasst die Konzeption der Softwarearchitektur, die Integration der Sensorik, die Validierung der Messergebnisse und die Bereitstellung einer voll funktionsfähigen Schnittstelle für die direkte Übergabe der Daten an das Unternehmenssystem.

2 Dokumentation Schiechtl Alexander

2.1 Einleitung

Zur Gewährleistung einer klaren Aufgabenverteilung und Modularität wird die Hardware des Scannersystems in funktionale Teilbereiche untergliedert. Diese Strukturierung erleichtert sowohl die Entwicklung als auch die spätere Wartung der Komponenten. Dabei erfolgt eine primäre Differenzierung zwischen der elektrotechnischen Systemarchitektur und der mechanischen Konstruktion.

2.1.1 Elektrische Systemkomponenten

Der elektrotechnische Bereich bündelt sämtliche sensiblen und steuernden Einheiten, die für die Datenerfassung und den Betrieb notwendig sind. Dieser gliedert sich in folgende Subsysteme:

- **Kamera-System:** Kameras zur Erfassung der optischen Merkmale.
- **Räumliche Vermessung:** Sensorik zur Ermittlung der räumlichen Dimensionen
- **Wägetechnik:** Einheiten zur präzisen Gewichtserfassung.
- **Beleuchtung:** Steuerung der Lichtverhältnisse für optimale Aufnahmebedingungen.
- **Spannungsversorgung:** Bereitstellung und Wandlung der benötigten Betriebsspannungen.
- **Steuerung:** Zentrale Recheneinheit zur Koordination aller Module.

2.1.2 Mechanische Konstruktion

Die mechanische Auslegung bildet das physische Grundgerüst des Systems und sorgt für den Schutz sowie die korrekte Positionierung der Elektronik. Zu diesem Bereich zählen:

- **Komponentengehäuse:** Spezifische Schutzhüllen für einzelne Sensoreinheiten.
- **Hauptgehäuse:** Die zentrale Gesamtkonstruktion, welche den Scan-Bereich umschließt.
- **Verkabelung:** Die physische Leitungsführung zur Verbindung aller Komponenten.

2.1.3 Elektrisches Blockschaltbild

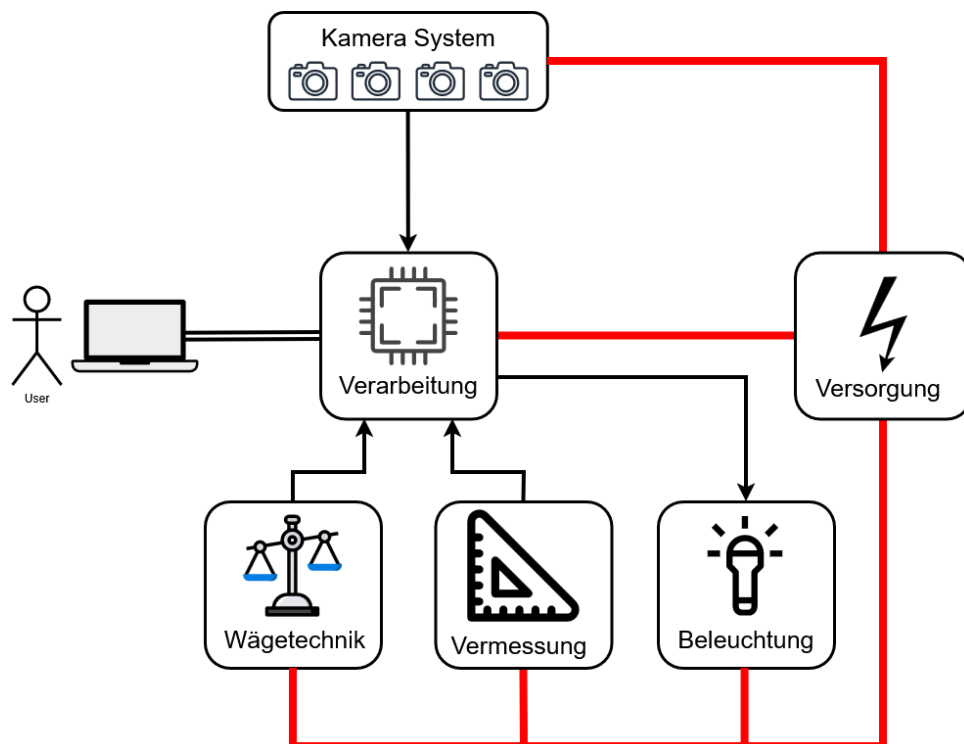


Abbildung 1: Blockschaltbild

2.2 Theoretische Grundlagen

2.2.1 Steuerung

Raspberry Pi

Der Raspberry Pi ist ein Einplatinencomputer, der von der britischen Raspberry Pi Foundation entwickelt wurde. Das System vereint alle wesentlichen Komponenten eines Computers auf einer einzigen Platine, die etwa die Größe einer Kreditkarte besitzt. Es existieren mittlerweile mehrere Generationen, die sich in Rechenleistung und Ausstattung unterscheiden, jedoch meist softwarekompatibel zueinander sind.



Abbildung 2: Raspberry Pi [1]

Das Herzstück des Raspberry Pi bildet ein System-on-a-Chip mit einem ARM-kompatiblen Prozessor. Die Spannungsversorgung erfolgt, je nach Modell, über eine Micro-USB- oder USB-C-Buchse mit einer Nennspannung von 5 V Gleichspannung. Ein zentrales Merkmal für die hardwarenahe Entwicklung ist die GPIO-Schnittstelle. Diese 40-polige Stiftleiste ermöglicht den direkten Anschluss von Sensoren, LEDs und anderen elektronischen Bauteilen. Wichtig ist hierbei der Logikpegel: Die GPIO-Pins arbeiten mit 3,3 V und sind im Gegensatz zu vielen Arduino-Modellen nicht 5-V-tolerant. Eine direkte Verbindung mit 5-V-Logik kann den SoC zerstören. Zudem bietet die Platine Schnittstellen wie I²C, SPI und UART sowie Hardware-Anschlüsse für Kameras (CSI) und Displays (DSI). [2]

Microcontroller

Ein Mikrocontroller ist ein kompaktes Computersystem, das sämtliche für den Betrieb notwendigen Komponenten auf einem einzigen integrierten Schaltkreis vereint. Im Gegensatz zu herkömmlichen Mikroprozessoren in Personal Computern, die auf externe Speicher- und Peripheriebausteine angewiesen sind, beinhaltet ein Mikrocontroller neben einem oder mehreren Prozessorkernen auch Arbeits- und Programmspeicher sowie programmierbare Ein- und Ausgabeschnittstellen. Das macht ihn zur idealen zentralen Steuereinheit für eingebettete Systeme.

Die Architektur von Mikrocontrollern ist primär auf Energieeffizienz und die direkte Interaktion mit Hardware ausgelegt. [3]

MicroSD-Karten

Die microSD-Karte ist das derzeit kleinste Flash-Speicherkartenformat, das sich elektrisch kompatibel zu SD-Karten verhält und Kapazitäten von wenigen Gigabyte bis zu mehreren Terabyte bietet. Sie arbeitet standardmäßig mit einer Betriebsspannung von 2,7 bis 3,6 V und verfügt über 8 Kontakte, wobei die Kommunikation entweder über den schnellen SD-Bus oder den für Mikrocontroller einfachen SPI-Modus erfolgt. [4]

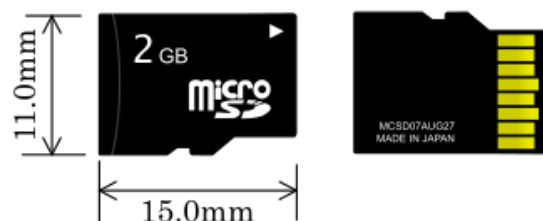


Abbildung 3: microSD [4]

2.2.2 Räumliche Vermessung

Um die Dimensionen der Objekte zu vermessen, gibt es mehrere Möglichkeiten. Hierzu gehören:

- Ultraschallsensoren
- Optische Sensoren
- Stereo-Vision-Tiefenkameras

Ultraschallsensoren

Ein Ultraschall-Distanzsensor ist ein elektronisches Messgerät, das hochfrequente Schallwellen nutzt, um die Entfernung zu einem Objekt berührungslos zu erfassen. Dabei emittiert ein piezoelektrischer Wandler hochfrequente Schallimpulse, die vom Objekt reflektiert und als Echo detektiert werden. Aus der gemessenen Zeitspanne zwischen Senden und Empfangen wird unter Berücksichtigung der Schallgeschwindigkeit die Distanz berechnet. [5]

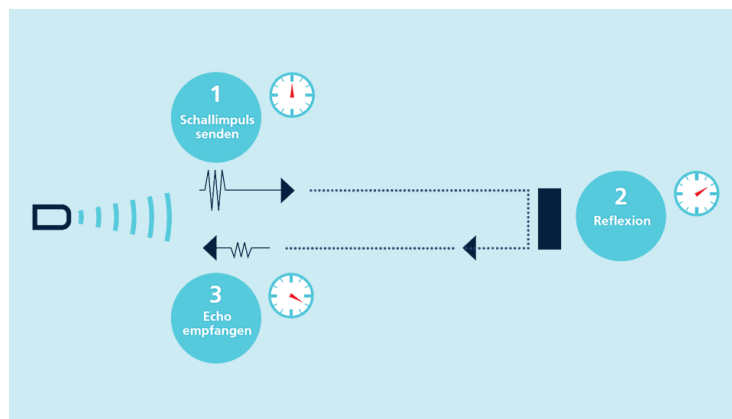


Abbildung 4: Ultraschallsensor [6]

Optische Sensoren

Optische Distanzsensoren messen Entfernungen berührungslos mithilfe von Licht, meist durch Laserquellen. Je nach Anforderung und Messbereich kommen unterschiedliche physikalische Prinzipien zum Einsatz.

Lasertriangulation: Dieses Verfahren projiziert einen Lichtpunkt auf das Objekt. Das reflektierte Licht trifft winkelabhängig auf ein Empfangselement. Eine Distanzänderung verschiebt die Position des Lichtpunkts auf dem Empfänger, woraus die Entfernung geometrisch berechnet wird. Dieses Prinzip wird auch von speziellen Tiefenkameras zur detaillierten 3D-Rekonstruktion genutzt. [7]

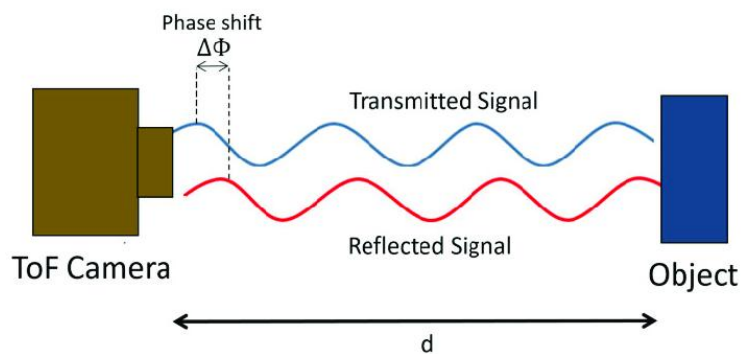


Abbildung 5: ToF Kamera [8]

Time-of-Flight (ToF): Hierbei wird die Laufzeit eines ausgesandten Lichtpakets zum Objekt und zurück gemessen. Dieses Verfahren eignet sich besonders für größere Distanzen. [7]

Durch die Erweiterung dieser Prinzipien auf den dreidimensionalen Raum – etwa beim LIDAR-Verfahren durch mechanische Ablenkung oder Multi-Sensor-Anordnungen – lassen sich räumliche Koordinaten erfassen und als Punktwolke generieren. [9]

Stereovision

Eine weitere Art der Entfernungsmessung ist die sogenannte Stereo-Vision. Weiteres zu diesem Konzept wird im **Kapitel 3.2.1** erläutert.

2.2.3 Wägetechnik

Messung

Die Bestimmung der Gewichtskraft erfolgt über den Einsatz von Wägezellen. Wägezellen sind Kraftsensoren, die eine auf diese gewirkte Kraft in ein elektrisches Signal transformieren.

Die in der Industrie am häufigsten auftretende Art der Zellen sind die Dehnstreifen-Wägezellen, diese bestehen aus einem Metallgehäuse und Dehnstreifen, welche direkt am Metall fixiert sind. Wenn nun eine Kraft auf die Zelle wirkt, verformt sich der Dehnstreifen entsprechend, was dessen Widerstand ändert. Synchron dazu wird konstant ein elektrisches Signal über den Streifen geleitet. Die Spannung dieses Signals verändert sich durch die Verformung ebenfalls aufgrund der Widerstandsänderung. Diese Änderung wird von einem Controller dann aufgenommen und in eine Kraft oder eine Masse umgerechnet.

Um eine möglichst hohe Genauigkeit zu erzielen, werden mehrere Messstreifen verbaut, welche intern als Wheatstone-Brücke verschaltet werden. [10]

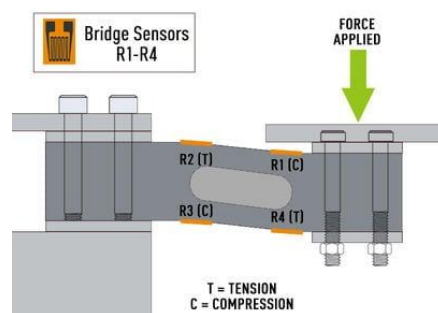


Abbildung 6: Wägezelle Aufbau [10]

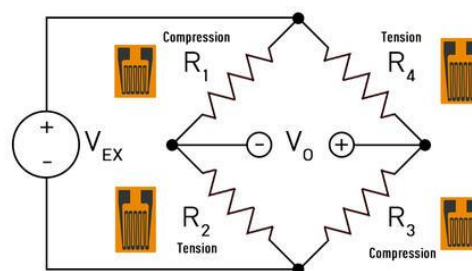


Abbildung 7: Wheatstone'sche Messbrücke [10]

Die Spannung V_{EX} wird von außen zugeführt und die Spannung V_0 ist diejenige die zum Controller für die Auswertung zurückkommt.

Die gesamte Änderung des Widerstandes lässt sich anschließend über das Ohm'sche Gesetz ermitteln.

$$V_0 = \frac{R_3}{R_3 + R_4} - \frac{R_2}{R_1 + R_2} * V_{EX}$$

Auswertung

Die Auswertung der Messergebnisse erfolgt über zwei Schritte: Die Wandlung für den Controller und die Summierung der Messwerte.

Viele Controller, wie zum Beispiel auch der Raspberry Pi 5, haben keine analogen I/O Pins verbaut. [2] Da das erhaltene Signal aber analog ist, muss dieses zuerst mittels Analog Digital Wandler gewandelt werden.

Ob zuerst die Summierung oder die Wandlung erfolgt, hängt davon ab, welche Art der Summierung verwendet wird. Es wird unterschieden zwischen der Software-Summierung und Hardware-Summierung. Bei der Software-Summierung werden die einzelnen gewandelten Werte im Programmcode einfach addiert.

Bei der Hardware-Summierung wird ein sogenannter Summierkörper verwendet. Die Aufgabe dieses Summierkörpers ist es, die einzelnen elektrischen Signale elektronisch aufzusummieren, um am Ausgang ein einziges Signal für den Controller bereitzustellen. Eine solche Konstruktion besteht in der Regel aus mehreren Komponenten: Verstärker, Filter und angepasste Leitungen.

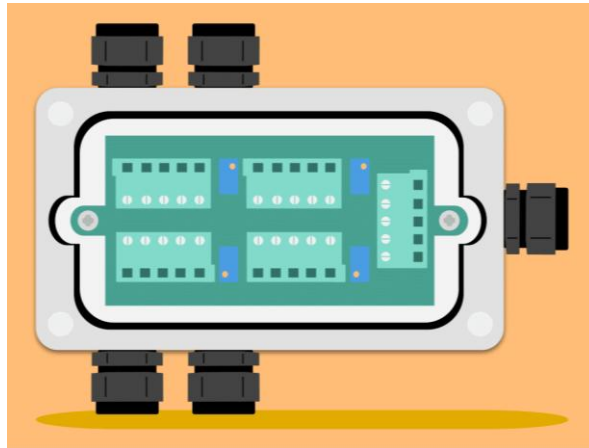


Abbildung 8: Summierkörper [11]

Der große Vorteil von Summierkörpern gegenüber der Software-Summierung ist, dass diese direkt Filter eingebaut haben, welche unerwünschte Signale rausfiltern. Sind diese richtig kalibriert, können diese daher viel genauer arbeiten. Ist das Signal schon auf der Leitung summiert, können auch ADCs eingespart werden, da nun nur mehr einer benötigt wird. [11]

2.2.4 Beleuchtung

RGB-LEDs

Eine RGB-LED vereint drei Dioden (Rot, Grün, Blau) in einem Gehäuse. Durch additive Farbmischung lassen sich durch Variation der Einzelintensitäten nahezu alle Spektralfarben erzeugen. Die vier Anschlüsse unterteilen sich in drei Steuereingänge und einen gemeinsamen Pol (gemeinsame Anode oder Kathode). [12]

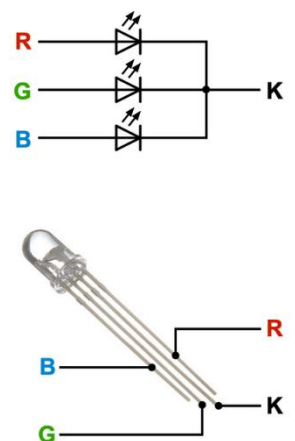


Abbildung 9: RGB-LED

WS2812-Controller

Die WS2812 ist eine RGB-LED mit einem integrierten Controller-IC, die sich besonders für kaskadierbare Beleuchtungsanwendungen eignet. Die Datenübertragung erfolgt über ein asynchrones Eindraht-Bussystem, bei dem die Datenbits nicht durch einen separaten Takt, sondern durch strikt definierte Impulslängen kodiert werden. Jede LED empfängt einen 24-Bit-Farbwert, speichert diesen und reicht alle nachfolgenden Datenströme an ihrem Data-Out-Pin zur nächsten LED in der Kette weiter. [13]

2.2.5 Verkabelung

Querschnitte

Die korrekte Berechnung bzw. die Wahl des Leitungsquerschnitts ist unerlässlich, um einen sicheren, verlustarmen Betrieb von elektrischen Verbrauchern zu gewährleisten und Überhitzung oder gar Kabelbrände durch zu hohe Leitungswiderstände zu vermeiden. Der erforderliche Querschnitt hängt von zentralen elektrischen Faktoren ab: der maximalen Stromstärke, der anliegenden Betriebsspannung, der Kabellänge, dem spezifischen Materialleitwert und dem noch vertretbaren Spannungsabfall. [14]

Abschirmungen

Bei elektrischen Leitungen und Kabeln dient die Abschirmung dazu, die inneren Adern vor externen elektromagnetischen Störfeldern zu schützen (Störfestigkeit) und gleichzeitig das Austreten eigener Signale zu verhindern. Technisch wird dies meist durch ein leitfähiges Geflecht aus Kupferdrähten oder eine umwickelte Aluminiumfolie realisiert, die die isolierten Innenleiter umschließt und wie ein faradayscher Käfig wirkt. Besonders bei der Datenübertragung (z. B. Koaxial- oder Netzwirkabel) ist eine effektive Schirmung essenziell. [15]



Abbildung 10: Abschirmung eines Kabels [16]

Differenzielle Leiterpaare

Bei der differentiellen Signalübertragung wird ein elektrisches Signal nicht über eine einzelne Leitung gegen eine gemeinsame Masse übertragen, sondern über ein Paar eng gekoppelter Leitungen, wobei eine Leitung das ursprüngliche Signal und die andere das exakt invertierte Signal führt. Der Empfänger wertet zur Informationsgewinnung lediglich die Spannungsdifferenz zwischen diesen beiden Leitern aus. Da externe elektromagnetische Störungen meist auf beide räumlich eng beieinanderliegenden Leitungen gleichermaßen einwirken (Gleichtaktstörung),

heben sich diese Störanteile bei der Differenzbildung im Empfänger auf, was eine hohe Störfestigkeit und Signalintegrität ermöglicht. [17]

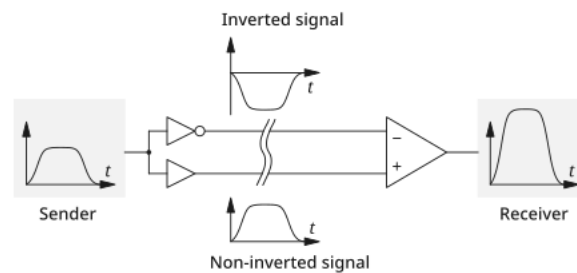


Abbildung 11: Differentieller Signalverlauf [18]

Um in diesen Leitungen nun Störungen zu verhindern, gibt es mehrere Möglichkeiten.

Um hauptsächlich Störungen von außen zu vermeiden, wird generell empfohlen, ein abgeschirmtes Kabel zu verwenden. Zusätzlich wird empfohlen, für die Datenleitung eine Twisted-Pair-Leitung, also ein verdrehtes Adernpaar zu verwenden. Wenn bei einem verdrehten Adernpaar Störeinflüsse von außen kommen, wirkt sich dieses gleichermaßen auf beide Adern aus und da es sich um ein differenzielles Adernpaar handelt, haben die Störeinflüsse quasi gar keinen Einfluss mehr auf das Ergebnis, da sich die Störungen durch die Bildung der Differenz der beiden Signaldrähte auslöschen. [19]

Ist eine höhere Messgenauigkeit erforderlich, kann zusätzlich noch ein differentieller Tiefpass verwendet werden. Dieser Filter filtert nur das differenzielle Signal und lässt die einzelnen Signale weitgehend unberührt.

Eine einfache Konfiguration hierfür wäre folgende:

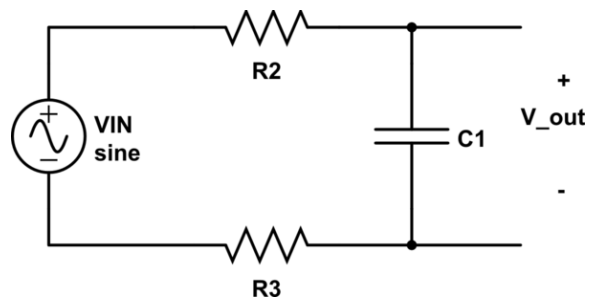


Abbildung 12: Differenzieller Tiefpass [20]

Da es sich aber um hochgenaue Anwendungsfälle handelt werden diese häufig als Filter höherer Ordnung aufgebaut. [21]

2.2.6 Schnittstellen

USB

Der USB ist ein internationaler Industriestandard. Er ersetzt eine Vielzahl älterer Schnittstellen wie serielle und parallele Ports. USB definiert sowohl die Kabel als auch Steckverbinder sowie die Protokolle für die Kommunikation und Energieversorgung zwischen einem Host und Peripheriegeräten. Ein wesentliches Merkmal ist die "Hot-Swap"-Fähigkeit, die es erlaubt, Geräte im laufenden Betrieb anzuschließen und zu entfernen, sowie "Plug and Play", wodurch angeschlossene Geräte oft ohne manuelle Konfiguration sofort einsatzbereit sind.

USB nutzt eine serielle Datenübertragung. In den ursprünglichen Versionen erfolgt die Signalübertragung differentiell über ein verdrehtes Adernpaar (D+ und D-), ergänzt durch zwei Adern für die Spannungsversorgung. Die Standardspannung beträgt 5 V DC. USB hat sich über die Jahre in Hinblick auf Geschwindigkeit sowie auf die Bauart stark verändert. [22]

relevante Bauarten für die Diplomarbeit:

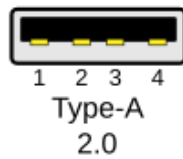


Abbildung 13: USB-A 2.0 [22]

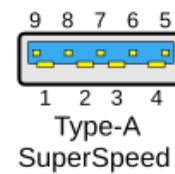


Abbildung 14: USB-A 3.0 [22]

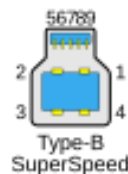


Abbildung 15: USB-B [22]



Abbildung 16: USB-C [22]

Ethernet

Ethernet ist der weltweit etablierte Standard für kabelgebundene, lokale Netzwerke, der die Spezifikationen für die Verbindung und den Datenaustausch zwischen Computern und anderen Netzwerkgeräten definiert. Die physikalische Verbindung erfolgt meist über Kupferkabel mit RJ45-Steckern. Die Kabel werden in verschiedene Kategorien eingeteilt, die maßgeblich die maximale Übertragungsgeschwindigkeit und Störfestigkeit bestimmen. Ethernet bietet die Grundlage für die Verbindung zum Raspberry Pi. [23]

CSI

Das Camera Serial Interface (CSI) ist eine Spezifikation, die eine standardisierte Schnittstelle zwischen einer Kamera und einem Host-Prozessor definiert. Ziel ist es, eine leistungsfähige, serielle Verbindung bereitzustellen, die hohe Datenraten für hochauflösende Bilder und Videos ermöglicht, dabei aber den Energieverbrauch und die elektromagnetischen Emissionen minimiert. Die bekannteste und am weitesten verbreitete Version ist CSI-2, die sich als Industriestandard für mobile Geräte durchgesetzt hat.

CSI-2 ist ein differenzielles, serielles Protokoll, das auf verschiedenen physikalischen Schichten basieren kann.

Der Raspberry Pi nutzt beispielsweise CSI-2, um das offizielle Kameramodul direkt an den Broadcom-Chip anzubinden, was eine wesentlich höhere Performance als USB-Kameras ermöglicht. [24]

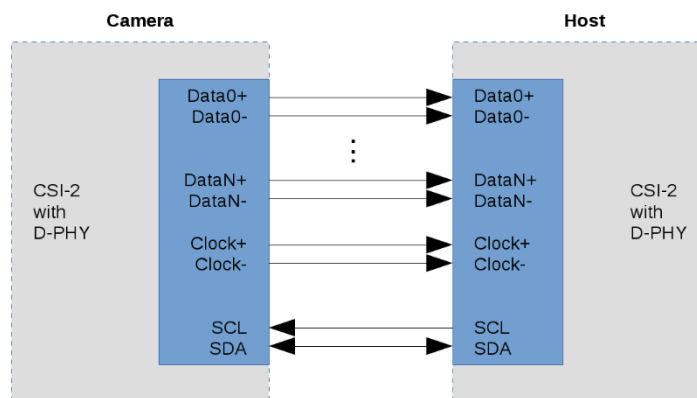


Abbildung 17: CSI Anschluss Block [24]

2.2.7 Gehäusetechnik

3D-Druck

Beim 3D-Druck unterscheiden sich die beiden dominierenden Verfahren FDM und SLA grundlegend in ihrer Funktionsweise:

FDM schmilzt Kunststofffilament und trägt es schichtweise auf, während SLA flüssiges Kunstharz präzise mit einem Laser bzw. mit UV-Licht aushärtet. Hinsichtlich der technischen Eigenschaften punktet FDM durch geringere Kosten und eine enorme Materialvielfalt für mechanisch oder thermisch belastbare Teile, weist aber sichtbare Druckschichten auf. SLA hingegen liefert eine hohe Detailgenauigkeit und nahezu spritzgussartige, glatte Oberflächen, ist jedoch bei den Materialkosten meist teurer. [25]

3D-CAD

Die Grundlage für 3D Druck bildet die rechnergestützte Konstruktion mittels CAD-Software. Mit Hilfe dieser Programme werden die dreidimensionalen geometrischen Modelle der Bauteile erstellt. Im Gegensatz zu herkömmlichen Grafikprogrammen arbeiten CAD-Systeme parametrisch und vektorbasiert. Dies gewährleistet eine exakte Maßhaltigkeit und die für technische Anwendungen erforderliche geometrische Präzision. Wurde ein Volumenkörper erstellt, ist der Prozess jedoch noch nicht fertig, da die erstellte .ipt-Datei von 3D Druckern nicht gelesen werden kann. Hier kommen STL-Dateien ins Spiel.

STL

Die STL-Datei ist das branchenübliche Standardformat für die Datenübertragung im 3D-Druck. Bei diesem Format wird die Geometrie eines dreidimensionalen CAD-Festkörpermodells nicht durch exakte mathematische Kurven, sondern durch ein Netz aus aneinandergereihten Dreiecken angenähert. Je komplexer und geschwungener die Oberfläche des Objekts ist, desto mehr Dreiecke und eine, umso höhere Auflösung sind nötig, um das Modell detailgetreu abzubilden. [26]

Die generierte STL-Datei wird mithilfe einer Slicing-Software in Schichten unterteilt, welche anschließend als Grundlage für den Schichtaufbau des Druckers dienen.

2.2.8 Verwendung von Literatur, Internetquellen und KI

Die theoretischen Inhalte dieses Kapitel stützten sich hauptsächlich auf Internetquellen und auf erworbenes Wissen aus den Fächern „Messtechnik und Regelungssysteme“, „Kommunikationssysteme und Netze“, „Hardwareentwicklung“ sowie auf die Inhalte des Werkstätten-Unterrichts. Für physikalische sowie elektrische Größen wurden sich die von den Herstellern zur Verfügung gestellten Datenblätter zu Nutze gemacht.

Zusätzlich wurde ein KI-System unterstützend zur sprachlichen Strukturierung und zur Erklärung grundlegender Zusammenhänge verwendet. Sämtliche fachlichen Inhalte wurden eigenständig überprüft und mit geeigneten Quellen abgeglichen.

2.3 Praktische Umsetzung

2.3.1 Projektplanung

Zu Beginn dieser Diplomarbeit wurden im Rahmen eines Praktikums bei der Firma Leitner der Projektablauf sowie die Anforderungen nach der PRINCE2-Methodik festgelegt. In dieser Phase entstanden wesentliche Projektdokumente, welche die strukturierte Entwicklung des Systems unterstützen. [27]

- Project Brief
- Project Initiation
- Product Description
- Risikotabelle

Project Brief

In der Project Brief werden die grundlegenden Funktionen sowie die Zielsetzung des Projekts definiert. Die darin festgelegten Punkte dienen als verbindliche Parameter für den weiteren Projektablauf. Eine Änderung dieser Definitionen ist ausschließlich nach Absprache mit den beteiligten Parteien zulässig.

Project Initiation

Das Project Initiation Document beschreibt den Projektumfang realitätsnah und detailliert. Es definiert die Mittel zur Zielerreichung sowie zulässige Toleranzen. Zudem wird eine Kommunikationsstrategie festgelegt, die Zeitpunkte und Anlässe für den Austausch mit beteiligten Parteien regelt. Ein integrierter Zeitplan definiert die Bedingungen für den Abschluss einzelner Projektphasen und den Übergang zur nächsten.

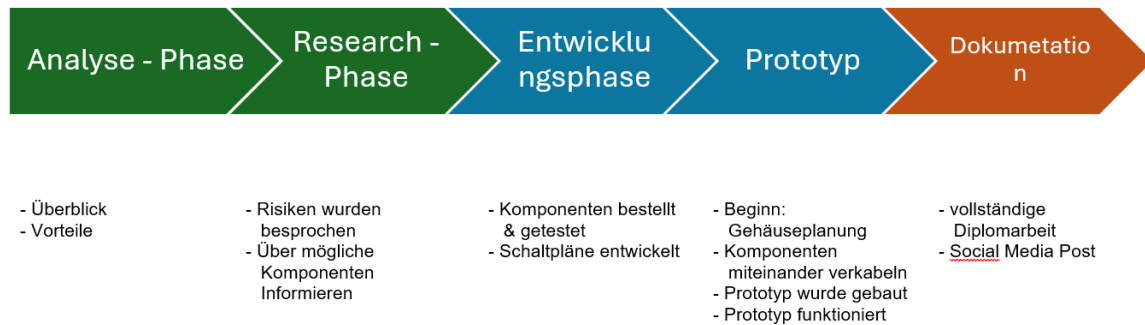


Abbildung 18: Phasenübergänge / Project Initiation

Product Description

Die Product Description beschreibt das zu entwickelnde Endprodukt im Detail und legt fest, welche Eigenschaften und Funktionen dieses erfüllen muss. Dabei werden sowohl der Aufbau als auch die einzelnen Komponenten des Systems definiert. Zusätzlich werden technische Anforderungen, wie beispielsweise Genauigkeit, Leistungsfähigkeit und Schnittstellen, festgelegt.

Risikotabelle

In der Risikotabelle werden potenzielle Ereignisse sowie kritische Entscheidungen nach ihrer Eintrittswahrscheinlichkeit kategorisiert und bewertet. Dies ermöglicht eine frühzeitige Reaktion sowie die proaktive Ausarbeitung von Maßnahmenplänen zur Bewältigung möglicher Probleme.

(Siehe Anhang: Risikotabelle)

2.3.2 Komponentenauswahl

Bei der Auswahl der Hardwarekomponenten sind zahlreiche Faktoren ausschlaggebend, wobei die Wirtschaftlichkeit der Entscheidungen eine zentrale Rolle spielt. Ein Produkt mag technisch ideal geeignet sein, kann jedoch den vorgegebenen Kostenrahmen überschreiten. Für die Firma Leitner ist zudem die Auswahl von Bauteilen aus ihrer bestehenden Lieferantenliste wesentlich, um die geforderten Qualitätsstandards sicherzustellen. Ein weiteres wichtiges Kriterium war die Verfügbarkeit einer umfassenden Dokumentation sowie detaillierter Datenblätter für die gewählten Komponenten.

Steuerung

Für die Ansteuerung der Hauptkomponenten erweist sich ein Raspberry Pi als die geeignetste Lösung. Dieser verfügt über ausreichende Ressourcen, um einen flüssigen und schnellen Systembetrieb zu gewährleisten. Ein wesentlicher Vorteil gegenüber herkömmlichen Mikrocontrollern, die meist nativ in C programmiert werden, ist die Zeitersparnis bei der Softwareentwicklung für komplexe Bauteile.

Aufgrund der Vielzahl an verfügbaren Raspberry Pi Modellen wurde eine Auswahl basierend auf den spezifischen Anforderungen getroffen. Da für die in dieser Diplomarbeit eingesetzten KI-Verarbeitungsprozesse eine hohe Rechenleistung erforderlich ist, fiel die Entscheidung auf den **Raspberry Pi 5**. Dieser ist mit einem Arm Cortex-A76 Prozessor ausgestattet. [28] Bei der Wahl des Arbeitsspeichers wurde darauf geachtet, eine Konfiguration zu wählen, die zukünftige Performance-Probleme ausschließt, ohne dabei unnötig Ressourcen zu verschwenden. Als Benutzeroberfläche dient das Raspberry Pi OS mit integrierter grafischer Oberfläche, welche direkt als intuitives Interface für den Anwender verwendet werden kann.

Ein kritischer Aspekt beim Einsatz eines Raspberry Pi ist die Zuverlässigkeit des Speichermediums. Erfahrungswerte zeigen, dass herkömmliche Micro-SD-Karten oft eine Schwachstelle darstellen und zu Systemausfällen führen können. Um die Betriebssicherheit zu erhöhen und einen frühzeitigen Hardwareausfall zu vermeiden, wird daher eine Micro-SD-Karte im „Industrial Grade“ eingesetzt.

Eine technische Einschränkung des Raspberry Pi im Kontext dieses Projekts besteht darin, dass er keine nativen PWM-Signale generieren kann, die für die direkte Ansteuerung der RGB-Streifen zur Beleuchtung erforderlich sind. Um den Betrieb der Beleuchtungseinheit dennoch zu ermöglichen, wird ein zusätzlicher Treiberbaustein mit passender Firmware zwischengeschaltet. Die Wahl fiel hierbei auf den **LOLIN D1 Mini**, einen Controller auf Basis des ESP8266. Dieser ist im Vergleich zum ESP32 kompakter und verfügt über eine geringere Anzahl an I/O-Pins. Aufgrund seines geringen Platzbedarfs und der hohen Energieeffizienz stellt er die ideale Lösung dar.

Tiefenmessung

Bei der Auswahl der geeigneten Sensorik für die Tiefenmessung wurde ein systematisches Ausschlussverfahren angewandt. Dabei erwiesen sich die folgenden Faktoren als entscheidende Auswahlkriterien:

- Ein kritischer Parameter ist der Mindestabstand zum Objekt. Da viele Hersteller ihre Geräte auf maximale Reichweiten optimieren, verfügen zahlreiche Sensoren über eine bauartbedingte minimale Distanz, unterhalb derer keine zuverlässige Datenerfassung mehr möglich ist. Für die Anforderungen dieses Projekts ist jedoch eine präzise Erfassung im Nahbereich essenziell.

- Idealerweise sollte das System über eine zusätzliche RGB-Kamera verfügen. Durch diese Kombination kann die Anzahl der benötigten Hardware-Komponenten reduziert werden.
- Die Kamera sollte aber auch nicht zu teuer sein und maximal ein Viertel des Gesamtbudgets ausmachen.

Da Ultraschallsensoren meist nur punktuelle Messungen ermöglichen, wurden diese für die geforderte Anwendung ausgeschlossen. In die engere Auswahl fielen daher optische Tiefenkameras und Stereo-Vision-Systeme. Während viele Startups in diesem Bereich einen starken Fokus auf KI setzen, dominieren industrielle Lösungen den Markt, welche jedoch mit Kosten im vierstelligen Bereich das Budget überschreiten. Alternativen von Sony basieren primär auf der ToF-Technik, weisen jedoch eine eingeschränkte Verfügbarkeit auf. [29]

Aufgrund der Anfälligkeit optischer Tiefenkameras gegenüber Lichtreflexionen wurde ein Stereo-Vision-Konzept bevorzugt. Die Wahl fiel auf die **Oak D-S2-AF** der Firma Luxonis. Diese Kamera erfüllt alle Anforderungen: Sie integriert mittig eine RGB-Kamera und ermöglicht durch ihren Autofokus eine sehr geringe minimale Reichweite von ca. 20 cm (bei einer Auflösung von 400p und Extended Disparity). [30] Mit einem Preis von ca. 250 € liegt sie zudem im vorgegebenen Kostenrahmen. Ein weiterer Vorteil ist die umfassende Dokumentation der Python-Library durch den Hersteller.



Abbildung 19: OAK-D S2 AF [30]

Da die Kamera über eine eigene CPU verfügt und die Berechnungen intern durchführt, liegt die Leistungsaufnahme bei etwa 5 W (1 A bei 5 V). Da der Raspberry Pi bei dieser Last an seine Kapazitätsgrenzen stößt und Systemabstürze drohen, wird ein aktiver USB-Hub zur stabilen Stromversorgung eingesetzt. Hierfür wurde das Modell U360-004-2F von EATON gewählt, welches eine Stromstärke von bis zu 3,5 A bereitstellt.

Kameras

Für die Ansteuerung über den Raspberry Pi 5 stehen primär zwei Schnittstellentypen zur Verfügung: USB und CSI. Aufgrund ihrer nativen Unterstützung und der hohen Datenübertragungsrate wären die Raspberry Pi Camera Modules prinzipiell ideal für dieses Einsatzgebiet. Die wesentliche Einschränkung des Raspberry Pi 5 besteht jedoch darin, dass lediglich zwei CSI-Anschlüsse zur Verfügung stehen.

Zur Überwindung dieser Limitierung wurden verschiedene Lösungsansätze geprüft. Der Hersteller Arducam bietet für diesen Anwendungsfall spezialisierte Adapter-Platinen an. Während die kostengünstigste Variante direkt über den Hersteller für etwa 50\$ beziehbar ist, lagen die Angebote bei gängigen Elektronikdistributoren wie Digi-Key zum Zeitpunkt der Auswahl im Bereich von 90€. [31] Aufgrund dieser Preisgestaltung erwies sich diese Option aus wirtschaftlicher Sicht als unvorteilhaft.

Aufgrund der Anschlusslimitierung des Raspberry Pi 5 wurde entschieden, das Kamerasystem vollständig auf USB-Basis aufzubauen. Dieser Ansatz minimiert sowohl den Programmieraufwand als auch die Fehleranfälligkeit bei der Datenübertragung.

Für die technische Auswahl sind die Auflösung und die Brennweite die entscheidenden Faktoren. Da die bereits implementierte Tiefenkamera hochauflösende Aufnahmen liefert, ist für die ergänzenden Kameras eine Bildqualität im mittleren Segment ausreichend. Eine kritischere Rolle spielt die Brennweite, da die Positionierung der Kameras in unmittelbarer Nähe zum Messobjekt erfolgt. Die Gehäusedimensionierung steht in direktem Zusammenhang mit diesem Parameter: Eine geringere Brennweite würde einen größeren Abstand zum Objekt und somit ein voluminöseres Gehäuse erfordern, um den gesamten Artikel erfassen zu können.

Die Wahl fiel auf das Modell **FIT0892** des Herstellers DFRobot. Diese Kamera verfügt über eine Auflösung von 720p und ein Weitwinkelobjektiv mit einem Sichtfeld von 140°. [32] Mit einem Anschaffungspreis von etwa 20 € über den Distributor Mouser erfüllt dieses Modul sämtliche technischen und wirtschaftlichen Anforderungen des Projekts.



Abbildung 20: FIT0892

Wägetechnik

Für die Gewichtserfassung wurden hochwertige Wägezellen des Typs **CP 20-2Y1** der Firma Kern gewählt. Mit einer maximalen Traglast von jeweils 20 kg bieten diese Sensoren ausreichende Kapazitäten und zeichnen sich durch eine robuste Bauweise sowie eine umfassende Dokumentation aus. Trotz des Stückpreises von ca. 40 € wurde dieser Kostenfaktor bewusst in Kauf genommen, um die langfristige Stabilität der Wägeplatte und eine dauerhafte Messqualität zu gewährleisten. [33]

Das System nutzt drei dieser Zellen, da eine Dreipunktlagerung systembedingt eine wackelfreie Ebene definiert. Die resultierende Gesamtkapazität von 60 kg im Zentrum der Konstruktion ist für die geplanten Anforderungen im Lagerbetrieb der Leitner AG mehr als ausreichend.

Bei der Signalverarbeitung wurde eine Software-Summierung implementiert. Im Vergleich zu Hardware-Summierkörpern bietet dieses Verfahren wirtschaftliche Vorteile und reduziert den zeitlichen Aufwand bei der Kalibrierung. Für eine präzise Messung wird für jede Zelle individuell ein Kalibrierungsfaktor ermittelt und softwareseitig verrechnet.

Als Analog-Digital-Wandler (ADC) kommt der **Adafruit NAU7802** zum Einsatz. Dieses Modul bietet mit einer Auflösung von 24 Bit sowie einer einstellbaren Verstärkung ideale Voraussetzungen für die Auswertung von Wägezellen. Da der ADC über das I2C-Protokoll mit einer festen Adresse kommuniziert, pro Zelle jedoch ein eigener Wandler benötigt wird, erfolgt die Anbindung an den Raspberry Pi über einen I²C-Multiplexer. Dies ermöglicht die Verwaltung mehrerer identischer Adressen an einem einzigen Bus-System. [34]

Beleuchtung

Die Beleuchtung stellt einen kritischen Faktor für die Systemperformance dar, da sie maßgeblichen Einfluss auf die Qualität der Fotoaufnahmen sowie auf die Präzision der 3D-Wertermittlung hat. Ziel der Konzeption ist eine homogene Ausleuchtung, die weder Übersteuerungen durch zu hohe Helligkeit noch Informationsverluste in zu dunklen Bereichen aufweist. Ein besonderer Fokus liegt dabei auf der Minimierung von Schattenwürfen, um die automatische Datenerfassung nicht zu beeinträchtigen.

Zur Erzielung einer gleichmäßigen Lichtverteilung ist die Installation eines RGB-Streifens an der Gehäusedecke vorgesehen. Um darüber hinaus optimale Bedingungen für die fotovisuelle Dokumentation zu schaffen, wird jede RGB-Kamera zusätzlich mit einer dedizierten Blitzlichteinheit ausgestattet.

Versorgung

Für die stabile Versorgung des Systems wurde die Gesamtleistungsaufnahme aller Einzelverbraucher ermittelt:

Bauteil	Anzahl	Gesamte Stromstärke
Raspberry Pi 5	1	5A
RGB-Kameras (USB-Hub)	3	660mA
Tiefenkamera (USB-Hub)	1	1A
RGB-Strip	1	2A
RGB-Ringlichter	3	1A
ADCs	3	6mA
	Summe:	9,666A

Auf Basis dieser Berechnung ist ein Netzteil mit einer Strombelastbarkeit von mindestens 10 A erforderlich. Zudem benötigt das System zwei verschiedene Spannungspegel: 5 V für die Logikkomponenten und 12 V für die Peripherie.

Die Wahl fiel auf ein Netzteil der Marke Mean Well (bezogen über RS Components), welches beide Spannungspegel nativ bereitstellt und an beiden Ausgängen jeweils 7,7 A liefert. Da die gesamteameratechnik über den aktiven USB-Hub mit 12 V versorgt wird und somit die Last auf beide Schienen verteilt ist, bieten die 7,7 A pro Ausgang eine ausreichende Kapazität für den sicheren Betrieb des Gesamtsystems.

[35]

Bauteilliste

Artikel	Lieferant	Anzahl	Lieferanten Teilenummer	Einzelpreis [€]	Gesamtpreis [€]
Raspberry Pi	Conrad	1	2980040 - UP	93,99	93,99
Wägezellen	Conrad	3	2340059 - 62	39,27	117,81
Netzteil	RS Components	1	644-6941	50,44	50,44
Micro SD	RS Components	1	262-9689	41,17	41,17
Kameras	Mouser	3	426-FIT0892	17,11	51,33
Tiefenkamera	Mouser	1	392-OAK-D S2 AF	257,14	257,14
Aktiver USB- Hub	Mouser	1	545-U360-004-2F	51,46	51,46
LED-Streifen	Digikey	1 (4m)	1528-1461-ND	46,83	46,83
ADCs	Digikey	3	1528-4538-ND	5,07	15,21
Ringlichter	Digikey	4	1738-DFR0888-16- ND	8,44	33,76
LOLIN D1 Mini	Privatbestand	1	-	-	-

2.3.3 Ansteuerung der Beleuchtung

Wie bereits dargelegt, kann der Raspberry Pi 5 systembedingt kein kompatibles PWM-Signal zur direkten Ansteuerung von WS2812B-RGB-LEDs generieren. Zur Überbrückung dieser technischen Limitierung wird ein Lolin D1 Mini als Treiberbaustein zwischengeschaltet. Die physische Ansteuerung des RGB-Controllers erfolgt dabei über die Pins GPIO12 und GPIO13 des Mikrocontrollers.

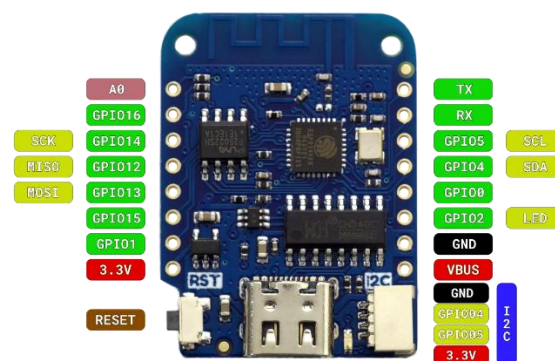


Abbildung 21: D1 Mini - Pinout [36]

Ursprünglich war vorgesehen, die Kommunikation zwischen dem Raspberry Pi 5 und dem D1 Mini über die jeweilige UART-Schnittstelle zu realisieren. Im Zuge der softwareseitigen Implementierung auf dem Raspberry Pi traten jedoch erhebliche Probleme bei der Konfiguration der TxD- und RxD-Leitungen auf, wodurch eine stabile Verbindung nicht zeitgerecht hergestellt werden konnte. Um die Projektsicherheit zu gewährleisten, wurde die Kommunikationsstrategie angepasst: Der D1 Mini wird nun über eine USB-Schnittstelle mit dem Raspberry Pi verbunden, über welche der Datentransfer erfolgt.

Auf dem D1 Mini ist eine spezifische Firmware programmiert worden, die den COM-Port kontinuierlich auf eingehende Strings überwacht. Sobald vordefinierte Steuerbefehle empfangen werden, setzt der Mikrocontroller diese in die entsprechenden Signale zur Ansteuerung der LED-Segmente um.

Verwendet wurde hierfür die FastLed Library mit der eine einfache Umsetzung dieses Vorhabens möglich ist.

Code in der Main Funktion:

```
1.  if (Serial.available())
2.  {
3.      String cmd = Serial.readStringUntil('\n');    // bis Zeilenumbruch lesen
4.      cmd.trim();                                  // ggf. \r entfernen
5.
6.      if (cmd.equalsIgnoreCase(F("Change")))
7.      {
8.          Serial.println(F("Waiting for command (1 = Blitz Ring 1)"));
9.          changeMode = true;                      // wir gehen jetzt in den Sub-Modus
10.     }
11.     else if (changeMode)
12.     {
13.         char c = cmd.charAt(0);
14.         switch (c)
15.         {
16.             case '1': Blitz(1); break;
17.             case '2': Blitz(2); break;
18.             case '3': Blitz(3); break;
19.             case 'a': All_ON(); break;
20.             case '0': All_OFF(); break;
21.             default: Serial.println(F("unknown sub-command")); break;
22.         }
23.         changeMode = false;                      // zurück in den Normalmodus
24.     }
25. }
```


Der Code prüft, ob eine serielle Verbindung zu einem anderen Gerät vorhanden ist. Wenn das der Fall ist, liest er konstant die Daten dieser Verbindung ein. Wird ein String mit dem Inhalt „Change“ übermittelt, schaltet der Controller in den changeMode und wartet auf eine weitere Eingabe. Je nach Eingabe wird dann eine vorher definierte Funktion ausgelöst.

```
1. //Alle LEDs einschalten
2. void All_ON(void)
3. {
4.     for (uint16_t i = 0; i < NUM_LEDS_RING; ++i)
5.     {
6.         leds_ring[i] = CRGB::White;
7.         FastLED.show();
8.     }
9.
10.    for (uint16_t i = 0; i < NUM_LEDS_STRIP; ++i)
11.    {
12.        leds_strip[i] = CRGB::White;
13.        FastLED.show();
14.    }
15. }
```

In diesem Beispiel einer Funktion sieht man gut, wie diese abläuft. In der for-Schleife wird definiert, wie viele LEDs in der Funktion eingeschaltet bzw. ausgeschaltet werden sollen. Über den „led = CRGB::Color“; Befehl werden die einzelnen LEDs dann in den gewünschten Zustand gebracht. [37]

2.3.4 Schaltpläne und Fertigungsunterlagen

Schematik

In den ersten Entwürfen des Schaltplans war eine hardwareseitige Aktivierung und Deaktivierung des Gesamtsystems mittels eines Tasters vorgesehen. Dieses Konzept basierte auf der Nutzung der integrierten Wake-Up-Pins des Raspberry Pi, welche das System bei entsprechender Beschaltung hochfahren sollten. Nach einer fachlichen Rücksprache mit dem Projektbetreuer der Firma Leitner wurde von diesem Ansatz jedoch abgeraten. Erfahrungswerte aus früheren Projekten des Unternehmens zeigten, dass diese Methode in der Praxis unzuverlässig und wartungsintensiv war. Das Herunterfahren des Systems erfolgt nun ausschließlich softwareseitig.

Ein weiteres ursprünglich geplantes Feature war die Implementierung von LeistungsmosFETs an der Masseleitung der peripheren Bauteile. Dies hätte es dem Raspberry Pi ermöglicht, einzelne Hardware-Komponenten gezielt ein- oder auszuschalten, um die Energieeffizienz zu steigern. Um jedoch die Systemkomplexität sowie potenzielle Fehlerquellen im Hinblick auf den engen Zeitplan zu minimieren, wurde auf dieses Feature verzichtet und die Komponenten direkt an die Masse angebunden.

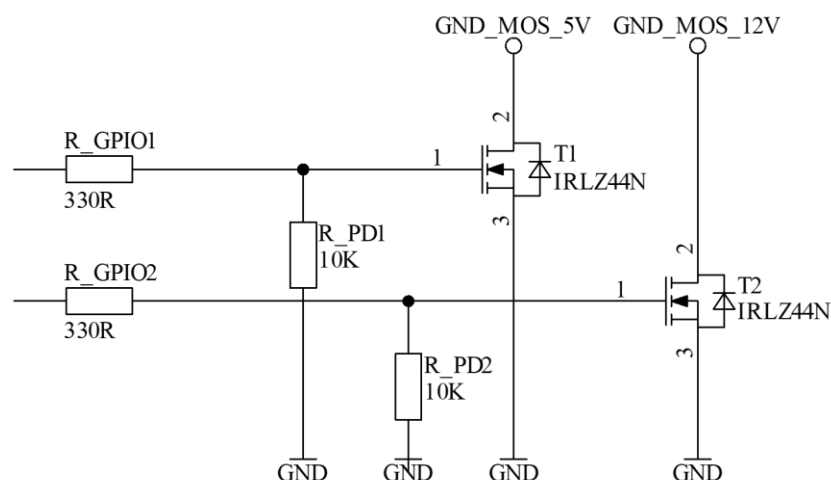


Abbildung 22: MOSFET Konzept Schaltung

Insgesamt wurden zwei Schaltpläne erstellt, um die elektrische Struktur des Systems vollständig zu dokumentieren. Der erste Plan fungiert als übergeordneter Gesamtstromlaufplan mit allen elektrischen Anschlüssen des gesamten Projekts. Diese Datei dient als Basis für die gesamte Verkabelung und soll auch dazu beitragen, dass das Gerät bei einem Hardware-Defekt schnell repariert werden kann. Dieser Schaltplan wurde als hierarchisches Design umgesetzt. Aufgrund der Vielzahl an integrierten Komponenten ermöglicht diese Strukturierung eine übersichtliche Darstellung der verschiedenen Funktionsgruppen und verbessert die Lesbarkeit des komplexen Schaltungsentwurfs erheblich.

Die zweite Schematik dokumentiert spezifisch alle Verbindungen und Komponenten der zentralen Leiterplatte. Diese detaillierte Darstellung ist für die Fertigung und Bestückung der Platine sowie für die Fehlersuche auf Bauteilebene essenziell.

Alle erstellten Schaltpläne befinden sich im Anhang.

Custom Libraries in Altium

Für den Gesamtstromlaufplan wurde eine Custom Symbolbibliothek in Altium Designer angelegt. Obwohl Portale wie SnapEDA oder UltraLibrarian eine umfangreiche Auswahl an fertigen Modellen bieten, erwies sich die Suche nach Bauteilen mit exakt passenden Spezifikationen und Pin-Belegungen als zeitintensiv. Um den Entwicklungsprozess zu beschleunigen und die Fehlerquote durch inkompatible Anschlüsse zu minimieren, wurden die benötigten Komponenten eigenständig generiert.

Da für die Schaltungsunterlagen keine Simulation der Bauteile erforderlich war, konnten die Symbole effizient mithilfe des Symbol Wizards erstellt werden. Dieses Werkzeug ermöglichte eine schnelle Definition der benötigten Pins sowie des dazugehörigen Layouts. [38]

Bsp. RGB-Ring:

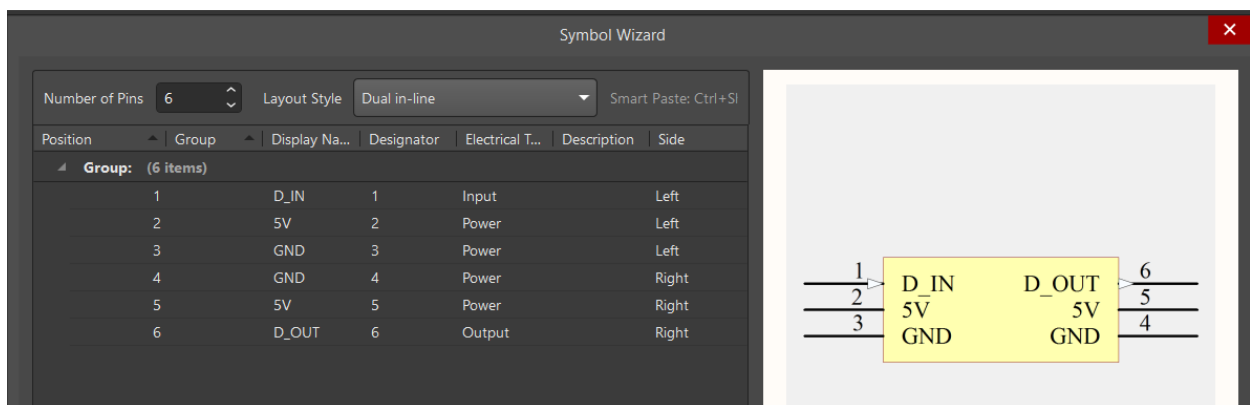


Abbildung 23: Symbol Wizard

Zur Vermeidung von Fehlermeldungen innerhalb der Schaltung wurde bei der Bauteilerstellung konsequent auf die korrekte Zuweisung der Electrical Types geachtet. Altium Designer nutzt diese Definitionen für den Electrical Rule Check, um die Logik der Verbindungen zu validieren. So erkennt das System beispielsweise unzulässige Verknüpfungen, wie das Zusammenschalten zweier Ausgangspins

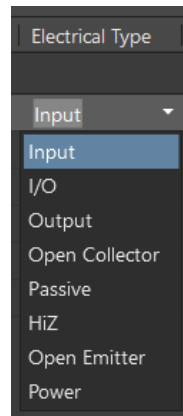


Abbildung 24: Electrical Types

PCB

Der primäre Zweck der Leiterplatte (PCB) ist die zentrale Energieversorgung und Signalverteilung für sämtliche Systemkomponenten. Um eine sichere und zugleich wartungsfreundliche Anbindung der Peripherie zu gewährleisten, wurden am Rand der Platine Schraubklemmen platziert.

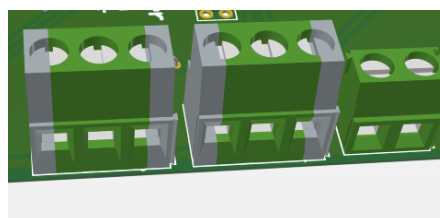


Abbildung 25: Schraubklemmen

Für die spezifischen Anforderungen der Hardware wurden folgende Schnittstellen auf dem Layout integriert:

- Eine USB-A-Buchse ermöglicht die direkte Stromversorgung des Raspberry Pi über die Platine.

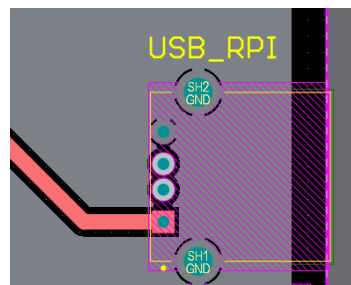


Abbildung 26: USB-A Anschluss

- Für den Lolin D1 Mini, den I2C-Multiplexer sowie die ADCs wurden Buchsenleisten vorgesehen. Dies erlaubt eine modulare Bestückung und einen einfachen Austausch der Komponenten.
- Ein dedizierter Steckverbinder kontaktiert den 40-Pin-Header des Raspberry Pi, um die Kommunikation zwischen der Steuereinheit und der Platine herzustellen.



Abbildung 27: 40 Pin-Header



Abbildung 28: 40 Pin-Kabel

Die finale Fertigung des Designs erfolgt durch den Export der Layoutdaten als PDF. Die Umsetzung wird als einseitige Leiterplatte in der schuleigenen Werkstatt der HTL Anichstraße realisiert.

Optionale Features

Auf der Platine wurden viele zusätzliche Anschlusspins verbaut, um mögliche andere Geräte zu versorgen. Außerdem wurde ein GPIO-Pin so beschalten, dass an einen Header Anschluss ein Schalter angesteckt werden kann, welcher so programmiert werden könnte, dass er signalisiert, ob die Tür geöffnet ist oder nicht. Dies wurde nicht weiterverfolgt, da es einerseits nie eine Vorgabe des Geräts war und schlussendlich auch die Zeit für zusätzliche Features gefehlt hat.

Button

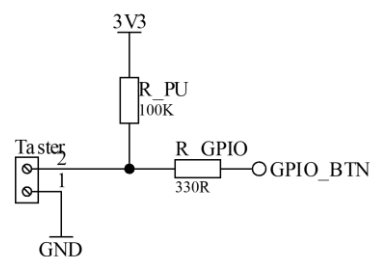


Abbildung 29: Optionaler Button

Zusätzlich wurde an den ADCs der DRDY Pin nach außen geführt. Dieser ist optional und gäbe ein Signal aus, wenn der ADC fertig gewandelt hat. Dieser ist für Debugzwecke gut geeignet, erfüllt im Normalbetrieb aber keine weitere Aufgabe. Deshalb wurde auch auf der Raspberry Pi Seite 3 GPIO-Pins nach außen geführt. Diese können bei Bedarf mit den DRDY-Pins verbunden werden, um Fehler zu analysieren.

2.3.5 Gehäuseentwicklung

Die mechanische Konstruktion des Systems unterteilt sich in zwei wesentliche Funktionsbereiche: die Gehäuse der dezentralen Kleingeräte sowie die zentrale Hauptgehäuseeinheit.

3D Design/Druck

Für sämtliche Kleingeräte – einschließlich der Kameras, der Blitzvorrichtungen, der zentralen Platine, des Raspberry Pi sowie des aktiven USB-Hubs – wurde jeweils ein individuelles Schutzgehäuse entwickelt. Aufgrund der kompakten Abmessungen dieser Komponenten eignet sich der 3D-Druck ideal als Fertigungsverfahren.

Die Erstellung der benötigten STL-Daten erfolgte mittels der CAD-Software Autodesk Inventor, welche umfassende Werkzeuge für die präzise 3D-Entwicklung bietet.

Das Kameragehäuse wurde dabei funktionspezifisch konstruiert: Es verfügt über passgenaue Aussparungen für die Kabelführung sowie einen frontseitig integrierten Ring zur Montage des RGB-Blitzlichts.

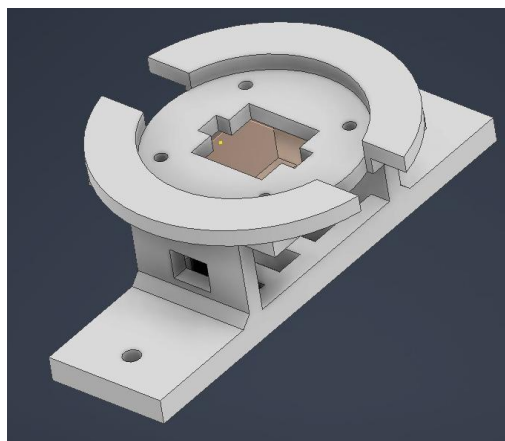


Abbildung 30: Kamera Gehäuse

Auch für die isometrische Kameraperspektive wurde eine maßgeschneiderte Halterung entwickelt.

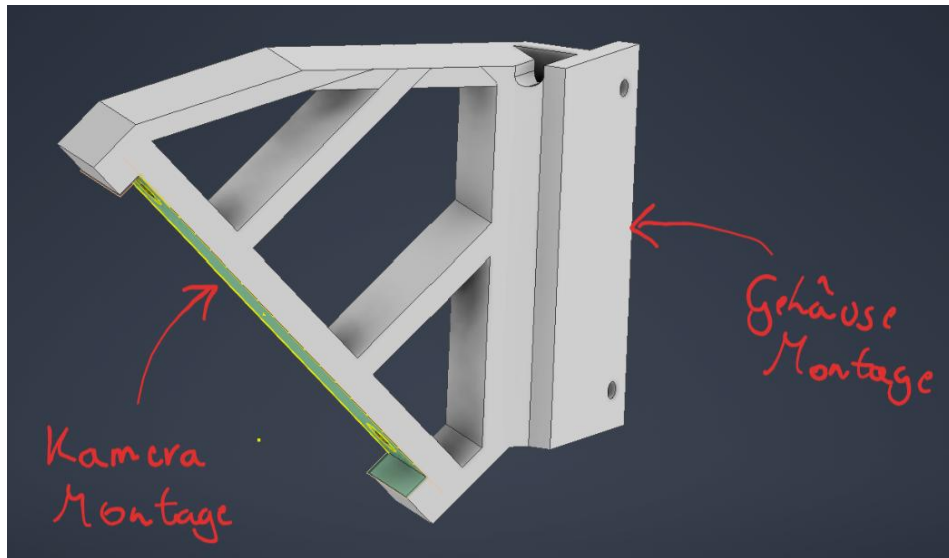


Abbildung 31: Isometrische Ansicht

Die Fertigung der Bauteile erfolgt im SLA-Resin-Druckverfahren. Um Materialverschwendung durch Fehldrucke zu minimieren, wurde vor Beginn der Produktionsphase eine vollständige Neukalibrierung des Druckers durchgeführt. Im Gegensatz zum FDM-Verfahren erfordert der Resindruck eine besonders sorgfältige Positionierung auf der Bauplattform sowie den Einsatz von Stützstrukturen, um ein späteres Ablösen der Modelle ohne Beschädigung zu ermöglichen. Nach Abschluss des Druckvorgangs werden die Bauteile in einer UV-Belichtungskammer nachgehärtet, um ihre finale mechanische Stabilität und Festigkeit zu erreichen.

Dimensionierung des Gehäuses

Die Grundmaße des Gehäuses werden direkt aus den optischen Parametern der Sensorik abgeleitet

Für die Bestimmung der Gehäusehöhe ist die minimale Messdistanz der Tiefenkamera das ausschlaggebende Kriterium, da diese erst ab einer Entfernung von etwa 25 cm valide Höhendaten liefert.

Diese minimale Entfernung ist mithilfe der Disparität begründbar. Bei der Stereo-Vision-Technik wird die Tiefe durch den Vergleich der Bilddaten zweier versetzter Kameralinsen ermittelt. Die horizontale Verschiebung eines Objekts zwischen beiden Bildern dient als Grundlage für die Entfernungsberechnung. Die Kamera-Firmware operiert dabei innerhalb eines fest definierten Suchbereichs für diese Disparität. Nähert sich ein Objekt der Kamera, vergrößert sich die Disparität exponentiell. Unterschreitet der Abstand den Schwellenwert von ca. 25 cm, liegt die Verschiebung außerhalb des vordefinierten Suchfensters, wodurch keine Tiefenberechnung mehr möglich ist.

Für die Konstruktion wird eine maximale Objekthöhe von 300 mm angenommen. Unter Berücksichtigung der minimalen Messdistanz und einer zusätzlichen Sicherheitstoleranz wurde daher ein vertikaler Abstand von 600 mm zwischen der Tiefenkamera und der Wägeplatte festgesetzt.

Für die Bestimmung der horizontalen Gehäusemaße ist primär die Brennweite der verwendeten Kameras ausschlaggebend. Ein zu geringer Abstand zum Messobjekt würde dazu führen, dass dieses nicht vollständig im Sichtfeld der Sensoren abgebildet werden kann. Um die vollständige Erfassung sicherzustellen, wurde mittels Winkelfunktionen der erforderliche Mindestabstand berechnet.

Als Grundlage für die Kalkulation wird eine zu erfassende Zielfläche von 400 mm x 400 mm (Länge x Breite) definiert.

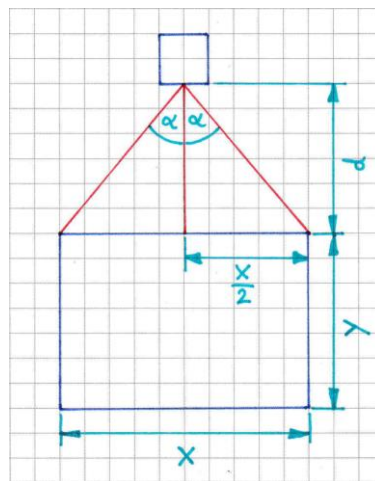


Abbildung 32: Skizze der Kamera Brennweite

$$\alpha = \frac{\text{Brennweite}}{2} = \frac{140^\circ}{2} = 70^\circ$$

$$x = 400\text{mm} \rightarrow \frac{x}{2} = 200\text{mm}$$

$$d = ?$$

$$\tan(\alpha) = \frac{\frac{x}{2}}{d}$$

$$d = \frac{\frac{x}{2}}{\tan(\alpha)} = \frac{200\text{mm}}{\tan(70^\circ)} = 72,8\text{mm}$$

Aus diesen Maßen lässt sich auf die Außenmaße des Gehäuses rückrechnen.

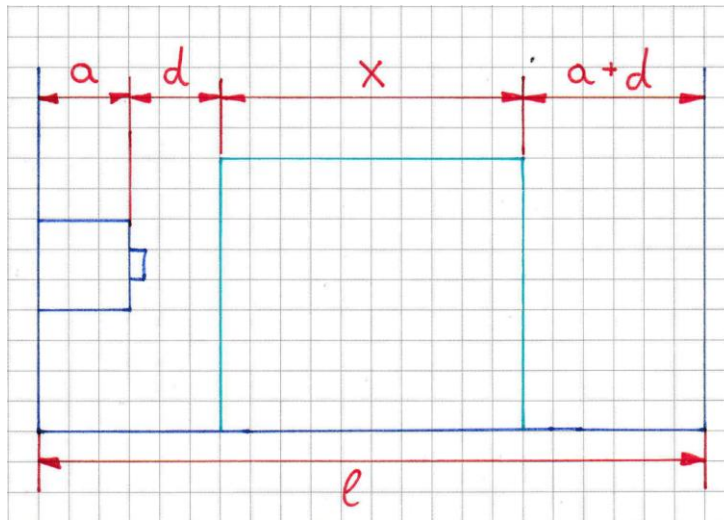


Abbildung 33: Skizze Gehäuse Maße

$$a = 40mm$$

$$d = 72,8mm$$

$$x = 400mm$$

$$l = x + (a + d) * 2 = 400 + (40 + 72,8) * 2 = 625,6mm$$

Um Toleranzen miteinzubeziehen, wird ein Außenmaß von 650mm x 650mm gewählt.

Gehäuseeinheit-Aufbau

Das Grundgerüst der Gehäuseeinheit ist als Holzkonstruktion aus Fichten-Vollholz ausgeführt, die sich funktional in zwei Rahmen unterteilt:

- Unterbau: Im unteren Rahmen wurden präzise Einfräsungen vorgenommen, um die Wägezellen bündig zu montieren. Diese Konstruktionsweise ermöglicht zudem eine geschützte Führung der Anschlussleitungen innerhalb der Aussparungen.

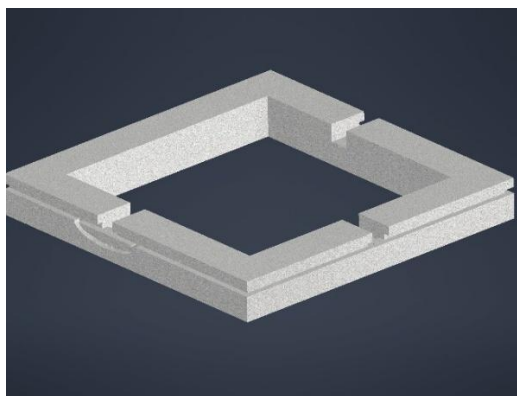


Abbildung 34: Unterbau

- Stützrahmen: Der obere Rahmen ist materialsparender dimensioniert und dient primär der Erhöhung der mechanischen Gesamtsteifigkeit sowie der Aufnahme der Wandelemente.

Für die Verkleidung werden 8 mm starke MDF-Platten eingesetzt. Diese verfügen über eine weiße, matte Oberfläche, die aufgrund ihrer Beschaffenheit kaum Lichtreflexionen aufweist.

Als Auflagefläche wurde ebenfalls eine 8 mm starke MDF-Platte gewählt. Sie wurde bewusst nicht fest mit den Wägezellen verschraubt. Da Vollholzrahmen auch nach der Montage auf Feuchtigkeits- und Temperaturänderungen reagieren, könnten bei einer starren Verbindung variable mechanische Spannungen entstehen. Diese würden unmittelbar auf die Sensoren wirken und das Messergebnis verfälschen.

Um dies zu verhindern, wurden an den Auflagepunkten der Wägezellen Bohrungen mit genügend Spiel in der Platte vorgenommen. Die Wägeplatte wird lediglich auf die Sensoren aufgelegt. Diese Methode gewährleistet eine ausreichende mechanische Toleranz und stellt sicher, dass ausschließlich die Gewichtskraft des Messobjekts und nicht die Materialspannung des Gehäuses erfasst wird.



Abbildung 35: Wägeplatte



Abbildung 36: Wägezelle

Um den elektronischen Bauteilen ausreichend Raum zu bieten und eine geordnete Kabelführung zu ermöglichen, wurde das Gehäuse mit einer doppelten Rückwand konstruiert. Dieser Zwischenraum dient als geschützte Montageebene für die Verkabelung sowie für alle nicht-sensorischen Elektronikkomponenten.

Den äußeren Abschluss der Konstruktion bildet eine passgenaue Tür, die den Zugang zur Hardware für Wartungszwecke erleichtert. Zur Vermeidung von Wärmestau wurde zudem ein Lüftungsgitter integriert.



Abbildung 37: Verkleidung Rückseite

2.3.6 Gehäusemontage

Nach der Fertigstellung des Hauptgehäuses und der individuellen Gehäusekomponenten, erfolgt eine finale Zusammensetzung aller Komponenten. In diesem Prozessschritt werden die erforderlichen Kabel- und Leitungsquerschnitte basierend auf der berechneten Strombelastbarkeit dimensioniert. Die folgende Tabelle dient als verbindliche Grundlage für die elektrische Installation [24]:

AWG	Durchmesser		Querschnitt		R (Ω /km)
	Zoll	mm	kcmil	mm ²	
0000 (4/0)	0,46	11,68	211,6	107,2	0,166
000 (3/0)	0,4096	10,4	167,8	85,03	0,2093
00 (2/0)	0,3648	9,266	133,1	67,43	0,264
0 (1/0)	0,3249	8,251	105,5	53,48	0,3329
1	0,2893	7,348	83,69	42,41	0,4197
2	0,2576	6,544	66,37	33,63	0,5293
3	0,2294	5,827	52,63	26,67	0,6674
4	0,2043	5,189	41,74	21,15	0,8416
5	0,1819	4,621	33,1	16,77	1,061
6	0,162	4,115	26,25	13,3	1,338
7	0,1443	3,665	20,82	10,55	1,687
8	0,1285	3,264	16,51	8,366	2,128
9	0,1144	2,906	13,09	6,634	2,683
10	0,1019	2,588	10,38	5,261	3,383
11	0,09074	2,305	8,234	4,172	4,266
12	0,08081	2,053	6,53	3,309	5,38
13	0,07196	1,828	5,178	2,624	6,784
14	0,06408	1,628	4,107	2,081	8,554
15	0,05707	1,45	3,257	1,65	10,79
16	0,05082	1,291	2,583	1,309	13,6
17	0,04526	1,15	2,048	1,038	17,15
18	0,0403	1,024	1,624	0,823	21,63
19	0,03589	0,9116	1,288	0,6527	27,27
20	0,03196	0,8118	1,022	0,5176	34,39
21	0,02846	0,7229	0,8101	0,4105	43,36
22	0,02535	0,6438	0,6424	0,3255	54,68
23	0,02257	0,5733	0,5095	0,2582	68,95
24	0,0201	0,5106	0,404	0,2047	86,94
25	0,0179	0,4547	0,3204	0,1624	109,6
26	0,01594	0,4049	0,2541	0,1288	138,2
27	0,0142	0,3606	0,2015	0,1021	174,3
28	0,01264	0,3211	0,1598	0,08098	219,8
29	0,01126	0,2859	0,1267	0,06422	277,2
30	0,01003	0,2546	0,1005	0,05093	349,5
31	0,008928	0,2268	0,0797	0,04039	440,7
32	0,00795	0,2019	0,06321	0,03203	555,8
33	0,00708	0,1798	0,05013	0,0254	700,8
34	0,006305	0,1601	0,03975	0,02014	883,7
35	0,005615	0,1426	0,03152	0,01597	1114
36	0,005	0,127	0,025	0,01267	1405
37	0,004453	0,1131	0,01983	0,01005	1772
38	0,003965	0,1007	0,01572	0,007967	2234
39	0,003531	0,08969	0,01247	0,006318	2817
40	0,003145	0,07987	0,009888	0,00501	3553

Abbildung 38: Querschnitt-Tabelle

Da die Beleuchtungselemente bei voller Helligkeit eine signifikante Stromaufnahme aufweisen, müssen die Leitungsquerschnitte ausreichend groß dimensioniert werden. Dies minimiert den Spannungsabfall über die Leitungslänge und verhindert eine unzulässige Erwärmung der Drähte.

Es handelt sich beim RGB-Streifen um ca. 100 RGBs, die jeweils auf ca. die Hälfte der maximalen Lichtstärke gestellt werden. Rechnet man mit ca. 20mA pro LED erhält man für den gesamten Streifen:

$$100 * 20mA = 2A$$

Und für die Ringe mit 16 RGBs pro Ring:

$$16 * 3 * 20mA = 960mA$$

Über das Ohm'sche Gesetz lässt sich der maximal zulässige Leitungswiderstand berechnen, aus dem anschließend der Mindestquerschnitt abgeleitet wird, wenn nicht mehr als 0,2V an den Drähten anliegen darf:

$$R_{Draht} = \frac{0,2V}{2A} = 0,1\Omega$$

Aus der Tabelle müssen die Widerstandswerte nun auf $\frac{\Omega}{m}$ umgerechnet werden. Die Drähte werden ca. 60-70cm lang sein. Da es wenig unterschied macht, wird eine Länge von 1m angenommen

$$R_m = \frac{R_{km}}{1000}$$

Der erste Wert der dann unter die 0,1 Ω fällt sind 0,0869 bei einer Leiterstärke von 24AWG. Diese Leiterstärke könnte theoretisch verwendet werden. Es ist aber ratsamer die Leitungen leicht zu überdimensionieren, deshalb werden die nächstgrößeren 22AWG Drähte verwendet.

Neben der Versorgung gibt es bei der Beleuchtung noch die Datenleitung. Bei dem WS2812 RGB Controller ist aber weniger wichtig wie viel Strom durch die Leitung fließt, da es nur sehr wenig ist, sondern dass die Spannung so stabil wie möglich ist. Als Leitungsdicke kann auch 22AWG genommen werden, aber es ist wichtig einen Widerstand von ca. 330Ω vorzuschalten damit dieser die Signalfanken glättet. Wichtig ist auch, dass die Leitung nicht zu lang wird, da der Spannungsabfall an dieser ab einer gewissen Leitungslänge zu hoch ist und das erzeugte PWM-Signal nicht mehr geeignet ist für den Controller.

Weitere wichtige Kabel sind die Versorgungen vom Netzteil zur Platine. Da beim Einschalten des Raspberry Pi ein Strom von bis zu 5A fließen kann, werden diese Querschnitte mit $2,5\text{ mm}^2$ sehr groß gewählt. Das USB-Kabel zum Raspberry Pi muss auch entsprechend gewählt werden, dass gegebenenfalls ein Strom von 5A fließen könnte.



Abbildung 39: Verkabelung Platine

Weitere Bilder finden sich im Anhang.

2.4 Weitere Individuelle Arbeitsschritte

Connection per VPN

Eine fertige Hardware hat wenig Sinn, wenn keine Software darauf gespielt werden kann, um das Gerät zu testen. Damit das Gerät aber nicht immer zwischen den Projektpartnern hin und her gereicht werden muss, wird eine temporäre VPN-Verbindung hergestellt.

Genutzt wurde hier der Service Tailscale, welcher eine intuitive und einfache Anwendung ermöglicht. In Tailscale wird anfangs ein Account erstellt. Der Service wird anschließend auf allen Geräten, die sich im VPN-Netz befinden sollten, installiert. Dort muss sich dann nur mehr mit dem zuvor erstellten Account eingeloggt werden und schon ist man mit dem Netzwerk verbunden.

Ein kleiner Stolperstein bei diesem System war der Raspberry Pi selbst. Dieser muss nämlich, um die Verbindung zu ermöglichen an das Internet angeschlossen werden. Der Raspberry Pi verhält sich jedoch nicht immer wie erwartet. So muss oft manuell das LAN-Interface gestartet werden, da er den Befehl, der eigentlich auch ins Startup geschrieben worden ist, nicht ausführt.

2.5 Zusammenfassung individueller Teil

Im Rückblick auf die Hardware-Entwicklung kann festgehalten werden, dass die gesetzten Projektziele vollumfänglich erreicht wurden. Der Prozess begann mit einer intensiven Recherchephase, in der alle benötigten Komponenten nicht nur identifiziert, sondern auch präzise auf die spezifischen Anforderungen des Systems abgestimmt wurden. Die daraus resultierenden Schaltpläne bildeten das Rückgrat für die spätere praktische Umsetzung, welche schließlich in einer strukturierten Gesamtschaltlogik sowie einer funktionalen, selbst gefertigten Leiterplatte endete. Auch im Bereich des Mechanischen konnte mit dem Aufbau eines stabilen Gehäuses, das den berechneten Größenanforderungen entspricht, ein wesentlicher Meilenstein erfolgreich abgeschlossen werden.

Über die rein technische Umsetzung hinaus bot die Durchführung dieser Arbeit einen erheblichen persönlichen und fachlichen Erkenntnisgewinn in den verschiedensten Disziplinen. Bereits in der ersten Phase wurde ein starker Fokus auf die Projektplanung gelegt, was wertvolle Einblicke in die professionelle Strukturierung von Industrieprojekten ermöglichte. Ein zentraler fachlicher Schwerpunkt lag in der Auseinandersetzung mit der Sensorik. Hier wurden zahlreiche Messverfahren zur Erfassung unterschiedlichster physikalischer Größen tiefgreifend erforscht und evaluiert. Die Entdeckung innovativer Hersteller und vielversprechender Start-ups bereicherte diesen Prozess zusätzlich. In der praktischen Implementierung wurden zudem kritische Parameter der Sensortechnik vertieft, wie beispielsweise der Einfluss von Oberflächenbeschaffenheiten auf die Messgenauigkeit oder die Auswirkungen von Leitungsquerschnitten auf die Signalqualität. Zusammenfassend lässt sich das Projekt daher als ein sowohl technischer als auch konzeptioneller Erfolg werten, der ein tiefes Verständnis für das komplexe Zusammenspiel von Elektronik, Mechanik und Planung vermittelt hat.

3 Dokumentation Granegger Felix

3.1 Einleitung

Im Rahmen der Diplomarbeit wird die Entwicklung der Software für ein mobiles System zur automatisierten Erfassung von Artikeldaten im Lagerbetrieb beschrieben. Ziel ist die Erstellung einer strukturierten Artikel-Datenbank sowie die Inbetriebnahme eines funktionsfähigen Prototyps mit vollständig integrierter Software.

Zentrale Inhalte sind die sensor- und kamerabasierte Bestimmung der Artikelmaße (Länge, Breite, Höhe) einschließlich der Berechnung möglicher Messabweichungen, die automatische Erkennung und Auswertung von QR- und Barcodes zur fehlerarmen Datenerfassung sowie die präzise Gewichtsermittlung über integrierte Waagen oder Gewichtssensoren.

Darüber hinaus wird ein intuitives grafisches Benutzerinterface entwickelt, das den Arbeitsablauf im Lager unterstützt. Ein weiterer Schwerpunkt liegt auf einem zuverlässigen Datenexport mit Integration in ein vorgegebenes SAP-System. Die Softwarearchitektur soll zudem eine hohe Ausfallsicherheit gewährleisten und Mechanismen zur schnellen Fehlersuche sowie Wiederherstellung bei Störungen bereitstellen.

Als geplantes Ergebnis entsteht eine vollständig integrierte Softwarelösung, welche die Sensoransteuerung, Datenauswertung und Visualisierung vereint und den stabilen Betrieb des Prototyps sicherstellt.

3.2 Theoretische Grundlagen

Für den Aufbau eines automatischen 3D-Scanner-Systems, das Artikel im Lager vermisst, wiegt und deren Barcodes liest, sind fundierte Kenntnisse über die Funktionsweise der einzelnen Technologien erforderlich. Dieses Kapitel behandelt die theoretischen Grundlagen und dient dazu, die technischen Konzepte sowie deren mögliche Umsetzungen zu verstehen. Eine konkrete Implementierung in Form von Software-Code ist nicht Bestandteil dieses Kapitels.

Zu Beginn dieses Kapitels wird die Funktionsweise eines Stereo-Vision-Systems zur Erfassung räumlicher Tiefeninformationen erläutert. Anschließend folgt eine Einführung in die Bildverarbeitung, insbesondere in die Verfahren zur Erkennung und Dekodierung von QR-Codes und Barcodes. Daraufhin wird die Gewichtserfassung mittels Wägezelle behandelt. Ergänzend dazu wird das Datenmanagement der erfassten Messdaten beschrieben, insbesondere im Hinblick auf deren strukturierte Speicherung und Weiterverarbeitung im Gesamtsystem. Abschließend wird das Interface des Systems beschrieben, das die einzelnen Komponenten koordiniert und zu einem strukturierten Scanprozess zusammenführt.

Zusätzlich werden verschiedene theoretische Umsetzungsansätze sowie deren praktische Einsatzmöglichkeiten betrachtet und die zu erwartenden Messgenauigkeiten der einzelnen Komponenten analysiert.

3.2.1 Tiefenmessung mit Stereo-Vision

Grundprinzip

Bei der Stereo-Vision werden zwei Kameras mit einem fest definierten Abstand voneinander montiert. Dieser Abstand wird als Baseline bezeichnet und ist eine wesentliche Größe für die Tiefenberechnung. Dieses Verfahren ist mit dem menschlichen Auge vergleichbar.

Die beiden Kameras erfassen somit ein Objekt über zwei verschobene Bilder. Aufgrund der unterschiedlichen Blickwinkel wird ein bestimmter Punkt eines Objektes in den beiden Bildern nicht an der exakt selben Pixelposition dargestellt. So entsteht eine horizontale Verschiebung zwischen linkem und rechtem Bild. Diese Verschiebung wird als Disparität bezeichnet.

Die Größe der Disparität hängt direkt von der Entfernung des Objektes ab. Befindet sich ein Objekt näher an den Kameras, so ist die Verschiebung größer. Mit zunehmender Entfernung nimmt die Disparität entsprechend ab. Aus dieser geometrischen Beziehung lässt sich mithilfe der bekannten Kameraparameter die Tiefe beziehungsweise der Abstand des Objektes berechnen.

Mit folgender Formel kann man mit Stereo-Vision die Entfernung berechnen:

$$\text{Tiefe} = \frac{\text{Brennweite} \times \text{Baseline}}{\text{Disparität}}$$

- Brennweite (focalLength): Gibt an, wie stark die Linse das Licht bündelt (kann aus der Kamera ausgelesen werden mit einem Befehl).
- Kameraabstand (Baseline): Der Abstand zwischen den beiden Kameralinsen.
- Disparität: Die gemessene Pixelverschiebung.

Für eine präzise Tiefenmessung mit dem 3D-Scanner, insbesondere bei festen Abständen zu den Objekten, müssen mehrere Faktoren berücksichtigt werden:

- Kameraabstand: Der Abstand zwischen den Kameras (Baseline) beeinflusst direkt die Disparität und damit die Messgenauigkeit.
- Bildauflösung: Bei bestimmten Distanzen müssen niedrigere bzw. höhere Auflösung verwendet werden.
- Oberfläche des Objekts: Eine einfarbige oder transparente Oberfläche erschwert der Kamera die zuverlässige Darstellung der Punkte, da Strukturen fehlen und Lichtreflexionen auftreten können.
- Ungünstige Lichtverhältnisse: Zu wenig Licht oder Schatten verschlechtern die Aufnahme durch einen geringeren Kontrast.

Für die Tiefenmessung wird eine OAK-D S2 AF Kamera (siehe Abbildung) eingesetzt. Sie verfügt über drei integrierte Kameras sowie einen internen Chip zur direkten Tiefenberechnung. Zu den technischen Kenndaten der Tiefenkamera zählen eine Basislinie von 7,5 cm und eine Auflösung von bis zu 1920×1080 Pixel.



Abbildung 40: OAK-D S2 AF [30]

Die zugehörige Herstellerdokumentation liefert eine umfassende Beschreibung der Kamera und ihrer Funktionen. Zunächst wird erläutert, wie sich kamerainterne Parameter wie die Brennweite auslesen lassen und wie daraus Tiefeninformationen für einzelne Pixel berechnet werden. Darüber hinaus werden der grundlegende Aufbau, die Funktionsweise sowie typische Einsatzbereiche der Kamera

beschrieben. Ergänzend dazu enthält die Dokumentation detaillierte Informationen zur Konfiguration der Kamera, beispielsweise zur Anpassung von Filtereinstellungen zur Reduktion von Bildrauschen. Die Steuerung wird sowohl für Python als auch für C++ dargestellt, wobei insbesondere die korrekte Ansteuerung des integrierten Prozessors der RVC2-Plattform zu beachten ist. [39]

Besonders relevant ist *Punkt 5* der Dokumentation, der sich mit der *Short-Distance Stereo Depth* beschäftigt. Hier wird erklärt, wie die Minimaldistanz (MinZ) zur Kamera berechnet wird, also die kleinste Entfernung, bei der ein Objekt noch zuverlässig erfasst werden kann:

$$\text{MinZ} = \frac{\text{Brennweite} \cdot \text{Baseline}}{95}$$

MINIMALDISTANZ	~ 20 CM	~ 40CM	~ 80 CM
AUFLÖSUNG P IN PIXEL	400P + extended disp	400P or 800P + extended	800 P

Angaben vom Hersteller von Pixel Auflösung [30]

Beispielhaft berechnet für die OAK-D S2 AF:

$$\text{MinZ [400P]} = \frac{441 \cdot 7,5\text{cm}}{95} \approx 35 \text{ cm}$$

Der *Stereo Extended Disparity Mode* reduziert die minimale Messtiefe von 35 cm weiter runter, sodass auch Objekte in sehr geringem Abstand zur Kamera erfasst werden können. Allerdings erfolgt diese Erweiterung auf Kosten der Messgenauigkeit.

Limitierungen der Distanzmessung:

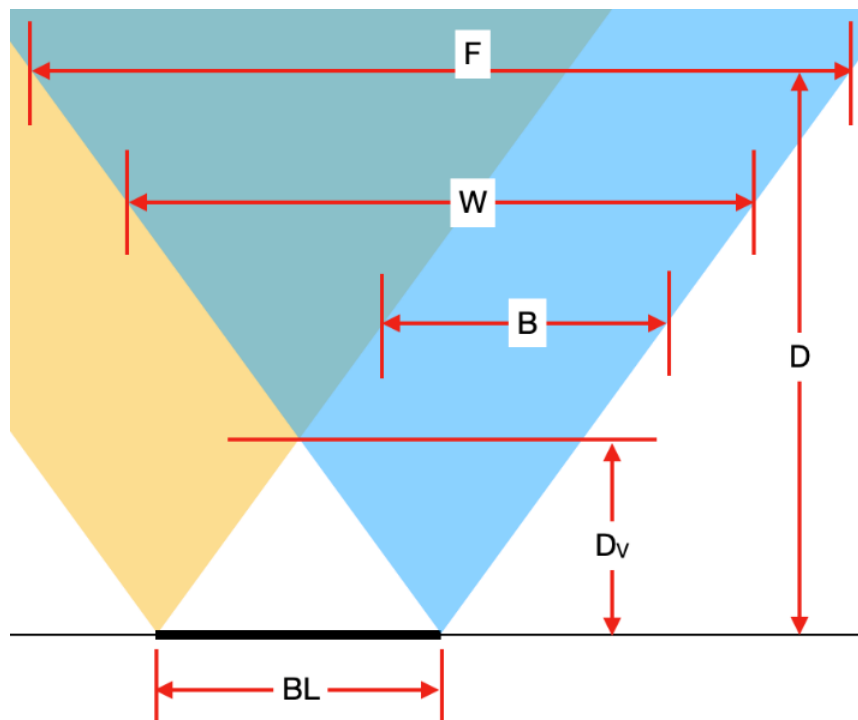


Abbildung 41: Limitierung der Distanzmessung von Luxonis Dokumentation [39]

Legende:

- BL [cm] – Abstand (Baseline) der beiden Kameras
- Dv [cm] – Minimale Distanz, bei der beide Kameras ein Objekt erfassen können (Depth-Berechnung möglich)
- B [px] – Bereich in dem keine Tiefenwerte berechnet werden können
- W [px] – Breite des Kamerabildes in Pixeln (Horizontal Pixels)
- D [cm] – Entfernung von der Kameraposition zu einem Objekt
- F [cm] – Bildbreite in der Entfernung D

Aus diesen Informationen lassen sich zwei mögliche Verfahren zur Bestimmung von Objektanmessungen ableiten:

- Punktwolke und dreidimensionale Geometrie
- Distanzabhängige Pixel-Umrechnung

Punktwolke und dreidimensionale Geometrie

Das Ergebnis der Tiefenmessung ist typischerweise ein Tiefenbild, bei dem jedem Pixel ein Entfernungswert zur Kamera zugeordnet ist. Durch die Umrechnung dieser Werte in ein kartesisches Koordinatensystem entsteht eine sogenannte Punktwolke. Eine Punktwolke besteht aus einer Vielzahl einzelner 3D-Punkte, die zusammen die Form und Oberfläche eines Objekts beschreiben. Je dichter die Punkte, desto präziser kann die Objektgeometrie erfasst werden. [40]

Für die Vermessungen werden charakteristische Kenngrößen der Punktwolke genutzt. Dazu zählen die minimale und maximale Ausdehnung entlang der drei Raumachsen (X, Y, Z), um Länge, Breite und Höhe zu bestimmen. Außerdem wird eine Bounding Box erstellt, die das Objekt in allen Richtungen umschließt und die dreidimensionalen Maße direkt liefert. Zur Verbesserung der Genauigkeit werden Ausreißer gefiltert, wodurch fehlerhafte Messpunkte entfernt und die Stabilität der geometrischen Berechnung erhöht werden.

Für die Filterung kommen verschiedene Methoden zum Einsatz:

- Mittelwertbildung, um einzelne extreme Messpunkte auszugleichen
- Medianfilter, der besonders robust gegenüber vereinzelt Ausreißern ist
- Statistische Ausreißerentfernung, bei dem Punkte, die weit außerhalb der Standardabweichung liegen, entfernt werden

Die Qualität der Punktwolke hängt stark von der Auflösung des Tiefenbildes, der Kalibrierung der Kamera sowie den Licht- und Oberflächenbedingungen ab. Glatte oder transparente Oberflächen können Messungen erschweren, da die Kamera weniger verlässliche Tiefenwerte liefert.

Distanzabhängige Pixel-Umrechnung

Die Abmessungen eines Objekts werden über Pixel bestimmt. Dabei entspricht die Länge oder Breite der Anzahl an Pixeln zwischen zwei erkannten Objektkanten. Diese Pixelwerte stellen jedoch noch keine realen Maße dar, sondern lediglich Bildkoordinaten.

Um daraus reale Längen in Millimetern zu berechnen, werden Kameraparameter wie die Brennweite sowie ein definierter Referenzabstand zur Grundfläche benötigt. Die Umrechnung erfolgt auf Basis des perspektivischen Kameramodells, das den Zusammenhang zwischen Objektentfernung und Bildgröße beschreibt. Eine vorherige Kalibrierung ist daher Voraussetzung für eine korrekte Maßberechnung.

Für die Höhenberechnung wird eine Referenzebene definiert, die der Auflagefläche entspricht. Aus dem Tiefenbild wird dafür ein charakteristischer Tiefenwert dieser Grundfläche als Referenz bestimmt. Anschließend werden die Tiefenwerte der zum Objekt gehörenden Pixel mit diesem Referenzwert verglichen. Die Höhe des Objekts ergibt sich aus der Differenz zwischen dem Tiefenwert der Grundfläche und dem Tiefenwert der obersten Objektpunkte. Dadurch kann die vertikale Ausdehnung des Objekts relativ zur Auflagefläche berechnet werden.

Aus dieser abgegrenzten Fläche werden die Ausdehnungen des Objekts in Bildrichtung ermittelt und in Pixeln gemessen. Anschließend erfolgt die Umrechnung dieser Pixelwerte in reale Längen. Dafür werden die zugehörigen Tiefeninformationen der Pixel sowie die bekannten Kameraparameter, wie Brennweite und Auflösung, verwendet. Dadurch lassen sich die tatsächlichen Abmessungen des Objekts im Raum bestimmen.

3.2.2 Bildverarbeitung und Barcode-Dekodierung

Um auf einem Artikel neben den Maßen auch einen QR-Code oder Barcode zu lesen, muss die Software drei aufeinanderfolgende Schritte durchführen: zunächst die Bildvorverarbeitung, anschließend die YOLO-Erkennung (You Only Look Once) auf dem Originalbild zur Lokalisierung des Codes und abschließend die Dekodierung des Inhalts innerhalb der von YOLO bereitgestellten Bounding Box.

Eine wesentliche Voraussetzung für präzise Ergebnisse ist eine gute Bildqualität. Die Kameraauflösung spielt dabei eine entscheidende Rolle. Zudem können durch eine Kalibrierung intrinsische Parameter wie Brennweite, Hauptpunkt und Verzerrungskoeffizienten bestimmt werden, um geometrische Verzerrungen rechnerisch zu korrigieren.

Damit die späteren Schritte zuverlässig funktionieren, müssen die Vorverarbeitungsparameter an die konkreten Gegebenheiten angepasst werden. Dazu gehören:

- **Padding** um die von YOLO gelieferte Bounding Box: Ein zusätzlicher Rand schafft Toleranz, falls der Code nicht exakt mittig erkannt wurde oder die Box zu knapp bemessen ist.
- **Skalierung** des Bildausschnitts: Codes können in unterschiedlichen Größen vorliegen; eine Vergrößerung (z. B. auf das 1,5- oder 2-Fache) kann die Erkennung kleiner Codes verbessern.
- **Rotation** des Ausschnitts: Da Codes auf dem Artikel verdreht sein können, werden verschiedene Winkel getestet (für aufrechte und kopfstehende Lagen)
- **Vorverarbeitungsmethoden** mit OpenCV, um die Dekodierung der Barcodes oder QR-Codes zu erleichtern

Vorverarbeitungsmethoden mit OpenCV

Die Grundlage der Bildanalyse ist die Python-Bibliothek OpenCV. Sie stellt vielfältige Funktionen bereit, um Bilder für die weitere Verarbeitung vorzubereiten, insbesondere die Umwandlung in Graustufen, die für die anschließende Dekodierung vorteilhaft ist:

- **CLAHE:** Kontrastverstärkung durch adaptives Histogrammgleichverfahren, angewendet auf Graustufenbilder, um Details in hellen und dunklen Bildbereichen hervorzuheben.
- **Unsharp:** Schärfung von Graustufenbildern mittels Unsharp-Masking, um Kanten und Strukturen zu betonen.
- **Invert:** Invertierung der Graustufenwerte, um Hell-Dunkel-Muster umzukehren und die Dekodierung von Codes zu erleichtern.

Die folgenden Abbildungen zeigen die Effekte dieser Vorverarbeitungsmethoden auf ein Beispielbild:



Abbildung 42: Verarbeitetes Originalbild



Abbildung 43: Verarbeitetes CLAHE-Bild



Abbildung 44: Verarbeitetes Unsharp-Bild



Abbildung 45: Verarbeitetes Invert-Bild

Diese Vorverarbeitung erfolgt erst nach der YOLO-Erkennung, da das Modell Farbinformationen benötigt.

Lokalisierung des Codes mit YOLO

Nach der Vorverarbeitung muss der Code im Bild gefunden werden. Klassische Bildverarbeitung könnte nach kontrastreichen Stellen, rechteckigen Formen oder spezifischen Mustern suchen. Diese Methoden sind jedoch anfällig, wenn der Code schräg, teilweise verdeckt oder schlecht beleuchtet ist.

Ein moderner, robusterer Ansatz nutzt künstliche Intelligenz: das neuronale Netz YOLO. Das in [41] beschriebene YOLO-Modell wurde mit tausenden Bildern trainiert, in denen Barcodes und QR-Codes markiert waren. Es kann in einem einzigen Durchlauf alle Codes im Bild lokalisieren und ist dabei unempfindlich gegen Drehung und Größenänderung. Es liefert für jeden erkannten Code eine Bounding Box, also die Umrandung der Bildkoordinaten des Barcodes.

Dekodierung mit pyzbar / ZBar

Sobald die Position des Codes bekannt ist, wird der entsprechende Bildausschnitt extrahiert. Nun muss der eigentliche Inhalt des Codes gelesen werden. Dafür kommt die Python-Bibliothek pyzbar zum Einsatz, die eine Schnittstelle zur etablierten C-Bibliothek ZBar bietet. ZBar analysiert die Hell-Dunkel-Muster im Bildausschnitt und wandelt sie zurück in Text oder Zahlen – unabhängig davon, ob es sich um einen eindimensionalen Strichcode (z. B. EAN, UPC) oder einen zweidimensionalen QR-Code handelt. [42]

Damit das Lesen zuverlässig und konstant funktioniert, ist eine gute Beleuchtung unerlässlich. Im praktischen Aufbau wird daher eine LED-Beleuchtung um die Kamera angebracht, um gleichmäßige Ausleuchtung und hohen Kontrast zu gewährleisten.

3.2.3 Gewichtserfassung

Neben den geometrischen Abmessungen kann für eine vollständige Artikelerfassung auch das Gewicht herangezogen werden. Die konkrete Auswahl und Umsetzung der dafür notwendigen Sensortechnik stellt dabei einen eigenen Schwerpunkt innerhalb des Gesamtprojekts dar.

Grundsätzlich basiert die elektronische Gewichtserfassung auf Sensoren, die eine mechanische Belastung in ein elektrisches Signal umwandeln. Dieses Signal wird anschließend verstärkt, digitalisiert und dem übergeordneten System zur Weiterverarbeitung bereitgestellt. Zur Digitalisierung wird ein Analog-Digital-Wandler eingesetzt, der eine stabile und ausreichend hochauflösende Erfassung der Messwerte ermöglicht. Für die theoretische Betrachtung ist vor allem relevant, dass die Gewichtsdaten als zusätzlicher Parameter in die Gesamterfassung eines Artikels integriert werden können. [10]

Da Messsignale von Gewichtssensoren häufig durch Rauschen, Vibrationen oder kurzzeitige Belastungsänderungen beeinflusst werden, kann softwareseitig ein Tiefpassfilter implementiert werden. Ein solcher Filter reduziert schnelle Signaländerungen und glättet den Messwertverlauf, wodurch eine stabilere und reproduzierbare Gewichtsanzeige erreicht wird. [20]

Die erreichbare Messgenauigkeit hängt dabei allgemein von mechanischen, sensorischen und kalibrierungstechnischen Einflussgrößen ab.

$$\text{Umrechnungs Faktor in Gramm} = \frac{\text{Bekanntes Gewicht in Gramm}}{\text{Rohwert der Wägezelle}}$$

$$\text{Gewicht} = (\text{Rohwert} - \text{Nullpunkt}_{\text{Offset}}) * \text{Umrechnungs Faktor in Gramm}$$

Die detaillierte hardwareseitige Realisierung wird in Abschnitt **2.2.3 „Wägetechnik“** beschrieben.

3.2.4 Datenmanagement

Die vom 3D-Scanner erfassten Informationen – wie Gewicht, Abmessungen und Barcodes der Objekte – werden strukturiert gespeichert, um eine zuverlässige Weiterverarbeitung zu ermöglichen. Diese geordnete Ablage stellt sicher, dass die Daten jederzeit für Auswertungen, Vergleiche oder den späteren Import in das SAP-System der Firma Leitner bereitstehen.

Als Standardformate werden CSV oder JSON genutzt. Diese Formate sind maschinenlesbar, leicht automatisierbar und ermöglichen eine einfache Integration in ERP-Systeme. Außerdem sorgt eine einheitliche Struktur dafür, dass neue Messgrößen oder Sensoren ohne großen Aufwand hinzugefügt werden können.

Die Firma Leitner stellt hierfür eine vorgegebene Tabellenstruktur bereit, die für einen fehlerfreien Import in das SAP-System umgesetzt werden muss:

Interne Material-nummer	Gewicht [kg]	Abmessung [mm] Länge x Breite x Höhe			EAN-Code	Iso Bild Path
1001001	0.120	30	80	100	1234567890128	Scans/ <u>date</u> /Fotos /Scan_ <u>date</u> _ <u>time</u> / iso_Bild.jpg

Vorgegebene Struktur vom *Leitner* für die Integration des SAP-Systems

Diese strukturierte Speicherung gewährleistet, dass Messergebnisse jederzeit reproduzierbar sind und für Auswertungen oder Vergleiche genutzt werden können. Gleichzeitig reduziert sie Fehlerquellen und ermöglicht einen direkten Import, wodurch Arbeitsaufwand und Zeitaufwand deutlich minimiert werden.

3.2.5 User Interface für die Datenverarbeitung

Die grafische Benutzeroberfläche (GUI) bildet die zentrale Schnittstelle zwischen dem Anwender und dem 3D-Scanner. Sie ermöglicht eine strukturierte Darstellung der Messwerte, die gezielte Steuerung der Hardware und die einfache Diagnose von Fehlern während der Datenerfassung. In industriellen Systemen erhöht eine GUI die Übersichtlichkeit, erleichtert die Bedienung komplexer Abläufe und trägt dazu bei, dass Messprozesse effizient und fehlerfrei durchgeführt werden können.

Die Wahl einer grafischen Benutzeroberfläche gegenüber einer reinen Kommandozeilensteuerung (CLI) bietet mehrere Vorteile: Eine GUI kann Ergebnisse visuell darstellen, ermöglicht die parallele Anzeige mehrerer Prozessschritte, erlaubt die Integration von Statusanzeigen, Warnhinweisen und Fortschrittsbalken und unterstützt interaktive Korrekturen durch den Benutzer. Diese Merkmale sind insbesondere in Produktionsumgebungen entscheidend, in denen schnelle Entscheidungen und eine intuitive Bedienung notwendig sind.

Für die Implementierung wurde die Programmiersprache Python gewählt. Python bietet neben einer klaren Syntax und leichter Wartbarkeit eine große Anzahl an Bibliotheken für Hardwareansteuerung, Bildverarbeitung und GUI-Entwicklung. Die GUI selbst wird mit *PyQt6* umgesetzt, da dieses Framework moderne, skalierbare und ereignisgesteuerte Benutzeroberflächen erlaubt. [43]

Ein entscheidender Vorteil von *PyQt6* ist die Unterstützung von Multithreading über *QThreads*. Dies erlaubt die parallele Ausführung rechenintensiver Prozesse – wie Bildaufnahme, Tiefenberechnung, Barcodeanalyse oder Gewichtsauswertung – ohne dass die Benutzeroberfläche währenddessen einfriert oder verzögert reagiert.

Multithreading ist besonders relevant, wenn mehrere Kameras simultan Bilder aufnehmen und gleichzeitig Analyseprozesse laufen. [44]

Die theoretische Konzeption einer mehrsprachigen Benutzeroberfläche ist ein weiterer wichtiger Aspekt: Im Unternehmen Leitner arbeiten Personen mit unterschiedlichen Sprachkenntnissen. Um die Bedienung barrierefrei zu gestalten, wird die GUI konzeptionell in Deutsch, Italienisch und Englisch ausgelegt. Dadurch werden Fehler reduziert und die Nutzung des Systems für alle Mitarbeitenden erleichtert. Die Mehrsprachigkeit erfolgt durch die zentrale Verwaltung der Texte in einer Translator-Klasse, die alle Einträge in Form von Dictionaries enthält. Diese Struktur ermöglicht eine einfache Erweiterung auf weitere Sprachen in der Zukunft.

Systemansteuerung

Die Ansteuerung des Gesamtsystems erfolgt über eine kabelgebundene Ethernet-Verbindung zwischen dem Bedienrechner (Laptop) und dem Raspberry Pi. Der Raspberry Pi übernimmt dabei die zentrale Steuerung der Sensorik sowie der angeschlossenen Hardwarekomponenten.

Zur Anzeige und Bedienung des User Interfaces wird eine Remote-Desktop-Verbindung mittels VNC-Viewer hergestellt. Über die IP-Adresse des Raspberry Pi kann so direkt auf dessen grafische Oberfläche zugegriffen werden. Dieses Vorgehen ermöglicht eine einfache Inbetriebnahme, Wartung sowie Fehlersuche, ohne dass zusätzliche Peripheriegeräte (Monitor, Tastatur, Maus) dauerhaft am Embedded-System angeschlossen sein müssen. Gleichzeitig bleibt das System flexibel in bestehende Netzwerkstrukturen integrierbar.

Flussdiagramm

Das zugehörige Flussdiagramm befindet sich im Anhang (siehe Anhang: Flussdiagramm Interface), da es auf einer Seite dargestellt werden muss und die ausführlichen Erklärungen sonst verloren gehen würden. Es bietet eine vereinfachte, konzeptionelle Übersicht des Systemablaufs: Beginnend mit dem Start der Anwendung, über die Initialisierung, bis hin zur Datenerfassung und -verarbeitung. Die Hauptfunktionen sind auf der Startseite zusammengefasst und zu Beginn werden nur die theoretisch notwendigen Funktionen angezeigt. Weitere Funktionen können im praktischen Teil ergänzt werden. Das Diagramm dient primär dem Verständnis der allgemeinen Struktur und bildet den tatsächlichen Programmablauf nicht im Detail ab.

3.2.6 Verwendung von Literatur, Internetquellen und KI

Die theoretischen Grundlagen in diesem Kapitel basieren auf verschiedenen Quellen. Grundlagen der Bildverarbeitung wurden eigenständig aus praktischer Erfahrung erarbeitet, während Kenntnisse im Bereich der Sensorik auf Unterrichtsinhalten aus der Messtechnik zurückgehen. Für spezielle Themen wie Stereo-Vision sowie die Ansteuerung und Auswertung von ADCs bei Wägezellen wurde auf Herstellerangaben und technische Dokumentationen zurückgegriffen.

Ergänzend wurden Kenntnisse aus der Softwaretechnik genutzt, insbesondere zur Hardwareansteuerung und zur Entwicklung grafischer Benutzeroberflächen. Inhalte aus dem Fach Künstliche Intelligenz und Systeme unterstützten zudem die Umsetzung der Barcode-Lokalisierung mit YOLO.

Technische Details und spezifische Informationen zu den eingesetzten Komponenten wurden aus entsprechenden Datenblättern entnommen und praktisch umgesetzt. Eine besonders wichtige Quelle war die umfangreiche Dokumentation von Luxonis, dem Hersteller der verwendeten Tiefenkamera, aus der viele der beschriebenen Zusammenhänge hervorgehen.

Zur sprachlichen Strukturierung sowie zur Veranschaulichung grundlegender Zusammenhänge wurde zusätzlich ein KI-System (Gemini) eingesetzt. Alle fachlichen Inhalte wurden eigenständig überprüft und mit geeigneten Quellen abgeglichen.

Für ein vertieftes Verständnis einzelner Inhalte wird auf die jeweiligen Originalquellen verwiesen, wobei ein wesentlicher Teil der Grundlagen aus dem Unterricht stammt.

3.3 Praktische Umsetzung

In diesem Kapitel wird die konkrete Implementierung des 3D-Scanner-Systems beschrieben. Der Fokus liegt auf der praktischen Realisierung der zuvor erläuterten theoretischen Konzepte in Form von Software und Hardware. Dabei werden die einzelnen Komponenten nicht nur funktional umgesetzt, sondern auch im Zusammenspiel integriert und getestet. Ziel ist es, ein vollständig funktionsfähiges System zu entwickeln, das die Messung, Verarbeitung und Ausgabe der Daten zuverlässig durchführt.

Der Quellcode wird in diesem Kapitel nicht im Detail dokumentiert, sondern auf den wesentlichen Ablauf und die Funktionsweise des Systems reduziert dargestellt. Dies ist dadurch begründet, dass große Teile des Codes selbsterklärend und leicht nachvollziehbar sind. Komplexere oder weniger offensichtliche Codeabschnitte wurden direkt im Code durch Kommentare erläutert. Zusätzlich verfügt jede Funktion über eine kurze Beschreibung in Form eines Docstrings, beispielsweise:

```
def beispiel() :  
    """Hier ist die Beschreibung des Codes der Funktion"""
```

Für ein vollständiges Verständnis wird empfohlen, den Quellcode parallel zur Beschreibung der Programmabläufe zu betrachten. Auf diese Weise lässt sich die Funktionsweise deutlich besser nachvollziehen. Ergänzend werden für die einzelnen Komponenten (Tiefenmessung, Bilderverarbeitung, Gewichtserfassung und Interface) Flussdiagramme erstellt, um insbesondere visuelle Zusammenhänge und Abläufe übersichtlich darzustellen und die Verständlichkeit weiter zu erhöhen.

Die Projektplanung erfolgte in Zusammenarbeit mit Alexander Schiechl und wird in Abschnitt **2.3.1 „Projektplanung“** näher erläutert.

3.3.1 Tiefenmessung mit Stereo-Vision

Im Rahmen der Entwicklung wurde eine zuverlässige Methode zur dreidimensionalen Vermessung von Objekten mittels Stereo-Vision untersucht. Dabei wurden zwei verschiedene Ansätze analysiert und praktisch getestet, von der vollständigen Punktwolkenverarbeitung (Point Cloud) bis zum vereinfachten pixelbasierten Verfahren.

Punktwolkenverarbeitung

Zu Beginn des Projektes wurde der Fokus auf die Verarbeitung einer Punktwolke gelegt. Dieser Ansatz stellt grundsätzlich die technisch sauberste Lösung dar, da die Tiefenkamera hierbei direkt dreidimensionale Koordinatenpunkte (X, Y, Z) liefert. Der wesentliche Vorteil besteht darin, dass reale Raumkoordinaten unmittelbar zur Verfügung stehen und geometrische Berechnungen ohne zusätzliche Umrechnungsfaktoren erfolgen können. Zudem wäre dieser Ansatz im Hinblick auf die Systemarchitektur vorteilhaft gewesen, da ein Teil der Tiefenberechnung direkt auf der Kamera erfolgt und somit die CPU-Belastung auf dem Raspberry Pi reduziert wird. [40]

Theoretisch wäre mit einer korrekt implementierten Punktwolkenauswertung eine sehr hohe Messgenauigkeit realisierbar gewesen. Trotz intensiver Versuche, schrittweise Optimierungen und einschließlich KI-Unterstützung, konnte jedoch keine stabile Implementierung erreicht werden. Die praktische Messabweichung betrug bis zu ± 5 cm und entsprach damit in keiner Weise den Anforderungen des Projekts. Aus diesem Grund wurde der Ansatz mit der Punktwolke verworfen und stattdessen eine alternative Methode verwendet, die auf Pixel-Umrechnung basiert.

Distanzabhängige Pixel-Umrechnung

Für die Realisierung der Abmessungen wurde eine vereinfachte, jedoch robuste Methode auf Basis der Distanzmessung umgesetzt. Das Verfahren beruht auf der Kombination aus gemessener Objektentfernung und einem distanzabhängigen Kalibrierfaktor zur Umrechnung von Pixeln in reale Längeneinheiten.

$$\text{Millimeter pro Pixel} = \frac{\text{Reale Größe in Millimeter}}{\text{Pixelgröße}}$$

Visualisierung und Filterung des Tiefenbildes

Zur Analyse der Tiefendaten wurde das Tiefenbild mittels *JET-Farbskala* dargestellt (siehe Abbildung). Dabei kennzeichnet Rot geringe Distanzen, Grün mittlere Entfernungen und Blau weiter entfernte Bereiche. Diese Darstellung dient ausschließlich der *visuellen Kontrolle* und erleichtert die Bewertung der erfassten Tiefenstruktur. [45]

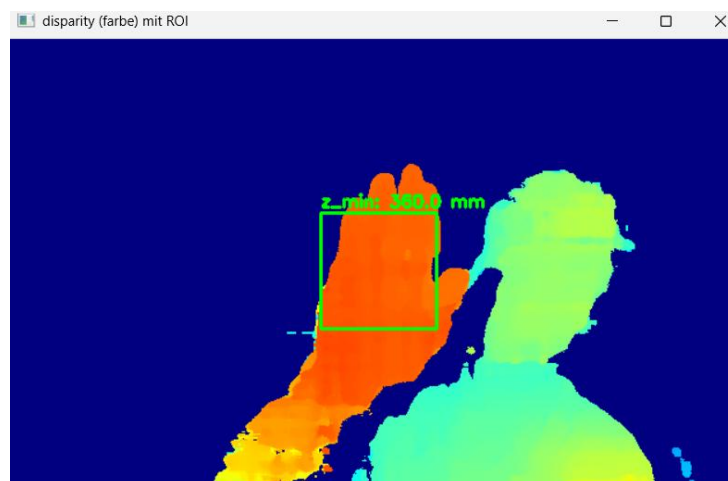


Abbildung 46: JET – Farbskala mit minimal Distanzmessung vom ROI

Für die Messung wird eine definierte Region of Interest (*ROI*) festgelegt (siehe Abbildung). Dadurch wird der auszuwertende Bereich gezielt auf die Wägeplatte begrenzt und irrelevante Bildbereiche werden ausgeschlossen (siehe Abbildung).

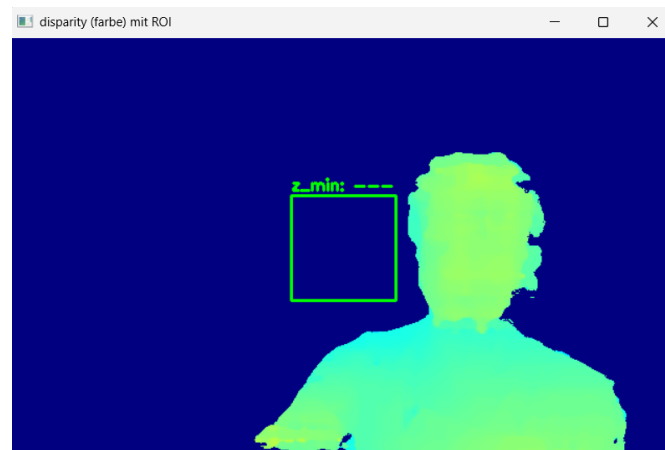


Abbildung 47: JET – Farbskala mit unbekannten Punkten im ROI

Es erfolgt zudem eine Tiefenfilterung: Alle Punkte, deren Z-Wert unterhalb des Referenzabstands zur Wägeplatte liegt, werden entfernt. Ein zusätzlicher Puffer von $+1\text{ cm}$ berücksichtigt Schwankungen in den Tiefenwerten der von der OAK-D S2 AF erzeugten Tiefenkarte und verhindert Fehlzusordnungen bei kleinen Objekten.

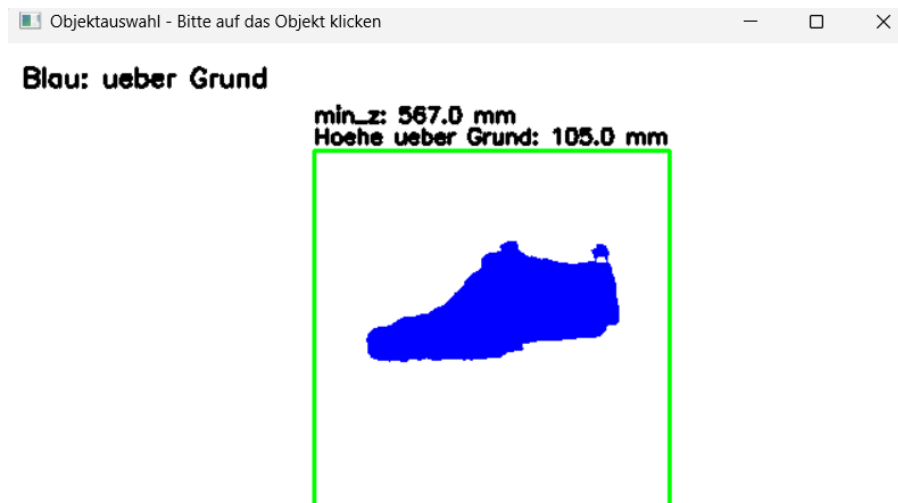


Abbildung 48: Tiefenwerte nach Objekt Filterung

Zur Kontrolle der Kalibrierung der Umrechnungsfaktoren wurde die Pixelauswertung live überprüft. Die *blau hervorgehobenen* Pixel der Objektoberfläche erleichterten dabei das Erkennen möglicher Objekte (siehe Abbildung). In der finalen Umsetzung erfolgt die Objekterkennung vollständig automatisch.

Programmablauf

Beim Starten des Programmes wird zu Beginn eine Kalibrierungsdatei geladen. Diese enthält die zentralen Referenzwerte, insbesondere den Abstand zwischen Kamera und der Wägeplatte sowie den zugehörigen Umrechnungsfaktor in Millimeter pro Pixel.

Anschließend wird das Tiefenbild aufgenommen. In einem Vorverarbeitungsschritt werden dann alle Pixel entfernt, die außerhalb der definierten Region of Interest (ROI) liegen oder deren Tiefenwert außerhalb des zulässigen Bereichs zur Wägeplatte liegt.

Die verbleibenden Objektpunkte werden mithilfe einer rotationsrobusten Bounding Box analysiert, die unabhängig von der Ausrichtung des Objekts stets die kleinstmögliche umschließende Box bestimmt. Aus dieser Bounding Box werden die Kantenlängen in Pixeln berechnet. Anschließend erfolgt die Umrechnung in reale Maßeinheiten, indem die ermittelten Pixelwerte mit dem zuvor bestimmten Kalibrierfaktor multipliziert werden. Dieses Ergebnis mit den berechneten Abmessungen wird direkt an das Interface übergeben.

Strukturierung des Übergabe-Dictionaries:

„distanz_measure03.py“

```
# Ergebnis-Dictionary
result = {
    "success": True,          # Bei erfolgreichem Scan
    "length": height_mm,     # Bounding Box Länge/Höhe
    "width": width_mm,       # Bounding Box Breite
    "height": hoehe_ueber_grund_obj,
    "volume": volume_cm3,
    "abmessung": f"{height_mm:.1f} x {width_mm:.1f} x {hoehe_ueber_grund_obj:.1f}",
    "error": None,
    "depth_frame": None
}
```

Flussdiagramm

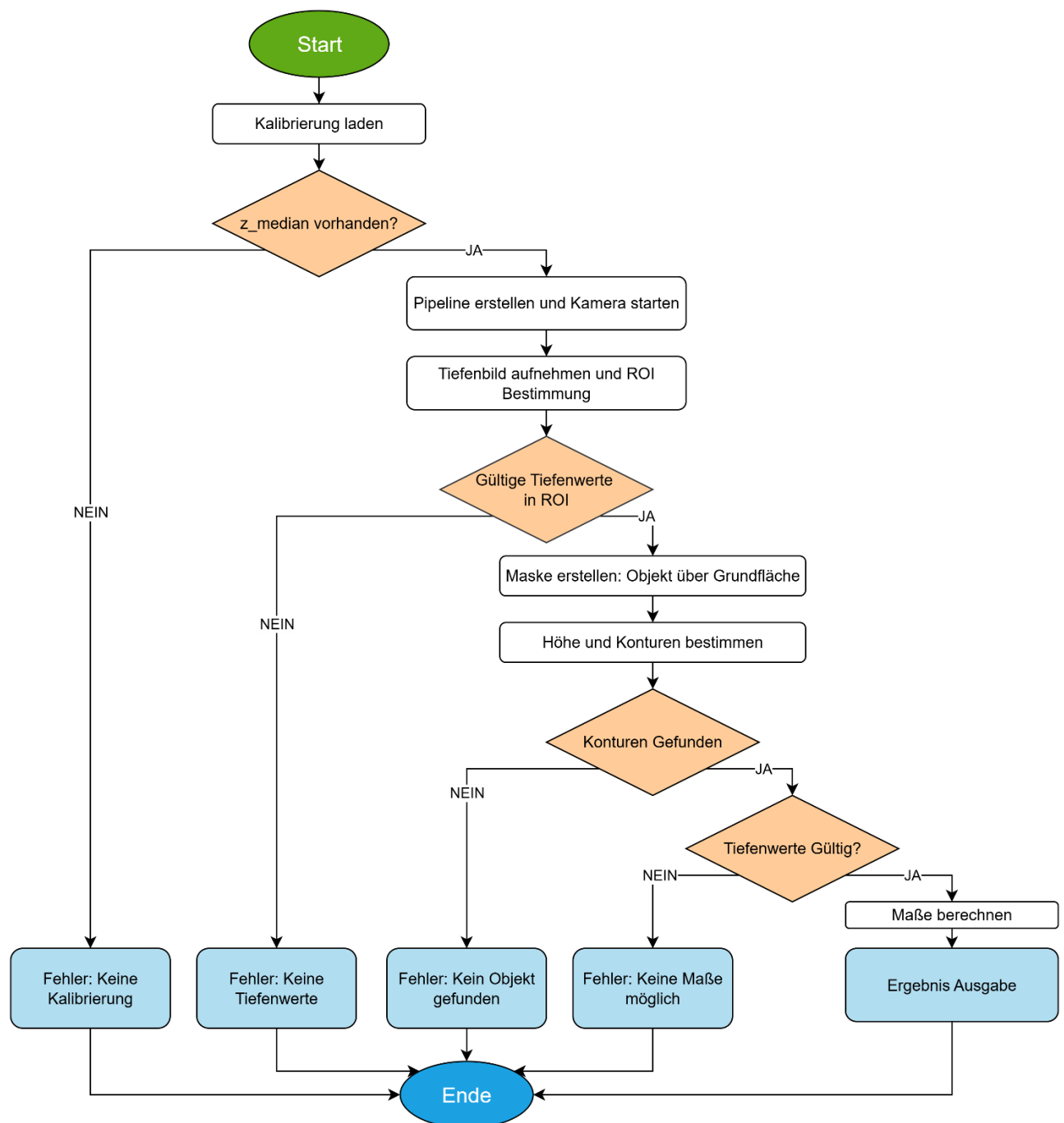


Abbildung 49: Flussdiagramm der `distanz_measure03.py` (eigene Darstellung)

Der Startpunkt entspricht dem Aufruf von `distanz_measure03.py` durch das Interface. Messergebnisse oder Fehler werden zurückgegeben. Rückgabefelder sind im Diagramm *hellblau* markiert. Die Ausgabe erfolgt stets als „Übergabe-Dictionary“.

3.3.2 Bildverarbeitung und Barcode-Dekodierung

Für die Bildverarbeitung sind die aufgenommenen Kamerabilder eine zentrale Grundlage, da in diesen Barcodes bzw. QR-Codes erkannt und anschließend ausgelesen werden sollen. Der entwickelte 3D-Scanner verfügt über vier Kameraperspektiven, wodurch ein Objekt aus mehreren Blickwinkeln erfasst wird. Dadurch erhöht sich die Wahrscheinlichkeit, dass sich mindestens ein Barcode vollständig sichtbar im Bild befindet und korrekt erkannt werden kann. Ziel dieser automatischen Erkennung ist es, den manuellen Arbeitsaufwand zu reduzieren und eine eindeutige Identifikation der gescannten Objekte zu ermöglichen.

Barcode-Erkennung

Zur Überprüfung der Barcode-Erkennung, wird das zuvor angesprochene YOLO-Modell verwendet, um die Funktionsfähigkeit des Modells mit einem Testdatensatz von 945 Bildern zu testen. Diese Bilder enthalten unterschiedliche Barcode-Positionen, Perspektiven und Beleuchtungssituationen. Das Modell erfasste die Umrisse der Barcodes in einem Testdurchlauf zu 99,91 % korrekt. Ein Ausschnitt dieses Testergebnisses ist in Abbildung dargestellt.

```
0: 480x640 2 barcodes, 172.0ms
Speed: 3.3ms preprocess, 172.0ms inference, 1.9ms postprocess per image at shape (1, 3, 480, 640)
PostBarcode944.jpg | 1/1, decodiert: 0 | 0.22s

0: 480x640 2 barcodes, 168.4ms
Speed: 4.0ms preprocess, 168.4ms inference, 1.6ms postprocess per image at shape (1, 3, 480, 640)
PostBarcode945.jpg | 2/2, decodiert: 0 | 0.22s

Gesamt: erkannt 1087/1088 (99.91%)
Gesamt: decodiert 1/1088 (0.09%)
PS C:\Users\grane\Felix_Schule\Diplomarbeit\3D_Scanner\S_Python_Files\BarCodes> █
```

Abbildung 50: Ergebnis von der YOLO-Erkennung

Ziel dieses Tests war primär die Überprüfung der zuverlässigen Barcode-Lokalisierung.

Die für den Test verwendeten Bilder stammen aus dem öffentlich verfügbaren Kaggle-Datensatz. Diese Bilder enthalten Barcodes in unterschiedlichen Größen, Positionen und Bildausschnitten, um reale Anwendungssituationen möglichst gut zu simulieren. Ein Beispiel solcher Testbilder ist in der Abbildung dargestellt, um einen Eindruck der verwendeten Bilddaten zu vermitteln.



Abbildung 51: Foto mit Barcode für die YOLO-Erkennung [46]

Barcode-Dekodierung

Nachdem der Barcode im Bild erkannt wurde, muss der Inhalt des Barcodes dekodiert werden. Hierfür wurden verschiedene Bibliotheken getestet. Zunächst wurde die Bibliothek ZXing eingesetzt, allerdings lieferte diese in der Praxis nur wenige erfolgreiche Dekodierungen. Aus diesem Grund wurde auf die Python-Bibliothek pyzbar umgestellt, welche eine deutlich stabilere und zuverlässigere Erkennung der Barcode-Informationen ermöglichte.



Abbildung 52: Foto mit Barcode für die Dekodierung [46]

Zur Überprüfung der Barcode-Dekodierung wird die Bibliothek *pyzbar* verwendet. Die Funktionsfähigkeit wurde mit einem Testdatensatz von *527 Bildern* getestet. Dieser Datensatz enthält Barcodes in unterschiedlichen Rotationen, Perspektiven sowie unter verschiedenen Beleuchtungssituationen.

Im selben Ordner wie die Bilder befindet sich zudem eine zugehörige *.txt-Datei*, welche die ROI-Ausschnitte der Barcodes bekanntgibt. Dadurch kann sich der Test ausschließlich auf die Dekodierung konzentrieren, da die Position der Barcodes bereits vorgegeben ist.

In einem Testdurchlauf konnten 94,12 % der Barcodes korrekt dekodiert werden. Ein Ausschnitt dieses Testergebnisses ist in der Abbildung dargestellt.

```
✓ ProductBarcode527.jpg erkannt! (Padding=30%, orig, scale=1.0, rot=-5°)
  Typ: CODE39, Wert: 8973997

=== STATISTIK ===
Padding-Werte (Erfolge): {0.3: 491, 0.4: 6}
Methoden (Erfolge): {'orig': 471, 'clahe': 16, 'unsharp': 9, 'clahe+adaptive+inv': 1}
Skalierungen (Erfolge): {1.0: 447, 1.5: 37, 2.0: 13}
Rotationen (Erfolge): {-90: 313, -15: 12, -135: 22, 0: 34, 90: 8, -5: 89, 180: 2, 15: 4, 5: 12, 45: 1}

Trefferquote: 496/527 erkannt (94.12 %)
Top-Paddings: [(0.3, 491), (0.4, 6)]
```

Abbildung 53: Ergebnis von der Barcode-Dekodierung

Um die Dekodierungsrate weiter zu steigern, wurden die Bildverarbeitungsschritte auf die extrahierten ROI angewendet. Diese Schritte kompensieren typische Probleme wie ungünstige Ausrichtung, zu geringe Größe oder schlechte Kontrastverhältnisse des Barcodes. Die wichtigsten Faktoren sind *Padding*, *Rotation* und *Skalierung* des ROI. Darüber hinaus kommen vier weitere Vorverarbeitungsmethoden zum Einsatz, die die Bildqualität gezielt verbessern.

Auf Python-Ebene werden diese Bearbeitungen über speziell definierte Hilfsmethoden umgesetzt. Sie kapseln die einzelnen Transformationen in wiederverwendbaren, klar strukturierten Funktionen. Dadurch bleibt die Hauptlogik schlank und übersichtlich, während komplexe Operationen konsistent und ohne Duplikation ausgeführt werden können.

```
@staticmethod
```

Diese Zeile vor eine Funktion erlaubt es, die Methoden unabhängig von einzelnen Objektinstanzen direkt über die Klasse aufzurufen, was die Wartbarkeit und Lesbarkeit des Codes deutlich verbessert.

Programm Ablauf

Die automatische Barcode-Erkennung im 3D-Scanner kombiniert die zuvor beschriebenen Schritte Vorverarbeitung, Lokalisierung und Dekodierung zu einem durchgängigen Ablauf. Die zentrale Funktion `detect_barcodes(images)` verarbeitet dabei vier Kameraperspektiven (`iso_Bild`, `top_Bild`, `left_Bild`, `behind_Bild`) gleichzeitig.

Zu Beginn wird das YOLOv8-Modell geladen [41], das speziell für die Erkennung von Barcodes trainiert wurde. Für die Auswertung wird eine Konfidenzschwelle von 0,5 festgelegt, sodass nur sichere Treffer berücksichtigt werden. Jedes Bild erhält einen sprechenden Namen entsprechend seiner Perspektive, um die Ergebnisse eindeutig zuordnen zu können.

Im nächsten Schritt identifiziert YOLO potenzielle Barcode-Bereiche im Bild. Jeder erkannte Bereich wird als ROI extrahiert und um einen kleinen Padding-Rand (20%) erweitert, um sicherzustellen, dass der Code vollständig enthalten ist. Danach wird

die ROI gemäß den vorher definierten Vorverarbeitungsparametern bearbeitet: Graustufen, Skalierung auf 1,5- oder 2-fache Größe, Rotation in verschiedenen Winkeln, Kontrastverstärkung, Schärfung oder Invertierung.

Für jede skalierte und rotierte Variante des Ausschnitts versucht *pyzbar*, den Barcode zu dekodieren. Sobald ein Barcode erfolgreich dekodiert wird, werden alle relevanten Informationen gespeichert, welche in einem zusammengefassten Dictionary enthalten sind:

„*BarCode_v03.py*“

```
barcode_info = {
    "image_index": image_index,
    "image_name": image_name,
    "value": barcode_data["value"],
    "type": barcode_data["type"],
    "confidence": float(box.conf[0]) if hasattr(box, 'conf')
                    else 0.0,
    "bbox": [x1, y1, x2, y2],
    "cropped_image": roi.copy()
}
```

Nach der Verarbeitung aller Bilder gibt die Funktion die vollständige Liste aller erkannten Barcodes zurück. Bei Fehlern oder wenn keine Barcodes gefunden werden, wird eine leere Liste zurückgegeben. Eine abschließende Logmeldung informiert über die Gesamtzahl der erkannten Barcodes.

Durch diese systematische Vorgehensweise lassen sich die Barcode-Ergebnisse eindeutig den Kameraperspektiven zuordnen, die Inhalte auslesen und die Bildausschnitte für Visualisierung oder weitergehende Analysen nutzen.

Flussdiagramm

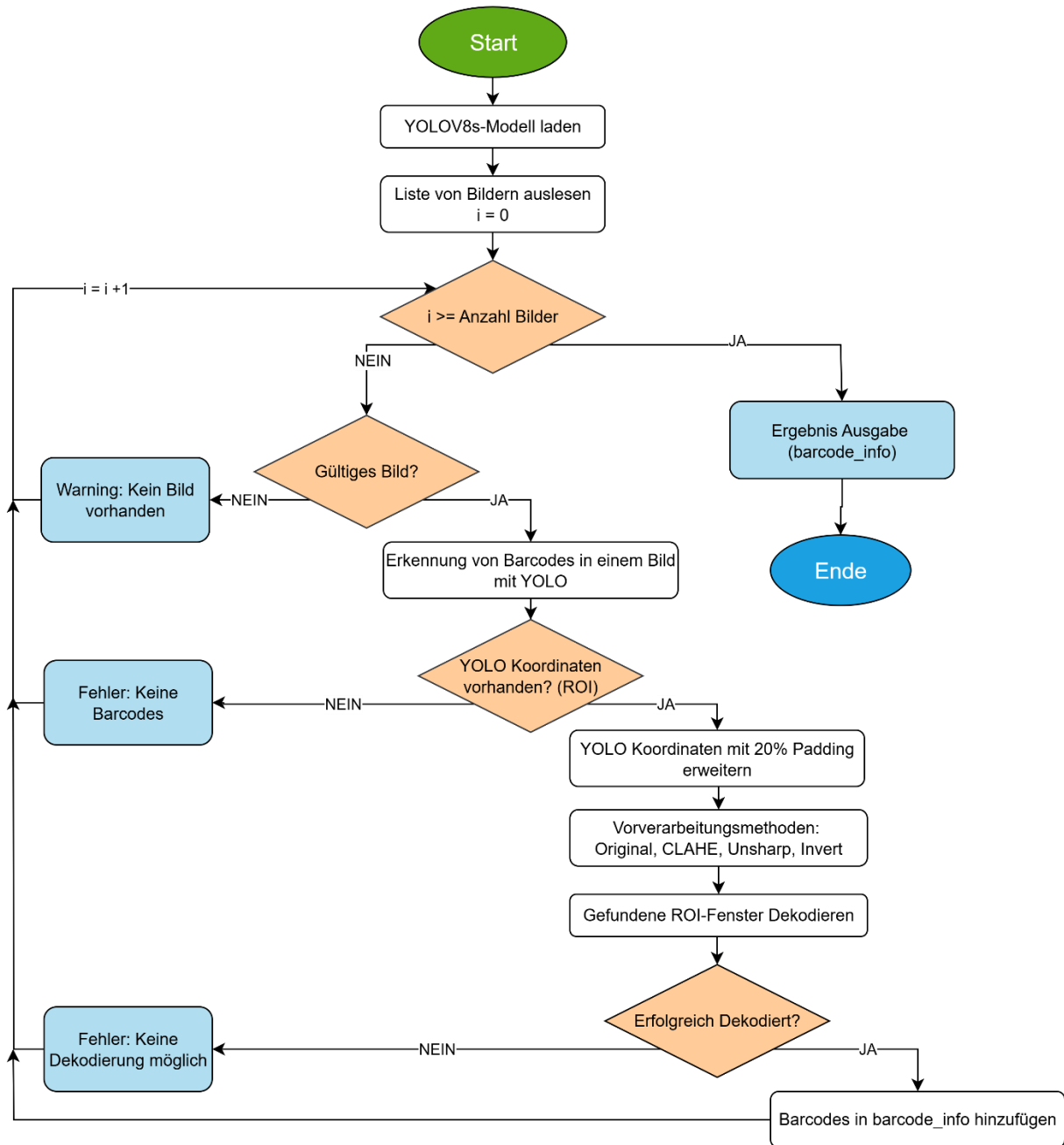


Abbildung 54: Flussdiagramm der BarCode_v03.py (eigene Darstellung)

Das Flowchart besitzt einen zentralen Startpunkt (*grün*), welcher die Initialisierung des YOLO-Modells sowie das Einlesen der Bilderliste einleitet. Im Prozess erfolgen die automatisierte Erkennung, Vorverarbeitung und Dekodierung der Barcodes in einer Schleife. Die Ausgabe von Fehlern, Warnungen und Ergebnissen erfolgt bei den *hellblau* markierten Feldern.

3.3.3 Gewichtserfassung

Die Gewichtserfassung basiert auf drei Wägezellen, deren Signale über Analog-Digital-Wandler (ADCs) erfasst und anschließend vom Raspberry Pi verarbeitet werden. Ziel ist es, aus den elektrischen Rohwerten ein präzises Gesamtgewicht in Gramm zu bestimmen.

Systemaufbau und Kommunikation

Die Wägezellen werden an ein Adafruit-Breakout-Board mit dem NAU7802 angeschlossen, welcher als ADC die analogen Sensorsignale erfasst und in digitale Messwerte umwandelt. Die Kommunikation erfolgt über die I²C-Schnittstelle, welche vom Raspberry Pi nativ unterstützt wird.

Da die eingesetzten ADCs nur auf zwei mögliche I²C-Adressen (*0x2A* und *0x2B*) konfiguriert werden können, kommt es beim Betrieb mehrerer Module am selben I²C-Bus zu Adresskonflikten. [34]

Zur Lösung dieses Problems wird ein Multiplexer mit der Bezeichnung *TCA9548A* eingesetzt. Der Multiplexer (MUX) ermöglicht es, mehrere ADC-Module mit identischer Adresse zu betreiben, indem die einzelnen Geräte auf separate Kanäle verteilt werden. Über diesen Multiplexer kann jeweils ein bestimmter Kanal aktiviert werden, sodass immer nur das gewünschte ADC-Modul mit dem Raspberry Pi verbunden ist. Die Kommunikation erfolgt somit indirekt über den Multiplexer, der den Zugriff auf die einzelnen ADCs steuert. [47]

Signalverarbeitung der Wägezellen

Die ADCs liefern Rohwerte, die proportional zur Verformung der Wägezellen sind. Um diese Rohwerte in physikalische Einheiten (Gramm) umzuwandeln, wird ein Kalibrierfaktor verwendet.

Zur Bestimmung des Kalibrierfaktors wird ein Referenzgewicht genutzt. Der gemessene Rohwert wird ins Verhältnis zum bekannten Gewicht gesetzt, wodurch ein individueller Kalibrierfaktor entsteht. Dieser Faktor wird in der Datei „weight_calibration.json“ gespeichert. Im Messbetrieb wird jeder Rohwert mit dem zugehörigen Kalibrierfaktor multipliziert, wodurch das Gewicht der jeweiligen Wägezelle in Gramm berechnet wird.

Programmablauf

Der Messvorgang erfolgt in einer festen Reihenfolge:

- Initialisierung der ADCs über den Multiplexer
 - Trieren, falls noch nichts vorhanden in der JSON-Datei
- Auswahl des entsprechenden Kanals für jede Wägezelle
- Auslesen der aktuellen Rohwerte der einzelnen Wägezellen
- Umrechnung der Rohwerte mithilfe der gespeicherten Kalibrierfaktoren
- Addition der Einzelwerte zur Bestimmung des Gesamtgewichts

Softwarelogik

Das Python *Gewichts_Messung02.py* Modul beinhaltet mehrere zentrale Funktionen:

- **tara()** – führt eine Nullstellung aller Wägezellen durch und speichert die ermittelten Offsets in einer JSON-Datei. Diese Trierung wird beim Start der GUI automatisch durchgeführt, kann aber auch manuell über die Benutzeroberfläche erneut ausgeführt werden.

- **calibrate_cell(index, known_weight_grams)** – kalibriert eine einzelne Wägezelle mit einem bekannten Referenzgewicht. Dabei wird der individuelle Kalibrierfaktor berechnet und gespeichert.
- **measure_cell(index)** – liest den aktuellen Rohwert einer Zelle aus, wendet den Kalibrierfaktor an und glättet den Wert zur Reduktion kurzfristiger Schwankungen.
- **get_weight()** – summiert die Einzelgewichte aller Zellen und liefert das Gesamtgewicht in Gramm.

Die Kalibrierungsfaktoren als auch die Tara-Offsets werden in der JSON-Datei gespeichert. Dadurch bleiben die Werte zwischen den Messungen erhalten, und reproduzierbare Messergebnisse sind gewährleistet.

„weight_calibration.json“

```
{
  "faktoren": [
    -0.004423,
    -0.00425,
    -0.004543
  ],
  "zero_offsets": [
    0,
    0,
    0
  ],
  "_comment_default_factors": "Original calibration factors
from code: [-0.004423, -0.00425, -0.004543]. Copy these values
into 'faktoren' to reset manually."
}
```

Flussdiagramm

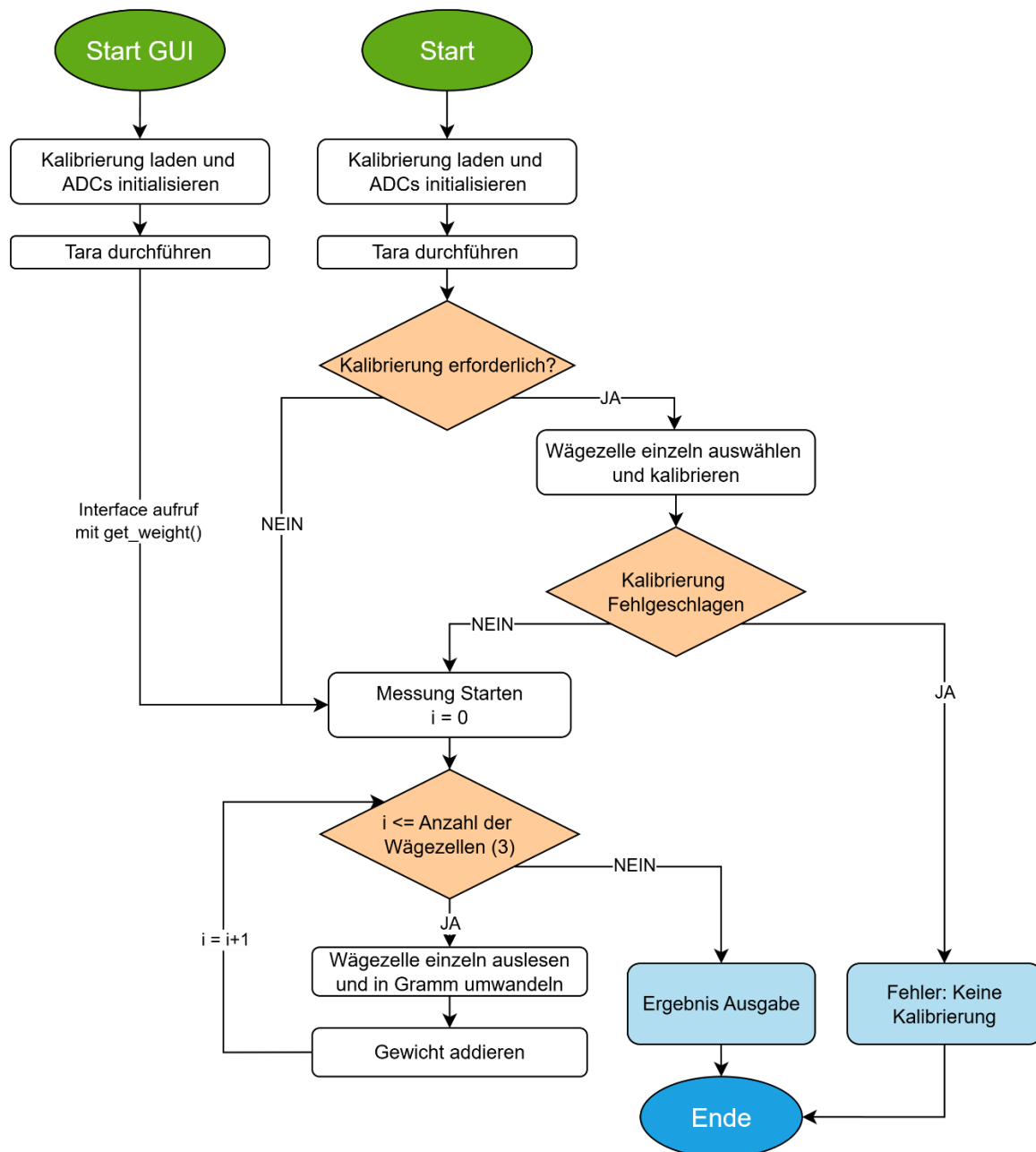


Abbildung 55: Flussdiagramm der Gewichts_Messung02.py (eigene Darstellung)

Das Flowchart besitzt zwei Startpunkte: den direkten Programmstart der Python-Datei sowie den Aufruf über das Interface (GUI). Beim direkten Start erfolgt die vollständige Initialisierung inklusive optionaler Kalibrierung der Wägezellen. Im Interfacebetrieb wird hingegen ausschließlich die Messfunktion verwendet. Die Ausgabe erfolgt bei den hellblau markierten Feldern.

3.3.4 Beleuchtung

Die LED-Beleuchtung wird über einen LOLIN D1 Mini (ESP8266) gesteuert, um die GPIO-Pins des Raspberry Pi zu entlasten und Störungen zu vermeiden. Die Verbindung erfolgt über einen USB-UART-Wandler (`/dev/ttyUSB0`) mit 9600 Baudrate, 8 Datenbits, 1 Stoppbit und ohne Parität. Jeder Befehl besteht aus einem ASCII-Zeichen mit Zeilenumbruch, vor jedem Ein- oder Ausschalten wird *"Change"* gesendet, um den D1 Mini in den Steuermodus zu versetzen.

Protokoll für die `_send_command` Funktion:

„Interface_v08.py“

```
class CameraManager:
    def _control_light(self, state: bool):
        """Steuert die Beleuchtung """
        ''' Mögliche Commands:  (nur 'a' & '0' verwendet)

        case '1': Blitz(1); break;
        case '2': Blitz(2); break;
        case '3': Blitz(3); break;
        case '4': Strip_On(); break;
        case 'a': All_ON(); break;
        case '0': All_OFF(); break;
        '''
```

Die Beleuchtungssteuerung wird in der Raspberry-Pi-Software von der Klasse *CameraManager* übernommen. `_send_command` öffnet bei Bedarf den seriellen Port und sendet die Befehle, `_control_light` schaltet die LEDs ein oder aus und sorgt kurz für Stabilisierung. Im Scanablauf leuchten die LEDs vor der ersten Aufnahme auf und werden danach wieder ausgeschaltet. Die Testfunktion „Beleuchtung prüfen“ schaltet die Lichter für fünf Sekunden ein.

Details zur Firmware sind in Abschnitt **2.3.3 „Ansteuerung der Beleuchtung“** beschrieben.

3.3.5 Datenmanagement

Für die Verarbeitung der erfassten Daten wurde eine CSV-Datei implementiert, die mit dem SAP-System der Firma *Leitner* kompatibel ist. Dadurch können die aufgenommenen Daten ohne zusätzliche Anpassungen in das System importiert werden. Der Export erfolgt in einem Format, das den Anforderungen des SAP-Imports entspricht und somit eine einfache sowie schnelle Weiterverarbeitung der Messdaten ermöglicht.

	A	B	C	D	E	F	G
1	Interne Materialnummer	Gewicht_kg	Laenge_mm	Breite_mm	Hoehe_mm	EAN	ISO_Bild_Pfad
2	354678612	3	0	0	0	1234567890123	Fotos\Scan_20260316_210017\iso_Bild.jpg

Abbildung 56: Auszug von der CSV-Datei für den SAP-Import

Ursprünglich wurde geprüft, ob eine direkte automatisierte Anbindung an das SAP-System umgesetzt werden kann. Eine solche Integration war jedoch nicht möglich, da von Seiten der IT-Abteilung der Firma *Leitner* keine Freigabe für eine direkte Schnittstelle erteilt wurde. Aus diesem Grund erfolgt die Datenübertragung über eine CSV-Datei, die anschließend manuell in das SAP-System importiert werden kann.

Die Speicherung der Daten erfolgt in einer klar strukturierten Ordnerhierarchie.

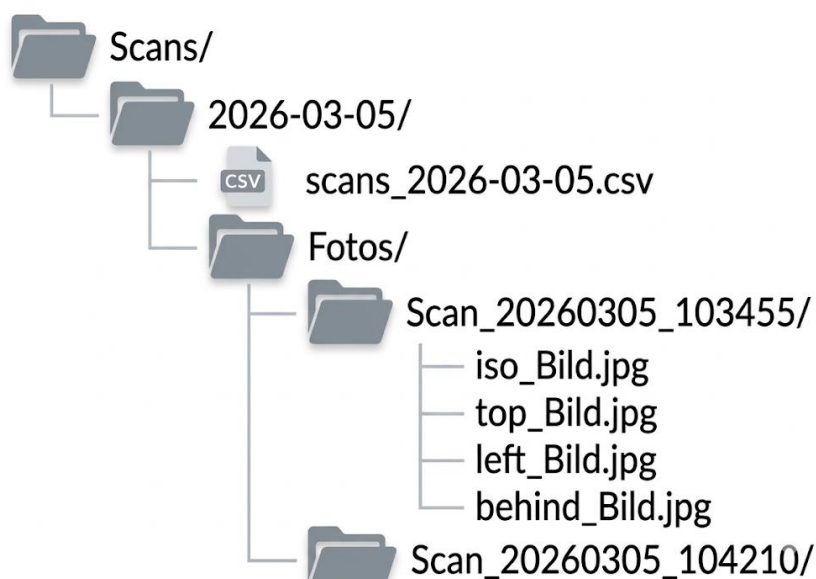


Abbildung 57: Ordnerstruktur (eigene Darstellung)

Auf dem Desktop des Raspberry Pi befindet sich der Hauptordner „Scans“, in dem alle vom Benutzer gespeicherten Daten abgelegt werden. Innerhalb dieses Ordners wird für jeden Kalendertag automatisch ein eigener Unterordner erstellt, der nach dem jeweiligen Datum benannt ist. In diesem Tagesordner werden sämtliche Scans gespeichert, die an diesem Tag durchgeführt wurden. Beginnt ein neuer Tag, wird automatisch ein neuer Ordner angelegt, wodurch eine chronologische und übersichtliche Struktur entsteht.

Im jeweiligen Tagesordner befindet sich die CSV-Datei, welche alle erfolgreichen Scans des Tages enthält. Dadurch werden sämtliche Messdaten eines Tages zentral in einer einzigen Datei gesammelt. Die aufgenommenen Bilder werden hingegen separat gespeichert. Hierfür existiert innerhalb des Tagesordners ein weiterer Unterordner, in dem für jeden einzelnen Scan ein eigener Ordner erstellt wird. Diese Scan-Ordner sind mit einem Zeitstempel versehen, sodass jede Bildaufnahme eindeutig zugeordnet werden kann.

Diese Struktur wurde bewusst einfach gestaltet, um auch bei einer großen Anzahl an Scans eine übersichtliche Organisation und schnelle Auffindbarkeit der Daten zu gewährleisten. Gleichzeitig unterstützt diese Vorgehensweise das grundlegende Ziel des Projekts: eine effiziente und benutzerfreundliche Erfassung sowie Verwaltung von Scan- und Messdaten.

3.3.6 Raspberry Pi – Installation und Systemeinrichtung

Betriebssystem

Für den Betrieb des Systems wird ein stabiles und gut unterstütztes Betriebssystem benötigt, das eine zuverlässige Zusammenarbeit mit Python-Programmen, Kameras und weiteren Schnittstellen ermöglicht. Aus diesem Grund wird **Raspberry Pi OS** verwendet. Dieses Betriebssystem basiert auf Linux und ist speziell für den Einsatz auf Raspberry-Pi-Systemen ausgelegt. Es bietet eine gute Unterstützung für Hardwarezugriffe, Python-Bibliotheken sowie für Anwendungen aus dem Bereich der Bildverarbeitung.

Die Installation des Betriebssystems erfolgt mithilfe des Programms *Raspberry Pi Imager*. Mit diesem Werkzeug kann das gewünschte Betriebssystem direkt auf eine microSD-Karte geschrieben werden. Für das vorliegende Projekt wird Raspberry Pi OS in der 64-Bit-Version mit grafischer Oberfläche verwendet, da der eingesetzte Raspberry Pi 5 mit 8 GB RAM eine 64-Bit-Architektur unterstützt und somit die verfügbaren Ressourcen effizient genutzt werden können.

Während der Installation können bereits grundlegende Einstellungen vorgenommen werden. Dazu zählen unter anderem das Festlegen eines Hostnamens für das System, in diesem Fall *3D-Scanner*, sowie das Aktivieren des SSH-Zugriffs. Die Aktivierung von SSH ermöglicht es, später über das Netzwerk auf den Raspberry Pi zuzugreifen, ohne dass eine direkte Verbindung über Bildschirm, Tastatur und Maus erforderlich ist.

Verbindungsaufbau

Nach der Installation des Betriebssystems wird der Raspberry Pi über ein Ethernet-Kabel mit dem PC oder mit einem Netzwerkgerät, beispielsweise einem Switch oder Router, verbunden. In vielen Fällen wird dem Gerät automatisch eine IP-Adresse durch einen DHCP-Server zugewiesen. Diese kann anschließend über die Benutzeroberfläche des Routers oder mithilfe von Netzwerkwerkzeugen wie *arp -a* oder *nmap* ermittelt werden.

Sobald die IP-Adresse bekannt ist, kann die Verbindung zwischen PC und Raspberry Pi überprüft werden. Dazu wird vom PC aus ein Ping-Befehl auf die entsprechende IP-Adresse ausgeführt. Ist das Gerät erreichbar, kann anschließend eine Verbindung über SSH hergestellt werden. Dabei wird beispielsweise folgender Terminalbefehl verwendet:

```
ssh pi@<IP-Adresse>
```

Nach der Eingabe des zugehörigen Passworts erhält man Zugriff auf die Kommandozeile des Raspberry Pi und kann weitere Konfigurationsschritte durchführen.

Falls im Netzwerk kein DHCP-Server vorhanden ist, besteht alternativ die Möglichkeit, eine statische IP-Adresse zu vergeben. Dabei wird sowohl auf dem PC als auch auf dem Raspberry Pi eine manuelle Adresse im selben Subnetz eingestellt, beispielsweise 192.168.189.x mit der Subnetzmaske 255.255.255.0.

Nach erfolgreicher Herstellung der Verbindung werden zunächst die Systempakete aktualisiert, um sicherzustellen, dass alle installierten Komponenten auf dem aktuellen Stand sind. Anschließend können weitere benötigte Programme und Bibliotheken installiert werden, beispielsweise Python und zusätzliche Softwarepakete für Bildverarbeitung oder grafische Benutzeroberflächen.

Die grafische Oberfläche kann zusätzlich mit einem Fernzugriff über VNC eingerichtet werden. Dabei wird auf dem Raspberry Pi ein VNC-Server aktiviert, während auf dem Computer ein entsprechender Viewer verwendet wird. Auf diese Weise kann der Desktop des Raspberry Pi über das Netzwerk angezeigt und bedient werden.

Verwendete Python-Bibliotheken

Für das Projekt wird zunächst eine virtuelle Python-Umgebung (venv3_10) erstellt und aktiviert. Anschließend werden alle benötigten Bibliotheken über die Datei *requirements.txt* installiert. Dadurch wird sichergestellt, dass alle notwendigen Abhängigkeiten in der korrekten Version vorhanden sind und das Projekt reproduzierbar auf anderen Systemen ausgeführt werden kann.

Die Datei *requirements.txt* enthält alle im Projekt verwendeten Drittanbieter-Bibliotheken. Die im Quellcode importierten Pakete sind darin vollständig enthalten. Die folgende Tabelle zeigt die verwendeten Bibliotheken, deren Importnamen sowie eine kurze Beschreibung ihrer Funktion im Projekt.

Bibliotheksname	Importname	Beschreibung
numpy	import numpy	Bibliothek für numerische Berechnungen und effiziente Verarbeitung von Arrays und Matrizen.
opencv-python	import cv2	Open-Source-Bibliothek für Bildverarbeitung und Computer Vision. Wird zur Verarbeitung der Kamerabilder verwendet.
pyserial	import serial	Ermöglicht die serielle Kommunikation mit externen Geräten über COM- bzw. USB-Schnittstellen.
Pillow	from PIL import Image	Python-Bibliothek zur Verarbeitung und Manipulation von Bildern.

depthai	import depthai	SDK zur Ansteuerung von OAK-Kameras und zur Nutzung von KI- und Tiefendatenfunktionen.
PyQt6	from PyQt6.QtWidgets import ...	Framework zur Erstellung grafischer Benutzeroberflächen (GUI) in Python.
smbus2	import smbus2	Bibliothek zur Kommunikation mit I ² C-Geräten über den SMBus.
PyNAU7802	from PyNAU7802 import NAU7802	Treiberbibliothek für den NAU7802-ADC zur Auslesung von Wägezellen.
ultralytics	from ultralytics import YOLO	Implementierung des YOLO-Modells zur Objekterkennung mittels neuronaler Netze.
pyzbar	from pyzbar.pyzbar import decode	Bibliothek zum Dekodieren von Barcodes und QR-Codes aus Bildern.

Externe Python-Bibliotheken zum Installieren

Die Python-Standardbibliotheken wie *os*, *csv*, *sys*, *time*, *json*, *datetime*, *dataclasses*, *typing*, *logging*, *subprocess* und *contextlib* müssen nicht in der *requirements.txt* aufgeführt werden, da sie bereits Bestandteil der Python-Standardinstallation sind.
[48]

Die in der *requirements.txt* definierten externen Bibliotheken können anschließend mit folgendem Befehl installiert werden:

```
pip install -r requirements.txt
```

3.3.7 User Interface

Auf Grundlage der theoretischen Grundlagen wurde die GUI im konkreten System implementiert. Sie ist so aufgebaut, dass sie den gesamten Arbeitsprozess des 3D-Scanners abbildet und die Bedienung für Anwender ohne tiefgehende technische Kenntnisse ermöglicht.

Startseite und Systemprüfung

Die Startseite der GUI stellt zentrale Steuerfunktionen und Diagnoseoptionen bereit:

- Überprüfung der Kamerakommunikation aller vier Kameramodule
- Aktivierung und Kontrolle der Scanner-Beleuchtung
- Tarieren der Wägezellen
- Verweis auf die Betriebsanleitung
- Starten eines Scanvorgangs
- Beenden des Programms

Bildaufnahme und Kontrolle

Beim Start eines Scanvorgangs werden vier Bilder erfasst. Die Aufnahmen erfolgen über drei USB-Kameras sowie die RGB-Kamera der OAK-D S2 AF. Anschließend werden die Bilder in der Benutzeroberfläche angezeigt, sodass der Benutzer deren Qualität überprüfen kann.

Die GUI bietet folgende Interaktionsmöglichkeiten:

- Akzeptieren oder Verwerfen der Bilder
- Erneutes Aufnehmen bei unzureichender Qualität
- Prüfung auf korrekte Objekterkennung und ausreichende Objektgröße

Durch diese Kontrolle wird sichergestellt, dass nur geeignete Bilddaten in die weitere Verarbeitung gelangen, was die Genauigkeit der Messungen erhöht.

Automatisierte Verarbeitung

Nach Bestätigung der Bildaufnahmen wird die automatisierte Verarbeitung gestartet. Diese erfolgt in mehreren parallel ausgeführten Verarbeitungsschritten innerhalb eines separaten *ParallelWorker*, der mittels QThreads aus der Bibliothek PyQt6 realisiert ist. Innerhalb dieses Parallelworkers werden folgende Aufgaben bearbeitet:

- **Geometrische Abmessungen**
- **Barcode-Erkennung**
- **Gewichtserfassung**

Der Fortschritt wird in Echtzeit über einen visuellen Fortschrittsbalken angezeigt. Tritt ein Fehler auf, wird dieser sowohl in einer Log-Datei dokumentiert als auch visuell in der GUI markiert. Felder mit unvollständigen oder fehlerhaften Daten werden als „*Undefiniert*“ angezeigt, um eine klare Rückmeldung zu geben.

Bedienung und manuelle Korrekturmöglichkeiten

Da bestimmte Objekte aufgrund von Größe, Oberfläche oder Form nicht vollständig automatisch erfasst werden können, sind alle Messfelder frei editierbar, sodass erkannte Werte jederzeit manuell ergänzt oder korrigiert werden können. Für die Barcode-Verarbeitung stehen Checkboxen zur Verfügung, mit denen gezielt relevante Codes ausgewählt werden können. [43]

Jeder Barcode wird in einem eigenen Widget angezeigt, das Bild, Eingabefeld, Typauswahl und Statusanzeige kombiniert. Farbcodierte Rückmeldungen zeigen sofort, ob ein Barcode erkannt oder leer ist. Dies gewährleistet Konsistenz und ermöglicht eine automatische Weiterverarbeitung, etwa für Speicherung oder SAP-Integration. Das System ist damit flexibel, benutzerfreundlich und unterstützt eine effiziente, teilweise automatisierte Erfassung von Mess- und Barcode-Daten.

Mehrsprachige Benutzeroberfläche

Die Mehrsprachigkeit wird über eine zentrale *TranslationManager-Klasse* realisiert. Alle Texte der Benutzeroberfläche sind dabei strukturiert in einem verschachtelten Dictionary organisiert und thematisch nach Seiten gruppiert. Jeder Eintrag enthält die Übersetzungen als Tupel in den unterstützten Sprachen (Deutsch, Englisch, Italienisch).

Die Methode *get_text(language, page, key)* kapselt den Zugriff auf die Übersetzungen und stellt sicher, dass bei fehlenden Einträgen definierte Fallback-Werte zurückgegeben werden. Diese Struktur ermöglicht es neue Sprachen oder zusätzliche Texte ohne Änderungen an der Programmlogik einfach zu ergänzen, da lediglich die zentrale Datenstruktur erweitert werden muss.

Zugriff auf die Betriebsanleitung

Auf der Startseite befindet sich ein Button, über den die Betriebsanleitung direkt geöffnet werden kann. Beim Anklicken wird eine PDF-Datei automatisch im Standardprogramm des Betriebssystems angezeigt:

„Interface_v08.py“

```
def show_instructions(self):  
    ...  
    if platform.system() == "Windows":  
        os.startfile(pdf_datei)  
    else:  
        subprocess.run(["xdg-open", pdf_datei])  
    ...
```

Diese Funktion ermöglicht einen schnellen Zugriff auf Anweisungen, ohne die Anwendung verlassen zu müssen. [49]

Datenspeicherung und Systemintegration

Die Daten werden standardmäßig lokal gespeichert. Die Umsetzung erfolgt in der Funktion `save_all_data_csv()`. Dabei werden:

- Messwerte in einer CSV-Datei abgelegt
- Bilddateien in einem definierten Verzeichnis gespeichert
- der Pfad zum isometrischen Bild in der CSV-Datei hinterlegt

Die Struktur der CSV-Datei ist mit dem internen ERP-System (SAP) kompatibel, sodass ein Import ohne zusätzliche Nachbearbeitung möglich ist. Optional ist eine direkte Datenübertragung über eine API vorgesehen, sofern entsprechende Netzwerkfreigaben und Genehmigungen durch die IT-Abteilung vorliegen. Diese Funktionalität wurde im Rahmen dieser Arbeit jedoch nicht umgesetzt.

Systemansteuerung

Die Ansteuerung des Gesamtsystems erfolgt über ein Ethernet-Kabel, das eine direkte Verbindung zwischen dem Bedienrechner und dem Raspberry Pi herstellt. Die GUI wird über einen VNC-Viewer aufgerufen, sodass der Zugriff auf das Interface direkt über die IP-Adresse des Raspberry Pi möglich ist. Beispielsweise kann der *RealVNC-Viewer* verwendet werden, der kostenlos heruntergeladen werden kann [50]. So kann jeder Anwender eine einfache Verbindung zum System herstellen.

Dieses Konzept erlaubt:

- einfache Inbetriebnahme
- schnelle Wartung
- Fehlersuche ohne angeschlossene Peripheriegeräte
- flexible Integration in bestehende Netzwerke

3.4 Zusammenfassung individueller Teil

Das Projektziel konnte grundsätzlich erreicht werden, jedoch nicht vollständig in dem ursprünglich geplanten Umfang. Insbesondere die Arbeit mit der Tiefenkamera erforderte deutlich mehr Entwicklungszeit als erwartet. Aufgrund verschiedener Probleme, wie einer zu geringen Anzahl an verwertbaren ROI-Punkten sowie instabilen Messergebnissen, musste schließlich auf ein einfacheres und weniger genaues Messverfahren umgestellt werden. Eine vollständige Ursachenanalyse wäre im Rahmen der Diplomarbeit mit einem unverhältnismäßig hohen Zeitaufwand verbunden gewesen.

Auch die geplante direkte Integration in das SAP-System der Leitner AG konnte nicht vollständig umgesetzt werden. Der Datenaustausch erfolgt daher über den Export einer CSV-Datei, die anschließend manuell in das System importiert wird. Diese Einschränkung entstand durch Sicherheitsvorgaben der IT-Abteilung der Leitner AG, da eine direkte Systemanbindung potenzielle Risiken für Datenkonsistenz und Systemstabilität darstellen könnte.

Der beeindruckendste Teil dieser Arbeit lag in der Entwicklung und Umsetzung der Gewichtserfassung. Dabei wurden mehrere Wägezellen über den Analog-Digital-Wandler NAU7802 ausgelesen und mithilfe eines I²C-Multiplexers (TCA9548A) angesteuert. Ergänzend dazu wurden ein Tarierverfahren sowie eine Softwarelogik zur Verarbeitung und Summierung der Messwerte implementiert. Die Gewichtserfassung erwies sich dabei als genauer als ursprünglich erwartet.

Ergänzend wurde eine mehrsprachige Benutzeroberfläche entwickelt, die eine einfache und übersichtliche Bedienung des Systems ermöglicht.

Im Verlauf der Diplomarbeit konnten zahlreiche praktische Erfahrungen gesammelt werden, insbesondere im Bereich der Ansteuerung unterschiedlicher Hardwarekomponenten über Schnittstellen wie I²C, USB und UART. Zudem zeigte sich, dass Wägezellen sehr zuverlässig arbeiten und stabile Messergebnisse liefern können. Auch im Softwarebereich wurden neue Erkenntnisse gewonnen. Die Programmiersprache Python erwies sich als gut strukturierte und leicht verständliche Lösung für die Implementierung der Systemlogik, auch wenn für grafische Benutzeroberflächen mittlerweile modernere Alternativen existieren.

Der Einsatz von KI-gestützten Systemen wurde als unterstützendes Werkzeug erprobt. Diese konnten insbesondere bei der Strukturierung von Code, der Fehlersuche sowie bei allgemeinen Lösungsansätzen hilfreich sein. Dabei zeigte sich jedoch, dass die generierten Inhalte nicht immer korrekt oder optimal auf die konkrete Problemstellung abgestimmt sind. In der Praxis war daher eine regelmäßige Überprüfung sowie teilweise mehrfache Nachkorrektur erforderlich, um funktionierende und saubere Lösungen zu erhalten. Der sinnvolle Einsatz von KI erfordert somit ein gutes technisches Verständnis, um fehlerhafte oder unpassende Vorschläge zuverlässig erkennen und anpassen zu können.

4 Zusammenfassung und Ausblick

4.1 Ergebnisse und Diskussion

Durch eine Verknüpfung von Hardware und Software konnten abschließend Messergebnisse ermittelt und evaluiert werden. In dieser Testphase wurde auf der Wägeplatte zusätzlich ein Schachmuster platziert, um der Tiefenkamera mehr Referenzpunkte zu geben und somit die gesamte Messung zu verbessern.

4.1.1 Messung 1

Bei dieser Messung wird ein Objekt verwendet, welches für alle Sensoren leicht erkennbar sein sollte, also nicht zu klein, aber auch nicht zu groß.

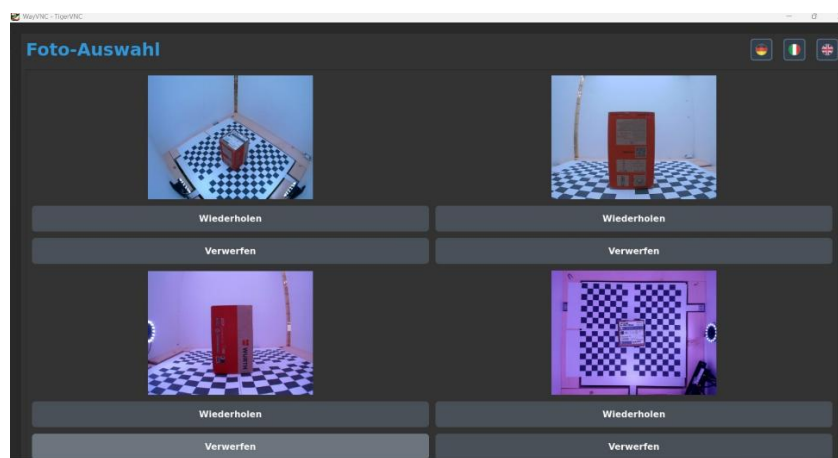


Abbildung 58: Aufnahme von Objekt 1

	Länge	Breite	Höhe	Gewicht	Barcode
Tatsächlich	90mm	90mm	155mm	757g	9 780201 379624
Gemessen	99,4mm	94,5mm	163mm	753g	9 780201 379624
Abweichung	9,4mm	4,5mm	8mm	4g	0 Stellen

4.1.2 Messung 2

Bei der zweiten Messung wird ein rundes Objekt in der gleichen Größenordnung gemessen, um das Verhalten des Scanners auf nicht rechteckige Objekte zu testen.

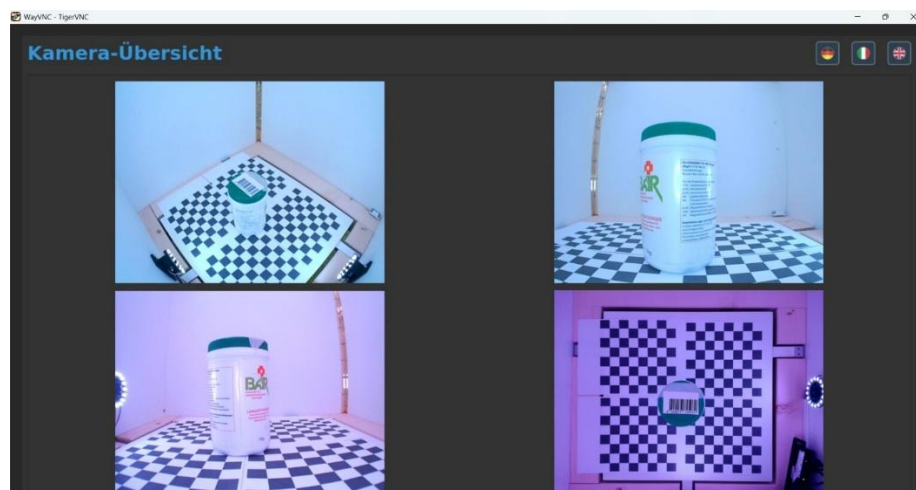


Abbildung 59: Aufnahme von Objekt 2

	Länge	Breite	Höhe	Gewicht	Barcode
Tatsächlich	90mm	90mm	170mm	772g	8 809077 745072
Gemessen	93,37mm	87,3mm	176mm	767g	8 809077 745072
Abweichung	3,37mm	2,7mm	6mm	5g	0 Stellen

4.1.3 Messung 3

Hier wurde ein Objekt gemessen, welches in allen Dimensionen kleiner ist als die vorherigen.

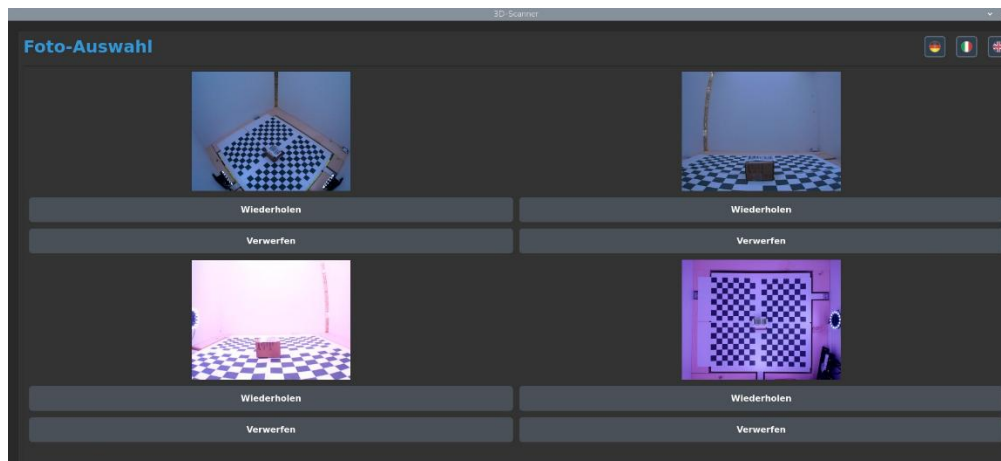


Abbildung 60: Aufnahme von Objekt 3

	Länge	Breite	Höhe	Gewicht	Barcode
Tatsächlich	60mm	60mm	40mm	79g	9 780201 338461
Gemessen	65mm	63,7mm	41mm	76,7g	9 780201 338461
Abweichung	5mm	3,7mm	1mm	2,3g	0 Stellen

Wichtig bei dieser Messung hinzuzufügen ist, dass der Barcode auf der Höhe des Objekts nicht erkannt worden ist. Das Objekt musste zuerst angehoben werden, damit der Code näher an der Tiefenkamera ist, um eine höhere Auflösung zu erzielen. Die Barcodes der Firma Leitner sind aber sehr groß und deswegen ist das bei einem realen Einsatz in der Firma ein weniger relevantes Problem.

4.1.4 Messung 4

In der letzten Messung wurde ein Objekt gemessen, welches mit der Höhe nur mehr knapp im Beobachtbaren Bereich der Tiefenkamera liegt.

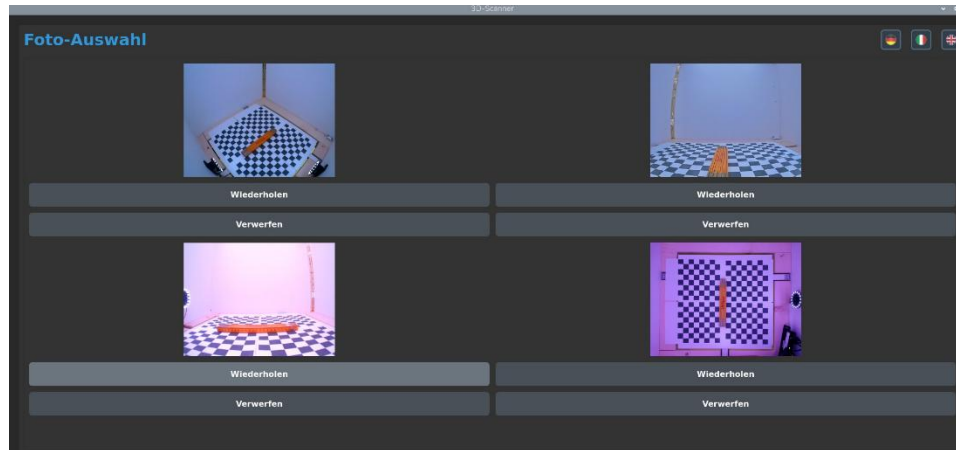


Abbildung 61: Aufnahme von Objekt 4

	Länge	Breite	Höhe	Gewicht	Barcode
Tatsächlich	235mm	35mm	15mm	94g	-
Gemessen	201mm	40,9mm	13mm	93,5g	-
Abweichung	34mm	5,9mm	2mm	0,5g	-

Aus dieser Messung geht hervor, dass Objekte ab einer gewissen Höhe nicht mehr gut von der Tiefenkamera erfasst werden können. Die Höhe wird zwar richtig gemessen, aber Teile des Objekts, welche weiter außerhalb des Sichtfensters der Kamera liegen, werden aufgrund der Disparität nicht mehr von beiden Stereokameras gesehen.

Zusätzlich führt der Kalibrierungs-Schwellwert von +1 cm dazu, dass niedrige Messobjekte aufgrund des Tiefenkamera-Rauschens ungenau erfasst werden.

4.1.5 Diskussion

Die im Rahmen der Testreihen ermittelten Messergebnisse sind als äußerst zufriedenstellend zu bewerten und liegen konsequent innerhalb der zu Projektbeginn definierten Toleranzbereiche. Besonders hervorzuheben ist die zweite Messreihe, in der erfolgreich nachgewiesen wurde, dass das System auch nicht rechteckige Geometrien zuverlässig erfassen und vermessen kann. Mit einer maximalen Abweichung von lediglich ± 1 cm bei Objekten, deren Umrisse noch korrekt erkannt werden, zeigt die optische Sensorik eine hohe Präzision. Diese entspricht den Anforderungen an ein zuverlässiges Erfassungssystem.

Auch die Integration der Wägetechnik erwies sich als hochpräzise. Die Gewichtswerte zeigten bei sämtlichen Messungen eine exzellente Reproduzierbarkeit ohne signifikante Abweichungen.

Zusätzlich konnte die Funktionalität der automatisierten Identifikation bestätigt werden: Barcodes werden vom System zuverlässig detektiert und interpretiert. Hierbei wurde jedoch festgestellt, dass die Erkennungsrate von der Dimensionierung der Barcodes abhängt, weshalb für einen optimalen Prozessdurchlauf eine entsprechende Mindestgröße der Codes eingehalten werden sollte.

4.2 Zusammenfassung der gesamten Arbeit

Zu Beginn der Diplomarbeit absolvierten beide Projektmitglieder eine gemeinsame Analyse- und Recherchephase. In dieser Phase wurden die theoretischen Grundlagen erarbeitet, geeignete Hardwarekomponenten evaluiert und deren Ansteuerung sowie die zugrunde liegenden Messprinzipien untersucht. Parallel dazu wurde ein erster Entwurf für den physikalischen Aufbau des Systems erstellt. Nach Abschluss dieser fundierten Vorbereitung erfolgte die Aufteilung in die individuellen Themenschwerpunkte.

Ein zentrales Element des Projektmanagements war die Abstimmung mit der Firma Leitner. Nach jedem Meilenstein wurden die Ergebnisse mit der zuständigen Ansprechperson überprüft, um auf Basis des erreichten Fortschritts die Zielsetzungen für die darauffolgenden Phasen anzupassen. Nach der isolierten Entwicklung der Teilbereiche folgte die Systemintegration, bei der Hardware und Software zusammengeführt wurden. Nach einer intensiven Phase der Fehlerbehebung und baulichen Optimierungen konnte das Gesamtsystem erfolgreich simuliert und dokumentiert werden.

Rückblickend nahm die Erarbeitung der Projektbasis einen deutlich größeren Zeitanteil in Anspruch als ursprünglich kalkuliert. Die hohe Anzahl an integrierten Komponenten erschwerte die präzise zeitliche Einschätzung. Insbesondere im Bereich der optischen Tiefenmessung, deren Konfiguration aufgrund komplexer Umgebungsparameter nicht unmittelbar die gewünschte Präzision erreichte.

Eine logistische Herausforderung stellte zudem die Dimension des Prototyps dar. Da die Anlage aufgrund ihrer Größe nicht mobil einsetzbar war, wurden zunächst regelmäßige Präsenztermine für die gemeinsame Arbeit am System vereinbart. Da dies durch unvorhersehbare Hürden und schulische Verpflichtungen nicht immer

realisierbar waren, wurde eine innovative alternative Lösung implementiert: Über eine VPN-Verbindung wurde ein Fernzugriff auf das System eingerichtet.

Ein zentrales Kriterium der Aufgabenstellung war die Kosteneffizienz. Mit Gesamtkosten von **ca. 910 €** konnte das Projekt nicht nur innerhalb des ursprünglich festgesetzten Budgets realisiert werden, sondern stellt auch eine wirtschaftlich hochattraktive Alternative zu kommerziellen industriellen Messsystemen dar. Dieser Erfolg wurde maßgeblich durch die strategische Auswahl von Komponenten und die Eigenfertigung wesentlicher Baugruppen ermöglicht.

Darüber hinaus fungiert die vorliegende Arbeit als technische Brücke zwischen verschiedenen Disziplinen des modernen Ingenieurwesens:

- **Informatik:** Implementierung komplexer Algorithmen zur digitalen Bildverarbeitung sowie die Integration von KI-gestützten Analysen zur Objekterkennung.
- **Elektrotechnik:** Konzeptionierung der gesamten Hardware-Architektur, Entwicklung individueller Schaltpläne und die Realisierung einer störungsfreien Systemverkabelung.
- **Maschinenbau:** Konstruktion und Fertigung eines belastbaren, funktionalen Gehäuses unter Berücksichtigung ergonomischer und thermischer Anforderungen.

Den Abschluss des Projekts bildet die offizielle Präsentation des Prototyps am Einsatzort in Sterzing. Nach einer Einweisung in die Systemfunktionalitäten wird die Anlage der Firma Leitner übergeben und für den regulären Betrieb im Bereich der Scan- und Logistikprozesse freigegeben.

4.3 Ausblick

4.3.1 Hardwareseitig

Im Rahmen weiterführender Arbeiten könnten auf der Hardwareseite mehrere Punkte auf den aktuellen Stand der Arbeit anknüpfen. Hinsichtlich der Wägetechnik würde sich das System noch viel genauer gestalten lassen. Mittels Filtersystemen, geeigneten Abschirmungen und Twisted Pair Leitungen (wie im Kapitel 2.2 Theoretische Grundlagen erklärt) würde das Rauschen auf ein Minimum reduziert werden. Da die Wägezellen ohnehin schon sehr genau sind, könnte man sie mit geeigneter Beschaltung als Präzisionswaage einsetzen.

Eine weitere Überlegung wäre die Integration eines Warnsystems, also akustische oder visuelle Signale für den Benutzer, wenn zum Beispiel die Türe während des Scanvorgangs geöffnet ist oder wenn der Scanvorgang fehlgeschlagen ist bzw. erfolgreich war.

4.3.2 Softwareseitig

Im Rahmen weiterführender Arbeiten lassen sich mehrere Ansätze identifizieren, die direkt auf dem aktuellen Entwicklungsstand aufbauen. Ein zentraler Punkt ist die Erweiterung der Messfunktionen, beispielsweise durch die Implementierung einer Point-Cloud-Messung. Dadurch könnten Objekte noch präziser dreidimensional erfasst werden, wodurch sich die Genauigkeit der geometrischen Abmessungen weiter steigern lässt.

Zusätzlich wäre die Integration eines Displays am Raspberry Pi sinnvoll, um das System unabhängig von einem Laptop betreiben zu können. Dies würde die Mobilität erhöhen und den praktischen Einsatz deutlich vereinfachen.

Darüber hinaus ist eine direkte Anbindung an ein ERP-System wie SAP über eine API denkbar. Dadurch könnte der derzeitige manuelle oder halbautomatische Datenexport ersetzt werden, was den Arbeitsprozess effizienter gestaltet, Zeit spart und potenzielle Fehlerquellen reduziert. Weitere mögliche Verbesserungen betreffen insbesondere die Qualität, Effizienz und Kostenoptimierung des Systems. Dazu zählen unter anderem die Optimierung der Bildaufnahme und Datenverarbeitung durch leistungsfähigere Algorithmen, die Erweiterung der Benutzeroberfläche um zusätzliche Kontroll- und Analysefunktionen sowie die Integration weiterer Sensoren oder Kameras zur Erfassung komplexerer Objekte.

Zudem könnten alternative Ansätze untersucht werden, um die Messergebnisse zu vergleichen. Dies ist beispielsweise durch den Einsatz unterschiedlicher Tiefenkamerasysteme oder anderer 3D-Messverfahren möglich, etwa durch eine Point-Cloud-Messung unter Verwendung fortgeschrittener Filteralgorithmen (z. B. RANSAC zur Ebenen-Erkennung). Solche Vergleiche ermöglichen eine genauere Bewertung der Stärken und Schwächen der aktuellen Lösung und unterstützen eine gezielte Weiterentwicklung.

5 Erklärung der Eigenständigkeit

EIDESSTATTLICHE ERKLÄRUNG

Ich erkläre an Eides statt, dass ich die vorliegende Arbeit selbständig und ohne fremde Hilfe verfasst, andere als die angegebenen Quellen und Hilfsmittel nicht benutzt und die den benutzten Quellen wörtlich und inhaltlich entnommenen Stellen als solche erkenntlich gemacht habe. Meine Arbeit darf öffentlich zugänglich gemacht werden, wenn kein Sperrvermerk vorliegt.

Ort, Datum

Schiechtl Alexander

Ort, Datum

Granegger Felix

6 Arbeitsnachweise

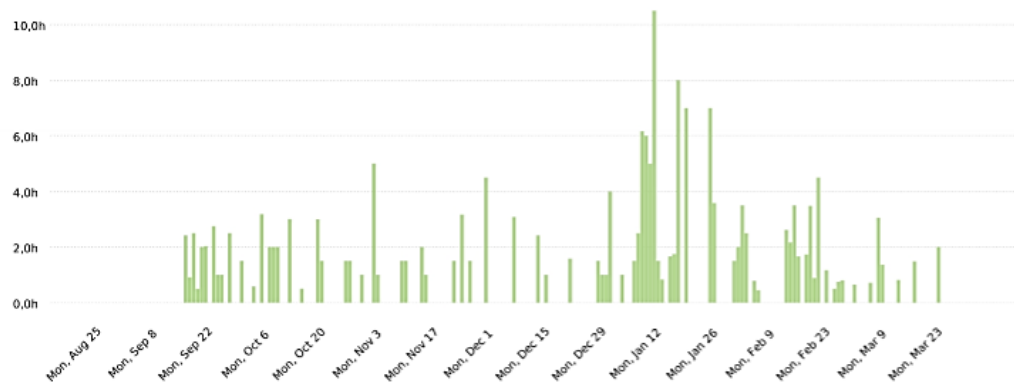
6.1 Schiechl Alexander

Summary report

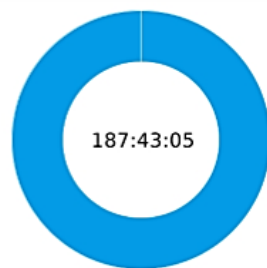
25/08/2025 - 31/03/2026



Total: 187:43:05 Billable: 00:00:00 Amount: 0,00 USD

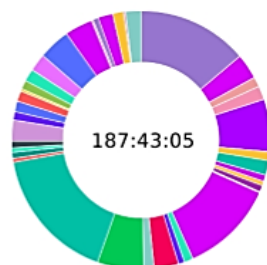


Project



Project	Time	Percentage
3D Scanner	187:43:05	100,00%

Description



Description	Time	Percentage
3D Design	26:00:00	13,85%
3D Druck	05:55:00	3,15%
ABA Portal	02:20:00	1,24%
Altium Library	03:10:57	1,70%
Altium	12:15:00	6,53%
Analyse	02:00:00	1,07%

● Bauteile	03:30:00	1,86%
● Bauteilliste/Schnittstellen	01:30:00	0,80%
● Bestellungen	01:00:00	0,53%
● Datenblätter	01:30:00	0,80%
● Dokumente Finalisierung	00:19:59	0,18%
● Doku	22:05:00	11,76%
● Einkäufe	02:00:00	1,07%
● Einrichtung VPN am Raspi	01:30:00	0,80%
● Firmware	05:40:00	3,02%
● Gehäuse Planung	02:30:00	1,33%
● Gehäuse	10:47:00	5,74%
● Gemeinsame Tests	31:30:00	16,78%
● Hardware Inspektion	01:00:00	0,53%
● Kleinarbeiten	01:25:00	0,75%
● Meeting	01:00:00	0,53%
● Meilenstein Meeting	01:30:00	0,80%
● PCB Design	05:00:00	2,66%
● PCB V1	01:55:00	1,02%
● PCB V2	02:28:52	1,32%
● Projekt Definition	02:30:00	1,33%
● Projektmanagement	00:34:51	0,31%
● Raspberry PI	02:00:00	1,07%
● Recherche	03:00:00	1,60%
● Research	04:02:00	2,15%
● Testaufbau	07:30:26	4,00%
● Tests	06:25:00	3,42%
● Tiefenkamera Test	00:30:00	0,27%
● Troubleshooting Hardware	01:17:00	0,68%

● VPN Troubleshooting	03:27:00	1,84%
● Verkabelung	02:30:00	1,33%
● Virtualisierung Zeitauffassung	00:30:00	0,27%
● Waage Kalibrierung	03:35:00	1,91%

Project / Description	Duration	Amount
3D Scanner	187:43:05	0,00 USD
3D Design	26:00:00	0,00 USD
3D Druck	05:55:00	0,00 USD
ABA Portal	02:20:00	0,00 USD
Altium Library	03:10:57	0,00 USD
Altium	12:15:00	0,00 USD
Analyse	02:00:00	0,00 USD
Bauteile	03:30:00	0,00 USD
Bauteilliste/Schnittstellen	01:30:00	0,00 USD
Bestellungen	01:00:00	0,00 USD
Datenblätter	01:30:00	0,00 USD
Dokumente Finalisierung	00:19:59	0,00 USD
Doku	22:05:00	0,00 USD
Einkäufe	02:00:00	0,00 USD
Einrichtung VPN am Raspi	01:30:00	0,00 USD
Firmware	05:40:00	0,00 USD
Gehäuse Planung	02:30:00	0,00 USD
Gehäuse	10:47:00	0,00 USD
Gemeinsame Tests	31:30:00	0,00 USD
Hardware Inspektion	01:00:00	0,00 USD

Kleinarbeiten	01:25:00	0,00 USD
Meeting	01:00:00	0,00 USD
Meilenstein Meeting	01:30:00	0,00 USD
PCB Design	05:00:00	0,00 USD
PCB V1	01:55:00	0,00 USD
PCB V2	02:28:52	0,00 USD
Projekt Definition	02:30:00	0,00 USD
Projektmanagement	00:34:51	0,00 USD
Raspberry PI	02:00:00	0,00 USD
Recherche	03:00:00	0,00 USD
Research	04:02:00	0,00 USD
Testaufbau	07:30:26	0,00 USD
Tests	06:25:00	0,00 USD
Tiefenkamera Test	00:30:00	0,00 USD
Troubleshooting Hardware	01:17:00	0,00 USD
VPN Troubleshooting	03:27:00	0,00 USD
Verkabelung	02:30:00	0,00 USD
Virtualisierung Zeitauffassung	00:30:00	0,00 USD
Waage Kalibrierung	03:35:00	0,00 USD

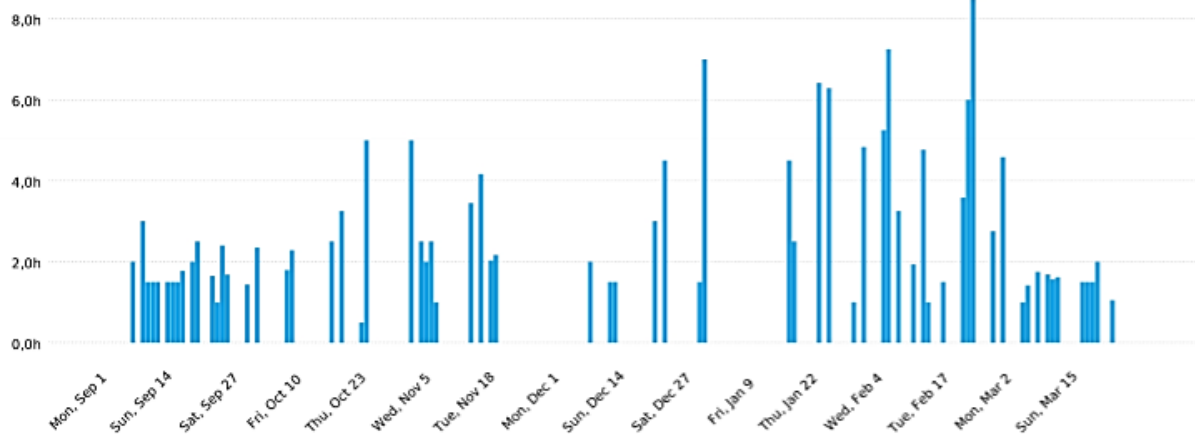
6.2 Granegger Felix

Summary report

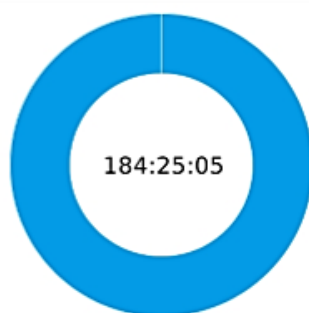
01/09/2025 - 27/03/2026



Total: 184:25:05 Billable: 00:00:00 Amount: 0,00 USD

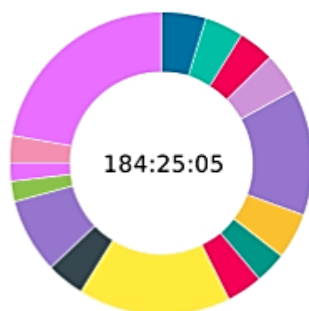


Project



• 3D Scanner 184:25:05 100,00%

Description



• Barcode Erkennung auf einem Bild	08:45:00	4,74%
• Beleuchtung - Ansteuerung	07:42:00	4,18%
• Bildverarbeitung / Auflösung	07:01:00	3,80%
• Datenmanagement	07:50:00	4,25%
• Dokumentation	25:06:00	13,61%
• Flow Chart / Softwareplan	09:04:51	4,92%

● Gewichtsmessung	06:10:00	3,34%
● GitHub aufräumen und Code Struktur bearbeiten	07:00:00	3,80%
● Graphisches User Interface	30:01:14	16,28%
● Organisatorisches	07:30:00	4,07%
● Prototyp Testen Gemeinsam	14:39:00	7,94%
● Raspberry Pi	04:00:00	2,17%
● Research	03:30:00	1,90%
● Software Zusammenfügen	05:30:00	2,98%
● Tiefenkamera	40:36:00	22,02%

Project / Description	Duration	Amount
3D Scanner	184:25:05	0,00 USD
Barcode Erkennung auf einem Bild	08:45:00	0,00 USD
Beleuchtung - Ansteuerung	07:42:00	0,00 USD
Bildverarbeitung / Auflösung	07:01:00	0,00 USD
Datenmanagement	07:50:00	0,00 USD
Dokumentation	25:06:00	0,00 USD
Flow Chart / Softwareplan	09:04:51	0,00 USD
Gewichtsmessung	06:10:00	0,00 USD
GitHub aufräumen und Code Struktur bearbeiten	07:00:00	0,00 USD
Graphisches User Interface	30:01:14	0,00 USD
Organisatorisches	07:30:00	0,00 USD
Prototyp Testen Gemeinsam	14:39:00	0,00 USD
Raspberry Pi	04:00:00	0,00 USD
Research	03:30:00	0,00 USD
Software Zusammenfügen	05:30:00	0,00 USD
Tiefenkamera	40:36:00	0,00 USD

7 Verzeichnisse

7.1 Quellenverzeichnis

- [1] „Farnell® UK“, Farnell® UK. Zugriffen: 23. März 2026. [Online]. Verfügbar unter: <https://uk.farnell.com/>
- [2] „Raspberry Pi“, *Wikipedia*. 12. März 2026. Zugriffen: 16. März 2026. [Online]. Verfügbar unter: https://de.wikipedia.org/w/index.php?title=Raspberry_Pi&oldid=265161222
- [3] „Microcontroller“, *Wikipedia*. 12. März 2026. Zugriffen: 16. März 2026. [Online]. Verfügbar unter: <https://en.wikipedia.org/w/index.php?title=Microcontroller&oldid=1343067897>
- [4] „microSD“, *Wikipedia*. 29. September 2025. Zugriffen: 16. März 2026. [Online]. Verfügbar unter: <https://de.wikipedia.org/w/index.php?title=MicroSD&oldid=260158380>
- [5] „Ultrasonic Distance Sensors – Definition, Working, Accuracy, Range & Best Models“, Ersae Electronics. Zugriffen: 14. Februar 2026. [Online]. Verfügbar unter: <https://www.ersaelectronics.com/blog/ultrasonic-distance-sensor>
- [6] „microsonic | ultrasonic sensors | Made in Germany“. Zugriffen: 23. März 2026. [Online]. Verfügbar unter: <https://www.microsonic.de/>
- [7] „Funktionsweise von Laser-Distanzsensoren“. Zugriffen: 14. Februar 2026. [Online]. Verfügbar unter: https://www.baumer.com/de/de/a/Know-how_Function_optical-distance-sensors
- [8] „ResearchGate | Find and share research“, ResearchGate. Zugriffen: 23. März 2026. [Online]. Verfügbar unter: <https://www.researchgate.net/>
- [9] „Lidar“, *Wikipedia*. 16. Dezember 2025. Zugriffen: 14. Februar 2026. [Online]. Verfügbar unter: <https://de.wikipedia.org/w/index.php?title=Lidar&oldid=262470045>
- [10] „How a Load Cell Works | Types of Load Cells“. Zugriffen: 13. November 2025. [Online]. Verfügbar unter: <https://www.800loadcel.com/white-papers/how-a-load-cell-works.html>
- [11] „Load Cell Summing: Junction Boxes, Signal Trim, and Excitation Trim“, Tacuna Systems. Zugriffen: 13. November 2025. [Online]. Verfügbar unter: <https://tacunasystems.com/knowledge-base/load-sell-summing-junction-boxes-signal-trim-and-excitation-trim/>
- [12] „RGB-LED“. Zugriffen: 22. Februar 2026. [Online]. Verfügbar unter: https://www.elektronik-kompendium.de/sites/praxis/bauteil_rgb-led.htm

- [13] „WS2812 Ansteuerung – Mikrocontroller.net“. Zugriffen: 22. Februar 2026. [Online]. Verfügbar unter: https://www.mikrocontroller.net/articles/WS2812_Ansteuerung
- [14] „Kabelquerschnitt berechnen » Formel & Infos zur Berechnung“. Zugriffen: 12. Februar 2026. [Online]. Verfügbar unter: <https://www.conrad.de/de/ratgeber/handwerk/kabelquerschnitt-berechnen.html>
- [15] „Abschirmung (Elektrotechnik)“, *Wikipedia*. 10. Juni 2022. Zugriffen: 1. März 2026. [Online]. Verfügbar unter: [https://de.wikipedia.org/w/index.php?title=Abschirmung_\(Elektrotechnik\)&oldid=23598522](https://de.wikipedia.org/w/index.php?title=Abschirmung_(Elektrotechnik)&oldid=23598522)
- [16] „Stecker Express: Großhandel für elektrotechn. & mechanische Produkte“, Stecker Express Online Shop. Zugriffen: 23. März 2026. [Online]. Verfügbar unter: <https://stex24.com/>
- [17] „What are Differential Pairs and Why Do We Use Them!“, DigiKey. Zugriffen: 1. März 2026. [Online]. Verfügbar unter: <https://www.digikey.at/en/maker/tutorials/2024/what-are-differential-pairs-and-why-do-we-use-them>
- [18] „Wikimedia Commons“. Zugriffen: 23. März 2026. [Online]. Verfügbar unter: https://commons.wikimedia.org/wiki/Main_Page
- [19] „Twisted-Pair-Kabel“. Zugriffen: 1. März 2026. [Online]. Verfügbar unter: <https://www.elektronik-kompodium.de/sites/net/0603191.htm>
- [20] A. aka, „Answer to ‚Should I always put a low-pass filter on an ADC input?‘“, Electrical Engineering Stack Exchange. Zugriffen: 21. März 2026. [Online]. Verfügbar unter: <https://electronics.stackexchange.com/a/157987>
- [21] „AN-1364: Differential Filter Design for a Receive Chain in Communication Systems | Analog Devices“. Zugriffen: 1. März 2026. [Online]. Verfügbar unter: <https://www.analog.com/en/resources/app-notes/an-1364.html>
- [22] „USB“, *Wikipedia*. 22. März 2026. Zugriffen: 23. März 2026. [Online]. Verfügbar unter: <https://en.wikipedia.org/w/index.php?title=USB&oldid=1344690333>
- [23] „Ethernet: Unterschied zu LAN & WLAN, Funktionsweise & Setup“. Zugriffen: 23. März 2026. [Online]. Verfügbar unter: <https://www.conrad.at/de/ratgeber/office-networking/kommunikationsstandards/ethernet.html>
- [24] „Camera Serial Interface“, *Wikipedia*. 23. Februar 2026. Zugriffen: 23. März 2026. [Online]. Verfügbar unter: https://en.wikipedia.org/w/index.php?title=Camera_Serial_Interface&oldid=1340081838

- [25] G. Klauser, „FDM vs. SLA 3D-Druckverfahren | Kosten / Qualität / Anwendung“. Zugriffen: 22. Februar 2026. [Online]. Verfügbar unter: <https://3dee.at/fdm-vs-sla-3d-druck/>
- [26] „Was ist eine STL-Datei? | 3D Systems“. Zugriffen: 22. Februar 2026. [Online]. Verfügbar unter: <https://de.3dsystems.com/quickparts/learning-center/what-is-stl-file>
- [27] „PRINCE2® Projektmanagement Schulung | DE“. Zugriffen: 24. Oktober 2025. [Online]. Verfügbar unter: <https://www.prince2.com/de>
- [28] „Datenblatt Raspberry Pi 5“.
- [29] „Home“, Sony DepthSensing Solutions. Zugriffen: 24. Oktober 2025. [Online]. Verfügbar unter: <https://www.sony-depthsensing.com/>
- [30] „OAK-D S2“, Luxonis. Zugriffen: 22. Dezember 2025. [Online]. Verfügbar unter: <https://docs.luxonis.com/hardware/products/OAK-D%20S2>
- [31] „Multi Camera Adapter Module V2.2 for Raspberry Pi Camera Module 3 12MP IMX708 / 5MP OV5647 / 8MP IMX219 / 12MP IMX477 Cameras“. Zugriffen: 8. März 2026. [Online]. Verfügbar unter: <https://www.arducam.com/multi-camera-v2-1-adapter-raspberry-pi.html>
- [32] „Datenblatt FIT0892“.
- [33] „Kern CP 20-2Y1 Wägezelle kaufen“. Zugriffen: 8. März 2026. [Online]. Verfügbar unter: <https://www.conrad.at/de/p/kern-cp-20-2y1-waegezelle-2340059.html>
- [34] „Datenblatt Adafruit-NAU7802“.
- [35] „Versorgung RD-125“.
- [36] „Wemos Lolin D1 mini v4.0 ESP8266“, arduino-projekte.info. Zugriffen: 23. März 2026. [Online]. Verfügbar unter: <https://arduino-projekte.info/products/wemos-lolin-d1-mini-v4-0-esp8266>
- [37] „FastLED: FastLED - The Universal LED Library“. Zugriffen: 7. Januar 2026. [Online]. Verfügbar unter: <https://fastled.io/docs/>
- [38] „Creating a Schematic Symbol“, Altium Documentation. Zugriffen: 8. März 2026. [Online]. Verfügbar unter: <https://www.altium.com/documentation/altium-designer/components-libraries/creating-schematic-symbol>
- [39] „Configuring Stereo Depth“, Luxonis. Zugriffen: 31. Januar 2026. [Online]. Verfügbar unter: <https://docs.luxonis.com>
- [40] „RGBD Point Cloud Visualization“, Luxonis. Zugriffen: 22. Dezember 2025. [Online]. Verfügbar unter: <https://docs.luxonis.com>
- [41] „Piero2411/YOLOV8s-Barcode-Detection at main“. Zugriffen: 18. Oktober 2025. [Online]. Verfügbar unter: <https://huggingface.co/Piero2411/YOLOV8s-Barcode-Detection/tree/main>

- [42] F. Zyprian, „ZBar: Document AI - Effiziente Extraktion von Barcodes“, Konfuzio. Zugriffen: 18. Oktober 2025. [Online]. Verfügbar unter: <https://konfuzio.com/de/zbar-python-barcode-decoder/>
- [43] „PyQt6 Widgets — QCheckBox, QComboBox, QPushButton, QLabel, QSlider“, Python GUIs. Zugriffen: 8. September 2025. [Online]. Verfügbar unter: <https://www.pythonguis.com/tutorials/pyqt6-widgets/>
- [44] R. Python, „Use PyQt’s QThread to Prevent Freezing GUIs – Real Python“. Zugriffen: 16. September 2025. [Online]. Verfügbar unter: <https://realpython.com/python-pyqt-qthread/>
- [45] „Stereo Depth“, Luxonis. Zugriffen: 22. Dezember 2025. [Online]. Verfügbar unter: <https://docs.luxonis.com>
- [46] „InventBar and ParcelBar: BenchmarkBarcodeDatasets“. Zugriffen: 18. Oktober 2025. [Online]. Verfügbar unter: <https://www.kaggle.com/datasets/chommaphatma/inventbar-and-parcelbar-benchmarkbarcodedatasets>
- [47] „Datenblatt TC9548A MUX“.
- [48] „The Python Standard Library — Python 3.10.20 documentation“. Zugriffen: 9. September 2025. [Online]. Verfügbar unter: <https://docs.python.org/3.10/library/index.html?>
- [49] donaldtrumpiscute, „How do you actually open (not read) a pdf file? os.system(fp)?“, r/learnpython. Zugriffen: 23. März 2026. [Online]. Verfügbar unter: https://www.reddit.com/r/learnpython/comments/1cb956r/how_do_you_actually_open_not_read_a_pdf_file/
- [50] „VNC Viewer von RealVNC® herunterladen“, RealVNC®. Zugriffen: 21. Dezember 2025. [Online]. Verfügbar unter: <https://www.realvnc.com/de/connect/download/viewer/>

7.2 Abkürzungsverzeichnis

AG	Aktiengesellschaft
AWG	American Wire Gauge
CAD	Computer-Aided Design
CLI	Command Line Interface
COM	Communications Port
CPU	Central Processing Unit
CSI	Camera Serial Interface
CSV	Comma Separated Values
DC	Direct Current
DHCP	Dynamic Host Configuration Protocol
DRDY	Data Ready
DSI	Display Serial Interface
FDM	Fused Deposition Modeling
GPIO	General Purpose Input/Output
GUI	Graphical User Interface
I/O	Input/Output
IC	Integrated Circuit
IP	Internet Protocol
JSON	JavaScript Object Notation
KI	Künstliche Intelligenz
LAN	Local Area Network
LED	Light Emitting Diode
LIDAR	Light imaging, detection and ranging
MDF	Mitteldichte Holzfaserplatte
MOSFET	Metal-Oxide-Semiconductor Field-Effect Transistor
PRINCE2	PROjects IN Controlled Environments v2
PWM	Pulsweitenmodulation
QR	Quick Response
RGB	Red, Green, Blue
ROI	Region of Interest
RxD	Receive Data
SD-Karte	Secure Digital Memory Card
SLA	Stereolithography Apparatus
SoC	System on a Chip
SPI	Serial Peripheral Interface
SSH	Secure Shell
STL	Standard Tessellation Language

ToF.....	<i>Time of Flight</i>
TxD.....	<i>Transmit Data</i>
UPC.....	<i>Universal Product Code</i>
UV	<i>Ultraviolett</i>
VPN.....	<i>Virtual Private Network</i>
YOLO	<i>You Only Look Once</i>

7.3 Abbildungen

Abbildung 1: Blockschaltbild.....	6
Abbildung 2: Raspberry Pi [1].....	7
Abbildung 3: MicroSD [4].....	8
Abbildung 4: Ultraschallsensor [6].....	9
Abbildung 5: ToF Kamera [8]	10
Abbildung 6: Wägezelle Aufbau [10]	11
Abbildung 7: Wheatstone'sche Messbrücke [10].....	11
Abbildung 8: Summierkörper [11].....	13
Abbildung 9: RGB-LED	13
Abbildung 10: Abschirmung eines Kabels [16].....	15
Abbildung 11: Differentieller Signalverlauf [18].....	16
Abbildung 12: Differenzieller Tiefpass [20].....	17
Abbildung 13: USB-A 2.0 [22]	18
Abbildung 14: USB-A 3.0 [22]	18
Abbildung 15: USB-B [22]	18
Abbildung 16: USB-C [22]	18
Abbildung 17: CSI Anschluss Block [24]	19
Abbildung 18: Phasenübergänge / Project Initiation.....	23
Abbildung 19: OAK-D S2 AF [30]	26
Abbildung 20: FIT0892	28
Abbildung 21: D1 Mini - Pinout [36].....	32
Abbildung 22: MOSFET Konzept Schaltung	35
Abbildung 23: Symbol Wizard	37
Abbildung 24: Electrical Types	38
Abbildung 25: Schraubklemmen	38
Abbildung 26: USB-A Anschluss	39

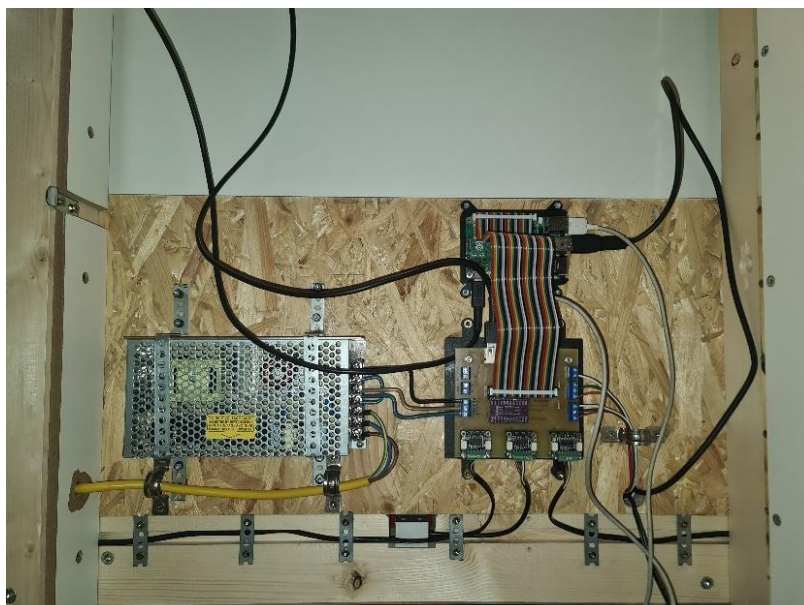
Abbildung 27: 40 Pin-Header	39
Abbildung 28: 40 Pin-Kabel	39
Abbildung 29: Optionaler Button	40
Abbildung 30: Kamera Gehäuse	41
Abbildung 31: Isometrische Ansicht	42
Abbildung 32: Skizze der Kamera Brennweite	44
Abbildung 33: Skizze Gehäuse Maße	45
Abbildung 34: Unterbau.....	46
Abbildung 35: Wägeplatte	47
Abbildung 36: Wägezelle.....	47
Abbildung 37: Verkleidung Rückseite.....	48
Abbildung 38: Querschnitt-Tabelle	49
Abbildung 39: Verkabelung Platine	51
Abbildung 40: OAK-D S2 AF [30]	57
Abbildung 41: Limitierung der Distanzmessung von Luxonis Dokumentation [39] ...	59
Abbildung 42: Verarbeitetes Originalbild	63
Abbildung 43: Verarbeitetes CLAHE-Bild	63
Abbildung 44: Verarbeitetes Unsharp-Bild	63
Abbildung 45: Verarbeitetes Invert-Bild	63
Abbildung 46: JET – Farbskala mit minimal Distanzmessung vom ROI.....	73
Abbildung 47: JET – Farbskala mit unbekannten Punkten im ROI.....	74
Abbildung 48: Tiefenwerte nach Objekt Filterung.....	74
Abbildung 49: Flussdiagramm der distanz_measure03.py (eigene Darstellung).....	76
Abbildung 50: Ergebnis von der YOLO-Erkennung	77
Abbildung 51: Foto mit Barcode für die YOLO-Erkennung [46].....	78
Abbildung 52: Foto mit Barcode für die Dekodierung [46].....	78
Abbildung 53: Ergebnis von der Barcode-Dekodierung.....	79
Abbildung 54: Flussdiagramm der BarCode_v03.py (eigene Darstellung)	82
Abbildung 55: Flussdiagramm der Gewichts_Messung02.py (eigene Darstellung)..	86
Abbildung 56: Auszug von der csv Datei für den SAP-Import	88
Abbildung 57: Ordnerstruktur (eigene Darstellung)	88
Abbildung 58: Aufnahme von Objekt 1	100
Abbildung 59: Aufnahme von Objekt 2	101
Abbildung 60: Aufnahme von Objekt 3	102
Abbildung 61: Aufnahme von Objekt 4	103

8 Anhang

Innenansicht



Installationsebene



Risikotabelle

Thema	Beschreibung	Priorität	Eintrittswahrscheinlichkeit	Auswirkung	Risikoantwort?
Raspberry Pi	Es sind keine 4 Schnittstellen für die RPI CAMs vorhanden. Es kann dafür ein zweiter Raspberry Pi verwendet werden.	hoch	hoch	kleiner Kostenanstieg und Kommunikation zwischen einander. Zusätzlicher Programmieraufwand, nicht abschätzbar (>8h)	eine andere Vorgehensweise wird gewählt
Raspberry Pi	Es sind keine 4 Schnittstellen für die RPI CAMs vorhanden. Es kann ein Modul Board für 4 Schnittstellen verwendet werden.	hoch	mittel	Mittlerer Kostenanstieg, aber bei der Kommunikation muss nichts extra gemacht werden. Zusätzlicher Programmieraufwand, nicht abschätzbar (>8h)	eine andere Vorgehensweise wird gewählt
Distanzmessung	Länge x Breite über Pixel erfassen, zu ungenau (+-2cm)	sehr hoch	mittel	Lagerplatz würde im SAP zu groß eingetragen werden	Eine sehr genaue Tiefenkamera wurde gewählt, welche manuell auf die Anwendung kalibriert werden kann,
Wasser Affinität	Flüssigkeit kann in den Scanner eintreten	sehr niedrig	sehr niedrig	Möglicher Scanner defekt	wird ignoriert
Zusatz Sensorik	Warnanlagen für den Benutzer	mittel	mittel	Benutzer verfälscht das Ergebnis nicht	Könnte über Blinken der Beleuchtung gemacht werden
Interface	Zusätzliche grafisches Display	niedrig	niedrig	Benutzerfreundlicher, Laptop nicht zwingend notwendig für die Anzeige	wird nicht weiter verfolgt
Raspberry Pi	Betriebssystem auf einer SSD NVMe laufen lassen, da eine Micro SD schnell kaputt gehen kann	mittel	hoch	Scanner würde viel länger als mit der Micro SD funktionieren. Eine Adapterplatine wäre nötig	Es wird stattdessen eine SD-Card mit Industrial Grade verwendet

Gewichtsmessung	Federn für die Gewichtsmessung	sehr hoch	sehr niedrig	mehr Fehler, schwerere Befestigung, Komplexere Verkabelung	Idee wird nicht weiter verfolgt
Gewichtsmessung	Wägezellen für die Gewichtsmessung	sehr hoch	hoch	Qualitativ hochwertiger, teurer	Wägezellen werden verwendet
Gewichtsmessung	Softwaremäßige Kalibrierung und Aufsummierung	hoch	hoch	einfacher als Hardwaretechnisch, weniger Fehleranfällig	Software-Kalibrierung
Gewichtsmessung	Hardwaremäßige Kalibrierung und Aufsummierung	hoch	niedrig	einfachere Umsetzung	Software-Kalibrierung
Raspberry Pi	Es sind keine 4 Schnittstellen für die RPI CAMs vorhanden. Es werden daher nur 2 RPI Cams und zwei USB CAMs verwendet	hoch	sehr hoch	keine zusätzliche Schnittstelle zwischen Raspberry Pis nötig, keine teurere Adapterplatine. Zusätzlicher Programmieraufwand, < 8h	andere Vorgehensweise
Datenbank	Sehr kleine Bauteile (kein Gewicht und fast keine Maße)	hoch	unausweichlich	Fehlende Daten für die Datenbank	Es werden manuelle Eingabefelder hinzugefügt. Das weitere vorgehen wird der Firma selbst überlassen
Barcode	Kein EAN-Code vorhanden	mittel	sehr hoch	Fehlende Daten für die Datenbank	Code wird manuell nachgetragen oder alternativ ein anderer Barcode-Typ (z.B. Code128) verwendet.
Raspberry Pi	Es sind keine 4 Schnittstellen für die RPI CAMs vorhanden. Es werden daher 4 USB CAMs verwendet	hoch	sehr hoch	keine zusätzliche Schnittstelle zwischen Raspberry Pis nötig, keine teurere Adapterplatine. Zusätzlicher Programmieraufwand, < 6h	wird so umgesetzt

